dsh-superagent 多通道

中文文档合集

版本 2.4.0 · 2026-11-14 · MIT

本合集由仓库 docs/ 下的中文文档生成；截图敏感信息已打码。

目录

项目总览

文档总览

000 设计哲学

001 领域模型

002 授权与准入

003 执行

004 提示词

005 面板

006 部署

007 架构

008 常见做法与排障

扩展：多通道

能力矩阵

ADR-005 微信后端

ADR-006 健康条目键兼容

ADR-007 微信能力对齐

ADR-008 出站附件

ADR-009 Projex 反馈与对齐

ADR-010 云效登录与状态

ADR-011 微信对齐飞书

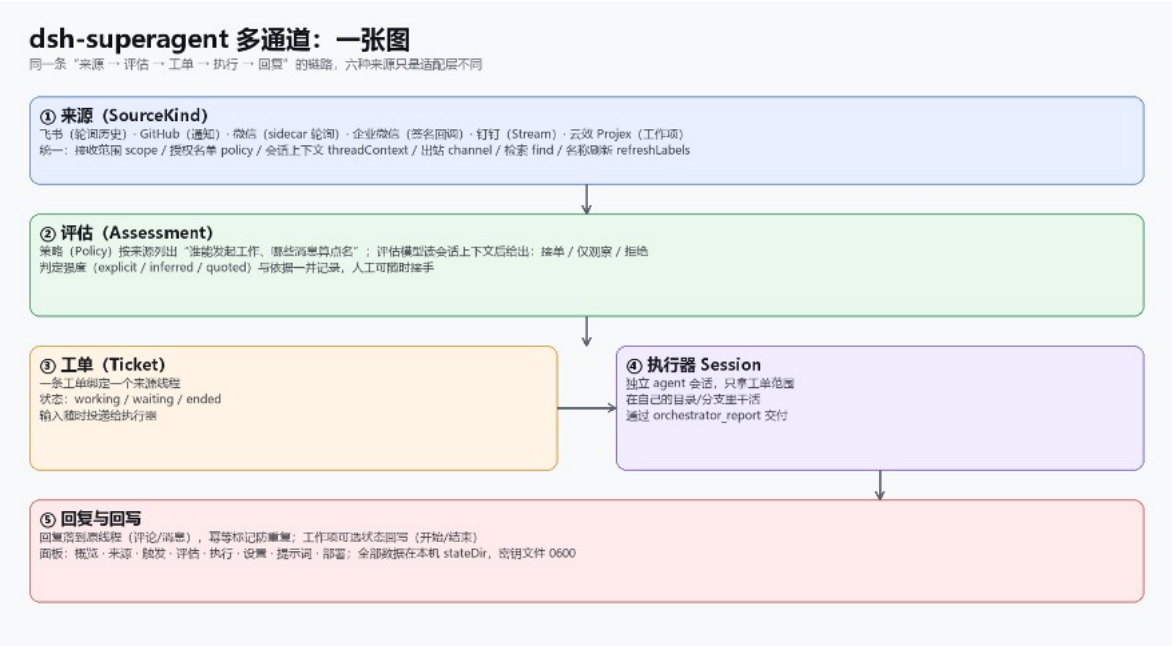
ADR-012 企业微信对齐飞书

dsh-superagent 多通道

把飞书、GitHub、微信、企业微信、钉钉、阿里云云效 Projex 接成同一套「评估 → 工单 → 执行 → 回复」闭环的 DSH 插件。

版本 `2.4.0` · 测试 102 文件 / 1624 通过 · 许可 MIT · 与官方 `dsh-superagent` 并存安装

> *本仓库是官方开源 superagent 的扩展：官方两种来源（飞书、GitHub）保留原样，**新增**微信个人号、企业微信、钉钉与云效 Projex，并把它们对齐到同一套能力面（接收范围、授权名单、会话上下文、worker 读原文、检索、名称刷新、回复与回写）。逐项对照见 能力矩阵。*



架构：一条链路，六种来源

一句话

有人在某个渠道里 @ 机器人或私聊它 → 插件按策略读上下文、评估要不要接 → 接了就开一个**独立执行器会话**在隔离目录里干活 → 结果回到**原来那条线程**，工作项还能顺手把看板状态往前推。

一条消息的一生



生命周期

- **触发**：六个来源都产出同一种 `Trigger`（带线程、发起人、结构化 @、原始 ID）。
- **评估**：策略按来源配置「谁能发起工作」「哪些消息算点名」；评估模型读会话上下文，输出 接单 / 仅观察 / 拒绝，并记录判定强度与依据。
- **工单**：一条工单绑定一个来源线程，后续消息作为输入投递给执行器。
- **执行**：执行器 Session 只拿工单范围，在自己的目录/分支里工作，结构化 `orchestrator_report` 交付。
- **回复与回写**：回复落到原线程（评论/消息，带幂等标记），工作项可选状态回写（开始/结束）。
- 全程写入本机 `stateDir`，重启后继续；人工可在任何一步接手。

三个真实场景

1. **云效缺陷单**：有人在需求/缺陷里 @ 机器人「登录偶发 502」→ 评估读工作项与评论 → 建工单 → 执行器在隔离目录修 →

评论里交付，看板状态自动从「处理中」走到「待测试」。

2. **微信群运维**：群里 @ 机器人「帮忙看下证书快到期」→ 只在这一条线程里回应，不给无关的人发消息。
3. **企业微信私聊**：产品私聊「这个文案改一下」→ 回到同一会话，改完的长文分段发送，需要时把文件作为评论/链接交付。

凭什么变成工作?



准入与评估

四道闸门（启用的来源 → 授权名单 → 结构化点名 → 评估）任何一道不通过，消息只进入线程上下文供人查看，**不会建工单、不会回复、不会改远程状态。**

功能清单

环节	已经有
接收	六种来源统一成 Trigger；群/私聊；结构化 @；可选图片；按平台消息 ID 去重
授权	每个来源独立的「能发起工作的人」名单 + 准入表（名单内外 × @/未 @/私聊）；人工可随时收紧
评估	规则开关、接单模式（关闭/仅观察/自动）、单次时限、模型与推理强度；每次记录判定、强度、依据
工单	一工单一线程；输入随时投递；可中断、可补充、可人工接手
执行	独立执行器会话与目录/分支；结构化报告；文件可回传
回复	回到原线程；评论幂等标记；工作项可选状态回写；失败重试可见
读原文	飞书/GitHub/微信/企业微信/钉钉/云效 各自的 worker 资源（只读）
面板	概览/来源/触发/评估/执行/设置/提示词/部署；提示词与六个来源技能可覆盖
安全	本机状态、0600 凭据、只被动回复、平台限制公开写明

六种来源，一套面板



六种来源

来源	接收方式	关键能力
飞书（官方）	轮询历史	话题/回复链、AppLink 深链、图片、worker 读 `feishu//…`
GitHub（官方）	通知轮询	Issue/PR 上下文、`gh` 读代码、PR/Issue 链接

来源	接收方式	关键能力
微信（新增）	sidecar 轮询	结构化 @、语音转写、图片可下载、worker 读 `weixin//message·thread·around`、重挂补读
企业微信（新增）	签名回调（插件自挂 `/wecom/`）	@ 判定、入站图片、通讯录名单、worker 读 `wecom//message`
钉钉（新增）	Stream（部署注入 SDK 连接）	staffId 结构化 @、Markdown 5000、worker 读 `dingtalk//message`
云效 Projex（新增）	工作项轮询	需求/缺陷/任务+评论、状态回写、附件评论、worker 读 `projex//item·comments`

面板导览

插件在侧边栏有独立入口「Superagent 多通道」，与官方入口互不覆盖。**以下截图为实机截图，项目 ID、令牌路径、组织 ID、姓名、群名等敏感信息均已打码。**

评估与策略

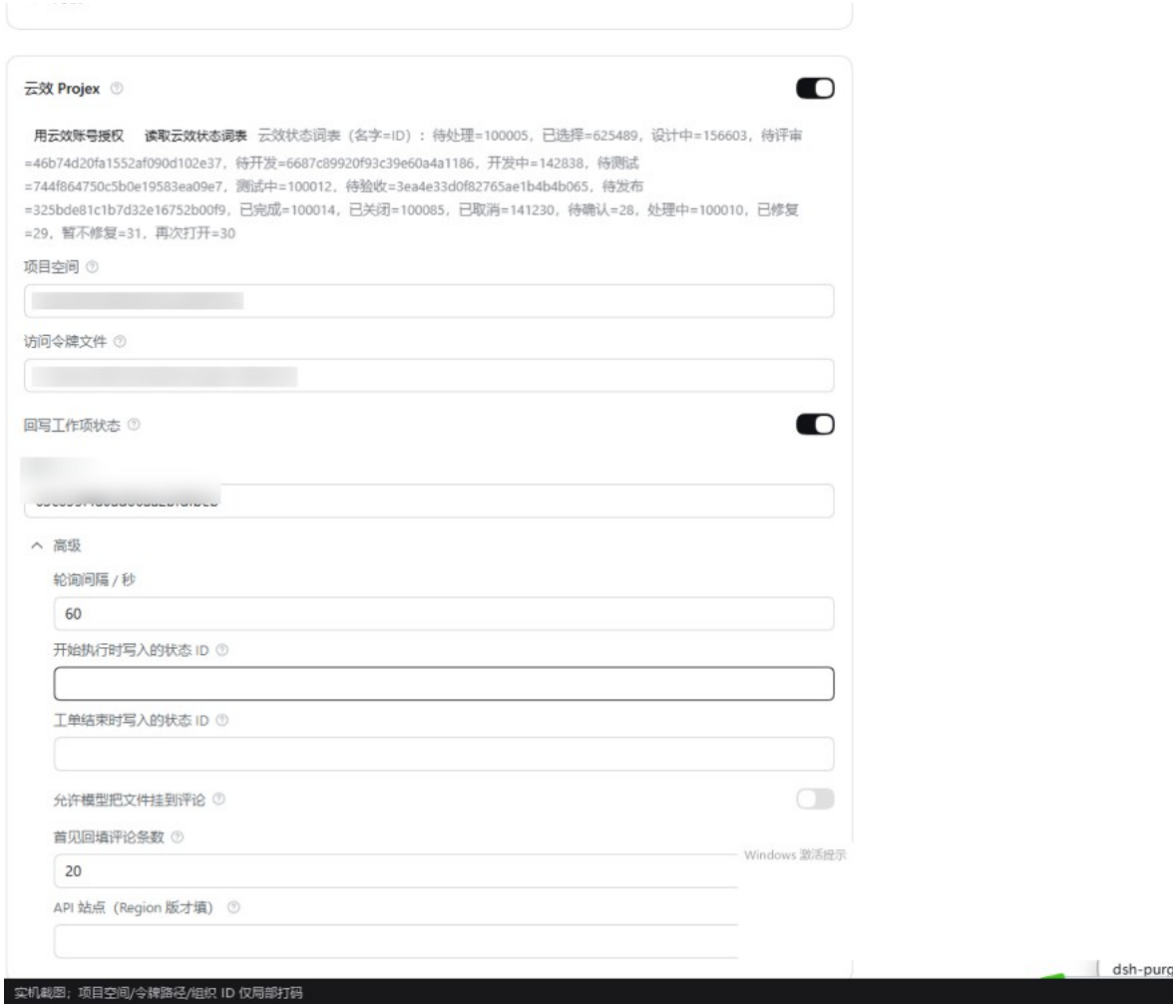
按来源列出「谁能发起工作」，并把准入表按行展示：名单内/名单外 × 群里 @/未 @/私聊，每一格都能改。



评估与策略

来源卡片（以云效 Projex 为例）

授权、读取状态词表、项目空间、令牌文件、状态回写开关、轮询间隔，两个回写状态是**下拉选择**（显示看板名字，存状态 ID）。



云效 Projex 来源卡片

提示词

主 Session 与执行器 Session 的每段文本都可查看与覆盖；六个来源技能（飞书、GitHub、微信、云效 Projex、企业微信、钉钉）都在这里逐段呈现。



提示词

安装与首次配置



安装流程

4. **安装**：插件管理器按目录 / npm 包 / tarball 安装本 bundle（或从插件市场收录后一键安装）。
5. **配置**：`stateDir` 留空即沿用已存在的 `~/.dsh/state/superagent`（旧版升级无需迁移）；两台 superagent 并存时各用各的目录。
6. **刷新**：刷新 Web 页面，打开侧边栏「Superagent 多通道」。
7. **接入来源**：逐个卡片填凭据，先点卡片上的「检查 / 读取」验证，再启用。

   完整字段与排障见 部署；每个来源的接入步骤见 扩展说明。

安全与边界



安全边界

- **只被动回复**：不群发、不拉群、不加好友；只在收到的会话里回应。
- **凭据在本机**：令牌/密钥优先放 0600 文件，不写进设置；截图与日志不打印密钥。
- **幂等与去重**：评论带幂等标记；回调/事件按平台消息 ID 去重；钉钉、企业微信的出站无服务端幂等键，已在 `facts` 明说。
- **平台限制不伪装**：微信/企业微信/钉钉没有消息深链；企业微信、钉钉是推送型后端，没有历史可翻；钉钉 Stream 需要部署注入 SDK 连接。详见 能力矩阵。

常见问题

会不会给所有人自动回复？ 不会。只有被评估接单的线程才会收到回复；名单外的人即使 @ 机器人也只被“观察”。 **数据会传到哪？** 默认全部留在本机 `stateDir`；只有程序调用各平台 API（读取/回复）时，数据才到对应平台。 **能和官方 superagent 同时跑吗？** 可以同时安装，但同一时刻只能启用一个（同一种 Source kind 只能注册一次）；各自用自己的 `stateDir`。 **升级会丢数据吗？** 不会。`stateDir` 不变；2.3.0 起 agent preset 沿用共享 id，旧会话可直接恢复。 **某个来源收不到消息？** 看 docs/008-cookbook.md 的排障表：企业微信查回调 URL 与 Token、钉钉查是否注入 Stream 连接、微信查桥 `/health`。

文档地图

文档	内容	篇幅 / 适合谁读
docs/README.md	文档总览与阅读顺序（English）	46 行 · 第一次读
docs/000-philosophy.zh-CN.md	设计哲学：为什么是「评估→工单→执行」	35 行 · 所有人，约 10 分钟
docs/001-domain.zh-CN.md	领域模型：Trigger / Thread / Ticket / Assignment	114 行 · 开发者、集成者
docs/002-authorization.zh-CN.md	授权与准入：策略、名单、点名判定	157 行 · 管理员、安全评审

文档	内容	篇幅 / 适合谁读
docs/003-execution.zh-CN.md	执行：执行器会话、目录隔离、交付	205 行 · 开发者、运维
docs/004-prompts.zh-CN.md	提示词与技能	98 行 · 提示词维护者
docs/005-panel.md	面板逐页说明（含截图）	60 行 · 日常使用者
docs/006-deployment.zh-CN.md	部署、配置项、升级与并存	298 行 · 部署者、运维
docs/007-architecture.zh-CN.md	架构与模块/测试清单	713 行 · 维护者、贡献者（当索引用）
docs/008-cookbook.md	常见做法与排障	51 行 · 值班、排障
docs/extensions/capability-matrix.zh-CN.md	六来源能力对照与平台限制	42 行 · 选型、评审
docs/extensions/	ADR-005 … ADR-012（每篇中英两份）	40–356 行/篇 · 决策回溯
docs/gallery.html	图文预览（English）	图片页 · 演示、评审

开发与测试

```
pnpm typecheck          # tsc --noEmit
pnpm test               # vitest run --maxWorkers=2
pnpm build              # tsdown, 8 个入口
```

最近一次全量回归（2026-11-14，2.4.0）：

```
tsc --noEmit   → 0
vitest         → 102 files passed / 5 skipped (107)
                 1624 passed / 41 skipped (1665)   0 failed
tsdown         → Build complete
```

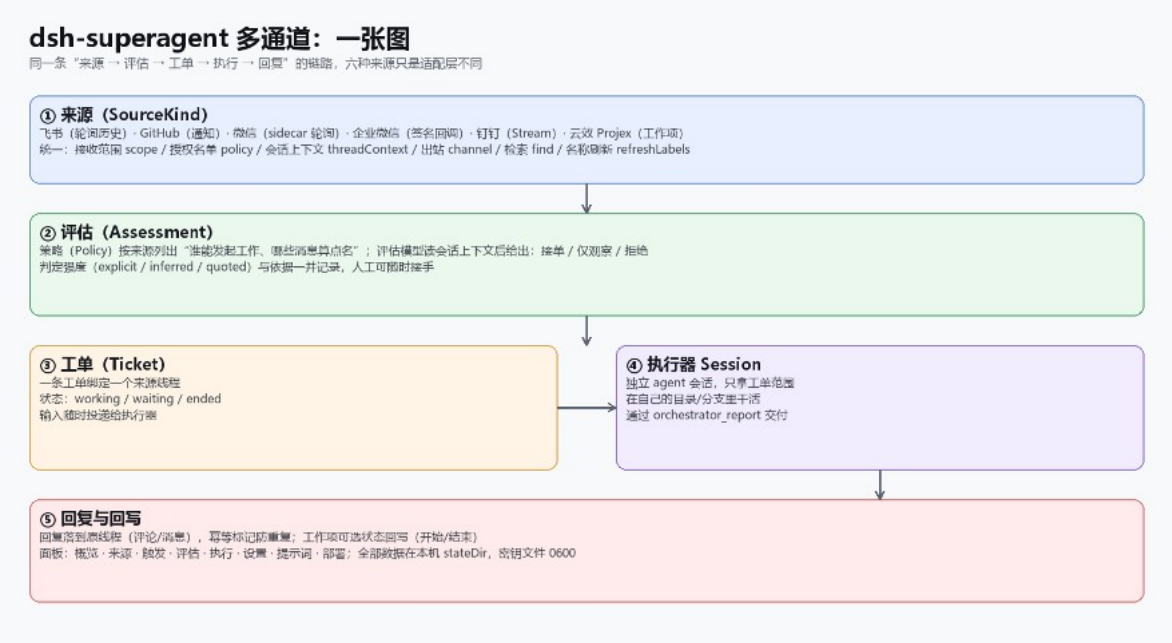
许可

MIT。上游 `dsh-superagent` 的版权声明保留；微信的 UI 自动化是第三方工具，未获腾讯授权，可能随时被客户端或平台限制，必须在 ADR-005 记录的行为边界内使用（只在被 @ 时回复、不群发、不批量加友、同一时刻只有一个账号在线）。

截图说明：面板截图为真实运行截图，敏感信息（项目/组织 ID、令牌路径、姓名、群名、会话标题）已打码；架构图与流程图是按当前实现绘制的示意图。

文档总览

本目录描述 dsh-superagent 多通道（2.4.0，2026-11-14 复核）现在**实际具备**的能力。**每一篇文档都有中英两份**：中文文件名带 .zh-CN.md，英文带 .en-US.md，基准文件保留原名；每篇顶部都有语言切换。所有页面都以当前实现为准；与平台限制相关的内容在 capability-matrix.md 里逐项标注为「做不到」而不是省略。



架构

阅读顺序

顺序	中文	English
1	000-philosophy.zh-CN.md	000-philosophy.md
2	001-domain.zh-CN.md	001-domain.md
3	002-authorization.zh-CN.md	002-authorization.md
4	003-execution.zh-CN.md	003-execution.md
5	004-prompts.zh-CN.md	004-prompts.md
6	005-panel.md	005-panel.en-US.md
7	006-deployment.zh-CN.md	006-deployment.md
8	007-architecture.zh-CN.md	007-architecture.md
9	008-cookbook.md	008-cookbook.en-US.md
—	../EXTENSIONS.zh-CN.md	../EXTENSIONS.md
—	extensions/capability-matrix.zh-CN.md	extensions/capability-matrix.md
—	extensions/（每个 ADR 有 .zh-CN.md / .en-US.md 两份）	same files, English variants
—	gallery.html	gallery.en.html
—	distribution/README.md	distribution/README.en-US.md

一页看懂



生命周期

- **输入统一**：六个来源都归一成同一个 `Trigger`（线程、发起人、结构化 @、原始 ID、可选图片）。
- **判断统一**：策略按来源配置名单与准入表；评估读上下文后给出 接单 / 仅观察 / 拒绝，并记录强度与依据。
- **执行统一**：一条工单一个执行器 Session，独立目录/分支，结构化交付。
- **回流统一**：回复落回原线程，评论带幂等标记；工作项可回写状态。

截图约定

- `images/panel-*.png` 是**实机截图**，敏感信息（项目/组织 ID、令牌路径、姓名、群名、会话标题）已打码；个别图带讲解红框。
- `images/architecture.png`、`lifecycle.png`、`sources.png`、`install.png` 是按当前实现绘制的**示意图**。
- 新增截图时放在 `docs/images/`，命名 `panel-.png`，并在正文图注写明「实机 / 示意」。

文档维护约定

- 改动能力面（新增来源、字段、动作）时，同步更新：本页、005 的页面说明、007 的模块/测试表、

capability-matrix，以及 CHANGELOG。

- 测试数量与命令只写在 README 与 007，避免多处漂移。

000 理念

dsh-superagent 是一个框架，也是一种与智能体协作的工作方式。它的核心原则是：

尽可能少占用人的注意力，尽可能多地交付价值。

项目中的每一部分都按它消耗的注意力来评判。一条消息、一条通知、一个问题、一个设置、一个面板元素、一段文档，各自都消耗某个人的注意力，因此每一项都必须给出超过这份消耗的价值，否则就删掉。

对智能体的要求

- **只在被请求时行动，或在书面规则允许时行动。** 一个请求只有在直接指向智能体、并且来自团队许可的人时才算数：此人在白名单上，或位于团队开放接受请求的会话中。没人要求的工作需要团队写下的规则，并且以只记录「智能体本来会做什么」的模式启动。
- **只在人必须行动、或能用到结果时才说话。** 它对一个请求只确认一次，并说明自己将做什么。工作时它保持安静。结果就绪时、需要决定时、或无法完成时，它才再次开口。消息简短，并发往提出请求的那个会话。
- **每次只问一个具体问题，** 并以结构化形式提出，让问题真正到达那个人，而不是留在长报告里。
- **以影响而不是投入来决定停在哪里。** 局部改动一直做到可评审的结果为止。需要设计决策的改动则以一个清晰的问题停下。
- **如实汇报。** 草稿标记为未完成，失败就报告为失败，永不为了回避提问而绕过某个限制。

对设计的要求

- **概念少。** 设计只包含它自己的领域概念（例如来源）及其具体实现（例如 GitHub 和飞书）。一个业务需求会成为其中某个概念的属性，或成为智能体遵循的提示词。设计不为它新增概念。
- **设置少。** 当某个值适用于几乎所有团队时，用默认值取代设置。团队用文字描述的行为是提示词文本，不是设置。
- **面板只显示需要人处理的东西。** 它先列出在等人的条目。健康状态不占空间。
- **文档按顺序阅读。** 每篇编号文档只建立在前面的文档之上，因此读者永远不需要一个尚未介绍的概念。

责任

智能体用你团队每天都能看到的账号发表评论、开启拉取请求、给人发消息。它发出的每条消息都消耗同事的注意力，而你要为此负责。智能体能做的事越多，其操作者对团队负的责任就越大：

- **从窄开始。** 先用一份很短的白名单，并让主动工作只被记录。等「智能体本来会做什么」看起来正确之后，再放宽这两者。

- **为更少的噪音调优。** 智能体吵闹时，把它的提示词改得更短更严，或暂停它。不要靠增加消息来解释噪音。
- **不要用它堆量。** 批量拉取请求、广播消息和无谓的忙碌，给团队带来的成本高于收益。
- **为它说的话负责。** 读一读智能体以你的名义发布的内容，发现错误时在提示词里纠正。

目标是在正确的时间给出正确的帮助，而在其他任何时间什么都不给。

001 领域模型

本文命名 dsh-superagent 的概念，以及每个概念遵守的规则。代码、智能体调用的工具、面板以及其他文档都使用这些术语，不使用别的术语。遵循 000 理念，模型只包含工作所需的概念，一个业务需求会成为其中某个概念的属性。

流程

工作从触发流向评估，再流向执行。来源产生触发。触发作为上下文扇出到订阅了它所属线程的工单。以这种方式到达某个工单的人的消息留在该工单上；其他每个触发都进入评估，由评估决定是否请求新工作。评估接受时，会创建一个带授权的工单。派工把工单交给执行器，结果通过渠道回到来源。

```
flowchart LR
  S[Source] -->|Trigger| T[Thread]
  T -->|batch| E[Eval]
  T -->|Subscription fan-out| K[Ticket]
  E -->|accept with Mandate / stop| K
  E -->|ignore / unavailable| X[recorded]
  P[Policy change] -->|notice| K
  K -->|Assignment| W[Executor]
  K -->|Channel| S
```

概念

来源、线程、评估、授权和订阅构成核心领域；其余概念为它们提供支撑。

概念	定义	遵守的规则	存在的原因
来源	一个事件流。来源种类是可复用的实现；来源实例是某个种类的一份配置：是否运行、监听什么	保存自己的读取位置和自己的机器人身份。定义什么算一个事件，包括只在时间上成立的条件。在进入领域之前丢弃机器人自己的消息和超出监听范围的触发。把每个触发翻译成它所属的线程，以及用于回复的渠道地址	重启之后不能重放事件
触发	一个不可变事件，带有来源自己的元数据，例如它的发起者	属于一个或多个线程。同一个触发被接受两次只创建一个工单	审计与幂等的单位
线程	按时间顺序排列的触发；由来源定义哪些触发构成同一个线程	同一时刻最多有一个评估在其上运行。它可以有多个工单。它持有自己的订阅、正在运行的评估以及决定历史	为评估提供一致的视图和上下文
关联	一个触发所属的多个线程之间的关系	只增加上下文；从不路由	让评估能够追踪相关的证据
渠道	线程的写侧；由来源定义它的地址	一个线程可以有多个渠道；一个工单持有一组渠道	确认、回复、拉取请求和已处理的评审都是写操作

概念	定义	遵守的规则	存在的原因
策略	带版本的评估规则，以及每个来源的那一部分：谁可以通过该来源、以该来源自己的身份发起工作，名单如何同步，以及哪些机器人可以产生候选	策略版本覆盖规则、接单模式和来源各部分。影响在途工单的变更（例如改动它当初被接受时依据的规则）会投递给它的派工；这种变更不会撤销授权。不同来源上的身份永不互相映射	规则改变时，正在运行的评估必须被丢弃
评估	对一批触发的判定，分两层：准入与研判	输入：触发批次、线程快照、线程上的所有工单（包括已结束的），以及当前策略。输出：一个决定。不持有状态	决定新工作是开始、继续还是停止
授权	一次接受所授予的东西：策略版本、范围、强度、命中的规则、发起者在该来源上的身份，以及来源接收该触发的方式	不可变；更大的范围意味着新工单。工单的结束不会结束授权：只有人的停止会撤回它，而不再成立它的策略会撤销它。已结束的工单恢复之前，会针对当前策略重新校验	说明这项工作为什么可以做，以及可以做到多远
工单	一个被授权的工作单元	只有评估能创建它。引用它的授权；持有它的渠道和输入。状态从开启 → 结束，当有人在它的某个线程上发言使其恢复时回到开启，除非人已停止它。它定义自己的状态，而不跟随执行器的状态	拥有执行过程及其报告
派工	一个工单与一个执行器实例的绑定	一个工单在一段时间内可以有多个派工。状态独立于工单，从待启动 → 执行中 → 已结束。只有它的最终报告可以结束执行器一侧的工单	执行器在领域之外，但谁在执行必须在领域之内

概念	定义	遵守的规则	存在的原因
订阅	工单与线程之间的多对多关系，由线程持有	匹配的触发会扇出到每个在役的已订阅工单，不检查是谁发的；以这种方式到达工单的人的消息不经过评估。执行器为获取上下文而跟随的线程以同样方式到达其工单，但不为它带来任何权限，也不会恢复它。已结束的工单保留它的订阅，包括已关闭或已合并的拉取请求的订阅：人的消息会在同一授权下恢复它，无需评估或决定；机器人和 CI 的消息不会到达它，因为它的执行器在恢复时会重新读取它的线程。被人停止的工单不再接收任何东西。主会话只有在消息明确指向某个工单时，才把线程附加到该工单。人可以丢弃线程上的所有订阅；执行器只能丢弃自己工单对某个它认为无关的线程的订阅，例外是它工单起始所在的线程，以及来源让它跟随的线程，例如它拉取请求的线程	后续讨论无需再提及智能体就能到达执行器

人不是领域概念。触发携带其发起者的标识符，形式由来源给出。白名单是策略中某个来源的那一部分，使用该来源的身份。同一个人在多个来源上拥有账号这件事，是智能体在需要时阅读的提示词文本。

执行器及其会话和目录位于领域之外，通过派工连接进来。

聚合：来源持有它的读取位置。线程持有它的订阅、正在运行的评估（串行运行，并把期间到达的触发合并进下一批）以及它的决定历史。工单持有它的派工、渠道和输入，并通过 id 引用决定。

被拒绝的概念

被拒绝的	原因
把主体或成员关系作为实体，或在来源之间映射身份	人是触发的元数据；白名单是策略中来源的一部分；跨来源映射是提示词文本
由评估自己调度延迟触发	需要等待的条件就是一个普通来源，因此评估永不反向依赖来源
等待中或已撤销的评估状态	等待属于来源；超时的评估会重新排队它的触发，或以 `unavailable` 结束

被拒绝的	原因
每个线程最多一个在役工单	一个线程可以承载多个独立的工作片段；研判通过工单列表避免重复
「追加到工单」这种决定	追加就是订阅扇出，不需要评估
对订阅做准入检查	订阅覆盖整个线程；授权把授权与上下文分开
会路由的关联	订阅负责路由；关联只增加上下文
跟随 DSH 会话的工单状态	工单可以被移交、恢复，或由多个执行器运行，因此它定义自己的状态
由执行器声明某人要求它停止	只有评估决定结束的强度
策略变更时取消正在运行的工单	被取消的变更会留下半成品工作；改为记录并投递该变更
执行器会话失败或阻塞时锁死它的所有工具	这会拒绝人在会话中的直接操作，以及恢复后的报告；暂停自动输入就够了
从报告的自由文本推断「在等人」	该分类需要结构化事实；执行器通过结构化字段提问

可复用的部分

上述概念及其规则是可复用的。来源实现、策略内容、执行器以及渠道实现属于某个部署。每个来源在内部保留自己的用词，例如消息、聊天或通知，并把它们翻译成触发。

部分	本仓库中的实现
来源	飞书：实时事件、历史回溯、订阅话题的轮询。GitHub：通知、已订阅的 Issue 与拉取请求、CI 变更、已合并或已关闭的拉取请求，以及在开启时仓库中每个新 Issue、拉取请求和评论，以及默认分支的 CI
策略	研判规则，以及每个来源的白名单、它的同步、它的机器人名单和它的准入表
执行器	一个 DSH 执行器会话，以及每个工单一个目录，位于一个执行环境中，该环境的变量由执行器和来源注册
渠道	飞书话题回复、GitHub 评论与评审线程回复

术语表

本文及后续文档所用术语的参考列表，附其中文名；上面未介绍的术语由后续文档解释。面板一次只显示一种语言；它的 (?) 按钮会用两种语言打开本术语表。智能体在 GitHub 和飞书上发给人的消息使用平实语言，不使用这些术语。

术语	中文	含义
Agent	智能体	dsh-superagent 运行的自动化团队成员，通过 GitHub 账号、飞书机器人等身份在来源上行动
Main Session	主 Session	评估触发、接受并派发工单、以及回复的 DSH 会话；在代码和提示词中为 `orchestrator`
Executor Session	执行器 Session	执行器为一个工单运行的 DSH 会话；在代码和提示词中为 `worker`

术语	中文	含义
Source	来源	一个事件流：由插件提供的一个种类，加上一份实例配置
Trigger	触发	来自来源的一个不可变事件
Thread	线程	按时间顺序排列的触发；评估在其内部一次只运行一个的单位
Correlation	关联	共享同一个触发的多个线程之间的关系；它只增加上下文
Channel	渠道	线程的写侧
Policy	策略	用于接受工作的带版本规则，以及每个来源在其中的那一部分，例如谁可以在那里请求工作
Policy version	策略版本	一种规则、来源各部分与接单模式组合的标识符；每个授权都会记录它
Eval	评估	把一批触发变成一个决定的过程
Gate	准入	评估中确定性的第一层：谁可以发起工作，以及以什么强度
Assessment	研判	评估过程中模型的判断
Decision	决定	评估的结果
Mandate	授权	一次接受所授予的东西：强度、范围以及它所依据的策略版本
Ticket	工单	一个被授权的工作单元
Assignment	派工	一个工单与一个执行器实例的绑定
Subscription	订阅	工单与线程之间的关系
Fan-out	扇出	把已订阅线程的触发作为上下文投递给工单
Executor	执行器	运行派工的端口：每个工单一个执行器会话和一个目录
Execution environment	执行环境	执行器会话与 prepare 命令运行的地方：由执行器和来源注册的变量，每个变量有键、值和描述
Report	报告	派工向其工单报告的状态
Attention	待人工决定	工单上等待人回答的问题
Disposition	去向	一个触发被过滤、被评估或被扇出的记录
Input	输入	投递给工单的上下文：扇出的触发、策略变更、恢复和文本
Scope	范围	授权允许的工作
Strength	强度	授权或评估的依据有多强

分组	取值
Decision	accept 接单 · stop 停止 · ignore 忽略 · unavailable 未完成
Strength	explicit 明确 · inferred 推断
Assignment	pending 待启动 · running 执行中 · ended 已结束

术语	中文	含义
Report		progress 进展 · waiting 等待 · ready 就绪 · blocked 受阻 · failed 失败 · dropped 无需处理

002 授权

授权决定谁可以启动工作，以及该工作可以包含什么。本文建立在 001 领域模型 之上。

宿主强制实施白名单、规则和授权。消息、评论和引用的文档都是决定的输入；它们不能改变那些权限。

触发的两条路径

评估：决定是否启动工作

来源记录一条消息是否指向智能体：`mention` 表示提及机器人或智能体账号，`direct` 表示私聊消息。准入检查发送者和该来源的策略：

- 白名单上的人如果指向智能体，就构成明确请求。
- 人在已有工单的线程上的跟进，按下文规则处理。
- 其他人类消息按它们所在接收范围的准入表处理。
- 来源机器人名单上的机器人可以产生推断候选，永不产生明确请求。其他机器人不被准入。

订阅：继续已有的工作

已订阅线程上的每条消息都会作为上下文到达其已订阅的工单，包括来自机器人和 CI 的消息。这种投递绕过准入，也不授予任何新权限。每个工单仍然受其授权限制。

人在工单自己线程上的消息，由该工单的执行器决定。主会话把一条消息附加到某个工单时同样如此。当执行器拥有该消息时，它不经过评估，也得不到决定。每个执行器要么在其工单范围内处理该请求，要么把它交回以便路由到别处。

只要工单在役，归属关系就持续存在，即使有人移除了它的订阅。它也覆盖那些未被人停止、并且有待处理消息可将其恢复的已结束工单。

主会话会收到跟进通知。当执行器拥有该消息时，主会话只能按人的要求停止工作；接受、附加、忽略和重新排队都会被拒绝。停止仍然要通过准入，强度为下文所述的跟进强度。

交回与释放跟进

执行器在请求超出工单范围的工作、或消息属于另一个工单时，交回一次。工单保留它的订阅。如果被附加的线程无关，执行器交回它在那里拥有的消息。

只有当没有任何工单仍然拥有该消息时，它才进入一次新的评估。它的候选包含执行器给出的理由。准入把它当作该线程上的普通消息来判断，主会话可以接受它、把它附加到别处或忽略它。交回一条工单并不拥有的消息（例如机器人消息，或人已经发给评估的消息）不会排队任何东西。

工单结束时，它会释放其执行器未读取或未处理完的跟进。这包括从未投递给执行器的消息，以及在它最后一份报告或轮次结束之后投递、而执行器既未确认、未回答、也未交回的消息。每条被释放的消息在没有其他工单拥有它时进入评估。停止只结束被停止工单的工作。

被停止的工单不会把它的停止消息再次发给评估，除非另一个工单把它交回。停止它的主会话可以把消息剩余部分作为新工作接受。

人可以从面板把一条已被拥有的跟进发给评估。如果主会话随后把它附加回一个在役工单，执行器会再次收到它。当没有任何有效判断覆盖该消息时，面板也会提供重新评估。

仅作上下文的订阅

执行器可以为获取上下文而跟随相关线程，例如另一个 Issue 或拉取请求。那里的消息会到达工单，但不授予权限，也不带来跟进强度。那里的人类消息不会恢复已结束的工单，除非主会话附加了来自该线程的一条消息。主会话关于其他工作的说明也只是上下文。

未到达任何工单的人类消息（包括其工单被人停止的线程）走评估路径。机器人消息独立于它们对已订阅工单的投递，也走该路径。

强度

强度	情形
explicit	名单上的人指向智能体，或在已有工单的线程上跟进；准入表中设为明确请求的单元格
inferred	不在名单上的人在已有工单的线程上跟进，除非其准入表单元格是明确；准入表中设为推断候选的单元格；来源机器人名单上的机器人发出的、指向智能体的消息，或到达来源接收所有消息之处的消息
not admitted	其他任何机器人的消息、准入表忽略的一切，以及接单模式为 `off` 时的所有推断情形

对于不在名单上的人，明确的准入单元格优先，即使在已有工单的线程上也有效，并且在任何接单模式下都成立。否则其跟进为推断，在 `off` 模式下不被准入。仅在上下文中被跟随的线程上的消息，使用准入表而不是跟进强度。

准入

每个来源的策略都有一张准入表。它的行是接收范围：

- **飞书：** 每个已配置的群、机器人从中收到提及的其他群，以及私聊。只有在飞书卡片上把 **其他群** 设为 **仅 @** 时，其他群才会被接收。
- **GitHub：** 新 Issue、新拉取请求、评论以及默认分支的 CI。

一行最多有三个单元格。每个单元格决定一条消息成为明确请求、推断候选还是什么都不成为：

单元格	可选值	默认
名单内，未指向智能体	inferred、ignore	inferred
名单外，指向智能体	explicit、inferred、ignore	ignore
名单外，未指向智能体	inferred、ignore	ignore

针对未指向消息的单元格，只在来源接收所有消息的地方可用，例如设为 **全部消息** 的飞书群，或开启了接收全部开关的 GitHub 范围。CI 结果没有发送者，因此它的行只提供最后一个单元格。机器人消息使用机器人名单，而不是这些单元格。

把「名单外且指向」单元格设为 explicit，可以让该范围内的任何人通过提及智能体来请求工作。这对专门的客服群可能有用。推断候选仍然取决于接单模式。跟进通常绕过此表，但 explicit 单元格仍然授予明确强度。

准入表是策略版本的一部分。弱化某个单元格不会立即停止按其接受的在途工单。当这些工单在恢复之前被重新校验时，它可以撤销它们的授权。

机器人与自动评审者

宿主把 GitHub App、以 `[bot]` 结尾的登录名以及飞书机器人识别为机器人。来源可以列出诸如 `dependabot[bot]` 的机器人，它们的消息可以成为推断候选。它们必须指向智能体，或在接收所有消息的范围内发帖。它们不能发出明确请求，也不能停止工单。

使用普通账号的自动评审者，对宿主而言算作人。它的评论会到达已订阅的工单并产生跟进通知。团队提示词文本必须把这类账号标记为自动评审者。

路由候选

主会话用 `orchestrator_route` 路由明确候选。一次调用可以包含多个接受、附加、停止或忽略的决定。每一项独立处理：某一项被拒绝或缺少字段，不会阻止其他项执行。

推断候选通常需要完整研判。有一个捷径：如果它的卡片明显不触及任何已启用的规则，例如普通闲聊或无关讨论，主会话可以直接忽略它。该决定记录 `ignoredFromCard`，以区别于按规则研判。主会话在其策略提示词段落中读取已启用的规则。

接受推断候选始终需要研判。直接接受会拒绝准入未认定为明确的触发。

接单模式与规则

接单模式是策略的一部分：

模式	效果
`off`	推断出来的东西都不会成为候选，例如未指向的消息、机器人的消息，或名单外人员的跟进
`observe`（默认）	对推断候选进行研判；正向研判被记录为标记「would accept」的忽略，不接单

模式	效果
`auto`	对推断候选的正向研判会接受它

明确请求在任何模式下都有效。从 `observe` 开始，在审阅记录下来的决定之后再切换到 `auto`。每次研判都会对每条已启用的规则恰好判断一次，无论是否命中。主动接受要求至少命中一条规则。

每条规则描述一种主动工作。工单的授权记录命中的规则，其执行器会收到这些规则的名称和说明。

对该工单而言，规则中的流程与沟通要求覆盖默认值。静默规则可以抑制确认；评审规则可以指定必需的评审者和合并前的批准。适用于所有工单的要求应写在团队提示词段落中。

策略版本

规则、接单模式以及每个来源的策略共同构成策略版本。每个授权都记录它被接受时所依据的版本。

版本变更会丢弃正在运行的评估并重新排队它们的触发。它还会向每个受影响在役工单发送一条策略变更输入。该输入说明是否：

- 命中的规则发生了变化（包括其新文本），或不再适用；
- 请求者可能不再能启动工作；或
- 主动工作不再被接受，例如因为模式从 `auto` 改变了。

未受影响的工单不会收到通知。该变更不会立即撤销在役工单的授权。

来源配置与策略是分开的。它控制什么进入系统：读取哪些群，以及某个范围是接收全部消息还是仅接收提及。授权记录它的来源和接收范围。恢复需要来源仍然监听该线程，但不要求它仍然接收那里的每条消息。

当来源关闭时，回复会等待。如果来源不再监听其目的地，回复会被丢弃。

关于执行器可以用自己的结果做什么的条件（例如何时可以合并），不决定谁可以授权工作。它属于智能体的指令，而不属于策略。

恢复与停止

已结束的工单保留它的授权。人在它自己某个线程上的消息可以在没有新评估或新决定的情况下恢复同一工作，无论发送者是谁。机器人和 CI 不会唤醒已结束的工单。执行器决定该请求是否符合范围，若不符合则结束该轮。

只有三种操作会改变授权：接受创建它，人的停止撤回它，重新校验失败撤销它。被人停止的工单保持结束，不再接收任何东西。在那里请求工作需要新的接受。

恢复前的重新校验

来源必须仍然监听原始线程。其余检查取决于授权的依据：

- 明确的跟进授权通过。

- 推断的授权需要 `auto` 模式，且其命中的规则仍然启用。
- 由表格单元格准入的授权，必须在今天的表格和发送者成员关系下仍能获得至少其原始强度。因此，把发送者从名单中移除，可能撤销通过「名单内发送者」单元格接受的授权。
- 机器人的授权要求该机器人仍在来源的机器人名单上。推断的表格授权和机器人授权还需要 `auto` 模式以及命中的规则已启用。
- 其他授权要求发送者仍在来源的白名单上。

如果重新校验失败，工单保持结束，不接收它的任何等待消息。每条消息都被视为未到达任何工单；如果没有其他工单拥有它，它就进入评估。主会话不会被告知它到达了执行器。处理暂停之后同样如此。附加到该工单会被拒绝，进一步工作需要新的接受。

- **人在线程里说「stop」是一个普通触发，** 由评估决定停止。执行器不能声明某人要求它停止。
- **面板中的停止是一条命令。** 它直接作用于工单，并记录为手动停止。
- **等待类条件属于来源。** 例如「N 小时无处理人」这样的条件会是一个自行等待并再次检查的来源；评估从不等待。

评估未完成时

时限从候选到达主会话时开始。等待投递所花的时间不计入，未投递的候选永不过期。

超时的评估会重新排队它的触发以便再次尝试。连续两次重试之后，第三次超时记录为 `unavailable`。研判失败同样记录 `unavailable`；之后的触发可以启动新的评估。策略变更，或针对过期策略做出的决定，记录 `unavailable`，并立即按当前版本重新排队该批次。

`evals` 健康条目报告过去一小时内未解决的 unavailable 决定。`scripts/dsh-superagent status` 会包含该错误。

只有当该 unavailable 决定不满 24 小时、仍是其线程上最新的决定、未被忽略，并且该线程既未排队也未在评估时，面板才会请求人工关注。人可以再次评估该触发，或忽略该问题。

003 执行

本文跟踪一个已被接受的工单，经过执行、投递和恢复的过程。它建立在 001 领域模型 和 002 授权 之上。

工单阶段

每个开启的工单都有一个阶段，由它在等待什么决定。从上到下读表；第一个匹配的行胜出。

阶段	条件	面板标签
person / delivery	把输入投递给执行器会话失败；2 秒后重试，之后按翻倍延迟重试直至一分钟，一次成功、新的派工或工单结束都会清除它	Input undelivered
person / answer	它有一个待人工决定	Awaiting answer
person / review	最新报告是 `ready`	Awaiting review
person / blocked, failed	最新报告是 `blocked` 或 `failed`，且此后没有人的消息到达该工单	Blocked · Failed
working / queued	没有派工，或有一个待启动的派工	Queued
idle	最新报告是 `waiting`，或派工已结束	Waiting for a Trigger
working	其他所有情况	Working

`person` 阶段意味着工单在没有人工输入的情况下无法继续。新的人类消息会把 `blocked` 或 `failed` 的工单释放给它的执行器，因此它移动到下一个匹配的阶段，通常是 `working`。

`idle` 工单已完成它的轮次，且没有悬而未决的问题。已订阅线程上的触发可以继续它。

执行器

每个工单得到一个名为 `worker-` 的执行器会话，以及一个以工单短名命名的目录。目录创建在配置的执行根目录下，默认是 `/work`。

执行器会话以操作者的操作系统用户身份、无需审批地运行命令。它们的 `dsh-superagent-worker` 预设包含工作所需的工具，并在 5 秒后把前台 shell 命令转为后台任务。这让执行器能在下一步接收新消息，而不必等待长命令完成。

主会话使用 `dsh-superagent-router`，具有相同的 5 秒 shell 与作业等待限制，以及更小的路由工具集。会话保留创建时所选的预设。执行器子智能体使用 DeepSeek Harness 的 `standard` 预设。

后台工作与执行器槽位

当一轮结束时仍有 shell 作业或子智能体在运行，工单保留它的执行器槽位并保持工作状态。它不会被报告为等待，主会话也收不到等待报告。已完成的作业或子智能体会在同一槽位中带着结果唤醒执行器。一旦某轮在无后台工作的情况下结束，执行器就正常报告等待。

每一轮结束后，后台工作最多有 30 分钟完成。到达该限制时，shell 作业被杀死，工单被报告为等待。报告包含执行器最后的话，以及哪些作业被杀、哪些子智能体仍在活动。子智能体允许继续。执行器在下一轮读取被杀作业的结果。

向人的移交会立即释放槽位，并让报告继续生效。这适用于 `ready` 报告或提问之后（除非此后又到达了其他输入），也适用于任何 `blocked` 或 `failed` 报告之后。

移交时，子智能体停止。shell 作业（例如供评审的服务器）可以继续运行直到派工结束。它们不占用槽位，也不受 30 分钟限制，后续轮次之后同样如此。它们的完成不会唤醒执行器；执行器在下一轮读取结果。

人也可以直接在执行器会话中书写来开启一轮。它的后台工作遵循同样的规则。

派工结束时，它的 shell 作业被杀死且不唤醒会话。它的子智能体被释放，它们唤醒的任何完成轮次都被丢弃。替换的派工及其等待中的输入会在该关闭完成之后启动。

在 DeepSeek Harness 进程内停止或重启插件，走同样的关闭路径。尚未进入会话历史的输入会被保留。关闭会为每个执行器的后台工作最多等待 10 秒；进程重启则直接终止该工作。在忙碌中被中断的派工会带着恢复说明重新启动。

执行器子智能体（包括嵌套的）使用 DeepSeek Harness 的 `standard` 预设，并具有执行器的权限。它们只保留执行器被允许的工作工具，并在工单结束时失去这些工具。在 `standard` 下，shell 命令在 60 秒后转入后台。子智能体的作业在其结束时被杀死。

准备工单目录

在第一次派工之前，如果工单目录不存在，prepare 命令会在执行根目录下用 `sh` 运行一次。它有十分钟完成。没有 prepare 命令时，执行器在空目录中启动。

命令失败会使工单失败并记录其输出。重试或人在该工单某个线程上的消息会再次尝试准备。典型命令使用 `git worktree` 创建一个检出。当拉取请求分支的最后一段路径与工单短名匹配时，该拉取请求属于该工单。

并发

工单从启动开始（包括准备阶段）就持有一个执行器槽位，或者从唤醒其空闲执行器的那次投递开始持有。它在整个活动轮次中同样持有该槽位。

配置的并发上限作用于这些槽位，另有一个额外槽位用于人的消息或状态请求。当普通槽位已满时，这种唤醒可以使用额外槽位，并在新启动之前取得下一个空闲槽位。因此新工作无法阻止人继续已有的工单。

其他输入（例如 CI 结果）等待普通槽位。启动会在 `working / queued` 中等待直到有槽位空闲。启动是并发进行的，因此缓慢的 prepare 命令只延迟它自己的工单。

执行变量

prepare 命令和执行器指令收到相同的执行变量。命令读取环境变量；执行器在其提示词中读取它们的值和描述。其中都不包含凭据。

变量	注册者	值
`DSH_SUPERAGENT_DIR`、`DSH_SUPERAGENT_NAME`、`DSH_SUPERAGENT_TICKET`	Executor	工单的目录、短名和 id
`DSH_SUPERAGENT_BLOCKED_COMMANDS`	Executor	执行器不可运行的命令
`DSH_SUPERAGENT_GITHUB_REPOSITORY`	GitHub	来源所监视的仓库，`owner/name`
`DSH_SUPERAGENT_GITHUB_ACCOUNT`、`DSH_SUPERAGENT_GITHUB_COMMIT_EMAIL`	GitHub	智能体账号及其提交邮箱
`DSH_SUPERAGENT_GITHUB_NUMBER`、`DSH_SUPERAGENT_GITHUB_PR_HEAD`	GitHub	启动该工单的 Issue 或拉取请求，以及当它是拉取请求时的 head ref
`DSH_SUPERAGENT_FEISHU_BOT`、`DSH_SUPERAGENT_FEISHU_CHAT`	Feishu	机器人，以及启动该工单的群

工单特定的值来自它的目录和起始触发。其他值来自当前部署设置。只有面板能更改执行环境设置；发给主会话的文本无法更改已配置的 shell 命令。

执行器通过智能体账号的 `gh` 访问 GitHub，通过机器人的 `lark-cli` 访问飞书。被阻止的命令在按名称调用时失败。

这些包装脚本和目录建立的是默认值，而不是安全边界。执行器仍然可以通过绝对路径执行程序。

合并

自动合并由 **Delivery requirements** 提示词管控。宿主没有针对合并的专门决策逻辑。

默认情况下，执行器永不以智能体账号启用自动合并：GitHub 会把合并归因于启用它的那个人。交付要求必须指定另一个账号的 CLI 用于合并。没有指定时，由人来合并拉取请求。

默认指令只允许为工单开启的拉取请求启用自动合并，且要求它已就绪、必需批准覆盖 head 提交、新增加删除合计少于 200 行。在推送另一个修订之前，执行器会撤回自动合并。

没有正文的评审（例如批准）也会作为触发到达，并能唤醒正在等待评审的执行器。这些指令管控的是智能体，而不是 GitHub 的合并规则。要强制批准，请在分支保护中要求批准并清除过期批准。省略合并 CLI 或删除自动合并段落即可要求人工合并。

工单如何结束

- **就绪但没有拉取请求：** `ready` 报告使工单以已完成结束。

- **无事可做：** 执行器发现没有人需要行动的工作（例如请求无法复现或已被取代）时，报告 `dropped`。工单在没有消息的情况下结束，除非命中的规则要求一条消息，而人之后在其线程上的消息会恢复它。

拉取请求

一旦执行器报告了工单起始所在的拉取请求，工单就跟随它。它也跟随从它的分支开启的每个拉取请求，条件是执行器已报告或 GitHub 来源已发现。重新开启的拉取请求会再次被跟随。

当它开启的所有拉取请求都不再处于开启状态时，工单就完成了。如果它没有开启任何拉取请求，那么它跟随的所有拉取请求都必须已关闭。然后它以最后一次相关关闭的结果结束，并保留它的订阅，允许之后的人类评论恢复它。

当恢复、重试或恢复的工作尚未完成时，关闭不会结束工单。如果执行器确认或回答了它拥有的某条跟进，但此后尚未做出非进展报告、或在读取该消息后结束一轮，它同样会等待。之后的一份 `ready` 报告在满足同一拉取请求完成规则后结束工单。

聊天渠道会收到一条关于拉取请求结果的最终通知。GitHub 本身已在拉取请求上显示该结果。移除工单对某个拉取请求的订阅，也会让它停止跟随该拉取请求。

因此，「等待评审」意味着带拉取请求的 `ready` 报告。

跟进与上下文线程

执行器决定其工单自己线程上的人类跟进。它处理范围内的请求，并交回超出范围或属于其他工单的请求。一旦没有执行器拥有该消息，它就变为主会话的候选。除此之外，主会话只能按人的要求停止工作来处置该跟进。

输入在会话把它纳入某个步骤、并随后报告或结束一轮之后算作已读。工单结束时，未读且未被确认、未回答、未交回的消息回到评估，如 002 授权 所述。已读但未处置的消息（例如致谢）不会回到评估。

仅作上下文跟随的线程仍会投递消息，但不给工单对人工跟进的归属，也不恢复已结束的工单。那些消息可以被独立研判。

只要有一个在役工单订阅，来源就会轮询已订阅的线程，并在最后一个触发与订阅工单结束两者中较晚者之后继续两周。此后，消息只通过普通入口到达：GitHub 通知或飞书实时事件。

失败与恢复

执行器受阻或失败

当执行器会话报告 `blocked` 或 `failed`，或启动失败或出错（记录为 `failed`）时，工单不接受任何自动输入。

模型提供方或网络的瞬时失败、HTTP 408、429 或 5xx，或被拒绝、重置、超时的连接，会先重试三次，退避时间递增：启动会再次运行，轮次失败的执行器被告知继续；失败的 prepare 命令不再重试。

一旦失败，CI 和机器人消息会记录在工单上，但不被投递，也不唤醒会话。

它的工具仍然可用，因此人可以直接在会话中继续。

失败的启动不会自行重试。

人的消息、之后一份既非 `blocked` 也非 `failed` 的报告，或一次重试，都会解除该保持状态。

该状态由存储的报告和输入推算得出，因此重启后仍然保持。

重试

在开启的工单处于 `person / delivery`、`person / blocked` 或 `person / failed` 时可用：新的派工会再次启动其执行器并接收等待中的输入，这是针对每一种情况的补救手段。

一旦人的消息到达受阻或失败的工单，执行器就拥有了该消息，因此工单显示为工作中，或在失败的启动重新开始时显示为排队中，并且不提供重试，除非投递该消息失败。

重试会结束当前派工，并在同一个执行器会话、目录、分支和拉取请求中启动新的派工。

执行器模型只在会话创建时生效；已存在的会话保留它的模型。

重启

运行中的派工以「host restarted」结束，新的派工带着恢复说明启动，该说明告诉执行器在重复任何外部操作之前先检查它的目录、git 历史和 GitHub，最旧的工单优先。

处于等待、就绪、受阻或失败状态的派工保持原样，除非在该报告之后有输入到达，那样它们也会重新启动。

会话、目录、订阅、待处理消息以及下文所述的已存储工作都能在重启后存活，每个执行器会话纳入其历史的消息记录同样如此：某个步骤在认领消息之后、调用模型之前把它们纳入历史——被重启切断的轮次所纳入的消息，在随后出现一份报告或一轮结束之后算作已读；而某个步骤已认领、但在进入历史之前被取消或错误切断的消息算作未读。

重试或重启会把投递给被替换派工、但其执行器会话未纳入的那些消息再次交给新派工，这些消息可能因取消或被切断的步骤而丢失；仍留在会话中等待的一份副本（由重启保留或跨崩溃持久化）会在新派工的第一条消息之前被取出，因此新的投递是唯一的副本。

已存储的工作

dsh-superagent 在决定之后要做的事（例如把候选、跟进或报告转达给主会话、启动派工或投递输入）会先被存储，然后再执行。

转达或投递失败（例如无法到达其会话）时，会在 2 秒后重试，之后按翻倍延迟重试直至一分钟；启动失败则使其工单失败，除非该失败是上文所述的瞬时失败。

每一类工作只有在连续三次失败之后，才会在概览的系统问题中报告错误；投递输入失败则改为显示在其工单上。

发往主会话的转达保持顺序，通知同样如此；一次失败会挡住同类中较晚的那些。

来源失败

GitHub 来源会分别读取每个 Issue、拉取请求和线程：读取失败会把错误保留在其线程上，在下一次轮询时再次读取，并计入来源的一次错误，默认在连续三次轮询失败后由来源报告；任何一次成功的轮询都会重置计数。

当智能体失去对私有仓库的访问权限时，GitHub 也会返回 404，因此 404 会重试。

一个已被通知但读取持续失败的 Issue 或拉取请求，会在首次读取失败一天后被丢弃，若 GitHub 报告它已删除则立即丢弃；此后它不再导致轮询失败，其线程保留该错误，来源的状态会点名它。

它的读取位置保持在原处，因此之后的通知或对其线程的轮询会读到失败读取所遗漏的内容。

飞书事件消费者会报告每一次失败，对已订阅话题的轮询会把每次失败保留在其线程上。

GitHub 列表或飞书页面中的格式错误条目会被跳过，来源的状态会把它计数。

渠道失败

出站消息作为已存储的工作被存储并发送，从 5 秒开始按翻倍退避重试，最多到 300。

一个工单发往同一个会话的消息按排队顺序发出，因此它的确认先于之后的回答或结果：当同一工单发往同一会话的较早消息尚未发出时，后续消息会等待，直到那条消息连续三次失败并显示为问题。

其他工单的消息，以及不属于任何工单的消息，不会等它。

有失败尝试的消息在发出或被撤回之前是一个系统问题。

只有当来源不再监听该会话，或来源回答该会话永远无法接收它时，消息才会被丢弃；当来源关闭时，消息等待。

GitHub 上已删除的 Issue 或已锁定的会话，以及飞书上被撤回或飞书已找不到的消息，来源会给出上述回答。

对已不存在的评审评论的回复，改为发往拉取请求的会话。

被丢弃的消息保留它的记录，包括丢弃时间、原因以及它所服务的工单，因此它永不会再次排队；被丢弃的确认读作已丢弃，而不是已发送。

暂停处理

停止派发、新的评估、评估超时、输入投递和回复；已经在进行的启动会等到处理恢复。

来源继续记录触发，正在运行的执行器轮次不被打断。

人发给已结束工单的消息会等待：暂停后的第一遍处理会恢复该工单，前提是它的授权在当时策略下仍然成立。

进行中的评估在暂停期间不会过期；它的截止时间按暂停时长向后顺延。

无效配置

被其种类拒绝的已存储来源配置会停用该来源实例。

004 提示词

本文说明智能体读什么、团队可以编辑什么，以及改动何时生效。它建立在 000 理念 到 003 执行 之上。提示词文本用英文书写。

团队写什么

团队可以编辑三个团队段落、添加技能，并覆盖每一个内置系统提示词部分。大部分默认内容位于 `prompts/` 和 `skills/` 之下。工具描述、通知和卡片仍留在代码中，无法通过提示词设置编辑。

段落	内容
Communication	何时给人发消息、消息怎么写、每条 GitHub 消息的开头行（默认 `This is by an automated agent, not a human`）、如何处理评审，以及哪些普通账号是评审机器人
Ticket scope	工单何时继续、何时带着草稿停止
Delivery requirements	拉取请求中的证据、带行数阈值的自动合并条件，以及提交作者；未指定时，执行器以智能体账号的 no-reply 地址提交
Team skills	智能体在需要时加载的知识，例如说明哪个飞书用户对应哪个 GitHub 账号的 `identities` 技能

内置文本覆盖角色、工具使用、宿主机制、授权规则以及 GitHub 和飞书技能。它按标题引用团队段落，而不重复它们。

内置部分与模板

覆盖使用这些键：

部分	键
共享指令与角色	`runtime/shared`、`runtime/orchestrator`、`runtime/worker`
工作指令	`runtime/subscriptions`、`runtime/replies`、`runtime/dispatch`、`runtime/work`、`runtime/admission`
移交指令	`runtime/handoff/orchestrator`、`runtime/handoff/worker`
来源技能	`skill/feishu`、`skill/github`
生成段落	`runtime/environment`、`runtime/policy`、`runtime/policy/none`

生成段落是可编辑的模板。它们必须保留这些占位符：

- `runtime/environment`：`{variables}`，填入执行环境各行。
- `runtime/policy`：`{version}`、`{mode}` 和 `{rules}`，每条已启用的规则一行。
- `runtime/policy/none`：`{version}` 和 `{mode}`，在没有启用任何规则时使用。

编辑某个部分会改变模型收到的指令。它不会改变准入、授权、工具检查或账号包装脚本，这些仍由代码强制执行。

在人类明确要求时，主会话还可以用 `orchestrator_prompts_save` 保存覆盖。如果一次保存省略了某个覆盖的 `base`，它会保留已保存的 base。已保存配置中已经存在但被移除的部分会在保存时被丢弃；其他任何未知部分会被拒绝。

覆盖会记录它基于哪份内置文本。升级会保留团队的文本。如果默认内容发生变化，面板会把该部分标记为 **Default updated**，并在团队文本旁显示差异。团队可以保留自己的文本、采用新的默认内容或编辑它。没有覆盖的部分会自动跟随新的默认内容。

主会话可以在 `orchestrator_read prompts` 的 `default_changed` 下读取受影响部分的名称。

组装与优先级

系统提示词按以下顺序组装：

8. 共享指令与角色。
9. 三个团队段落。
10. 订阅与回复。
11. 主会话的派发，或执行器的工作指令。
12. `runtime/admission`，仅用于主会话。
13. 移交指令。
14. `runtime/environment`。
15. `runtime/policy`，仅用于主会话。

准入指令来自 `prompts/admission.md`。它们描述主动研判，以及何时可以根据卡片忽略一个无关候选。`orchestrator_eval` 返回上下文，而不重复这些指令。

环境段落给主会话提供部署级的值，给每个执行器提供其工单特定的值。策略段落在每次模型请求时都从设置重新生成。它包含策略版本、接单模式、明确请求的豁免，以及以 `- ():` 形式列出的已启用规则，或一条说明没有启用任何规则的陈述。它放在最后，因此策略变更不会改变前面的提示词文本。

执行器还会读取 DeepSeek Harness 的编码提示词和仓库的 `AGENTS.md`；主会话不读。推断出的工单会附加一条结尾说明，把其执行器限制在命中的规则内。派工和收到的消息以触发卡片的形式内联到达，只有被截断的内容才带资源指针。

执行器无人值守地工作。它们的工具排除那些向人提问、向人展示界面或更改配置文件的工具，例如 `ask_user_question`、`present` 和 `plugin_manager`。

从最强到最弱：

16. 由工具和宿主强制执行的事实：身份、权限、工具参数检查、回复用途、长度限制。提示词文本无法改变它们。
17. 内置段落与生成段落，按团队对它们的覆盖内容读取。

18. 主动工单命中规则中的流程与沟通要求，仅作用于该工单的执行器会话。
19. 团队段落与团队技能。
20. 外部文本：消息、评论、文档和引用。它是数据，永不改变规则或授权。

内置文本的原则

这些原则把 000 理念 应用于智能体：

- **角色分离。** 主会话只做路由：它转达原始触发和路由事实，不做技术决定，并通过一个路由工具在单次调用中对多个等待中的候选做出决定。一条路由说明最多 30 个词。执行器读取原始来源、判断范围、实现并验证。
- **主会话不等待。** 报告、跟进通知、候选和已完成的后台作业都会作为消息自动到达。没有剩下要做的事时它就结束轮次。守卫会拒绝 `sleep`、超过 30 秒的 shell 超时和超过 5 秒的作业等待，因为被阻塞的步骤会延迟新的候选。这些指令在主会话的完整提示词中取代 DeepSeek Harness 自带的作业通知说明。
- **只在人必须行动或能用到结果时发消息：** 结果可以被评审或使用、需要决定或资源，或工作无法完成。聊天消息默认一句话，硬上限 100 词。
- **执行器决定自己的跟进。** 每个执行器处理并确认范围内的请求，并交回超出其工单或属于其他工单的请求。只有当一条消息的所有拥有者都交回它、或结束而未处理它之后，它才成为新工作。主会话会收到跟进通知，但只能按停止工作的目的处置它们。交回的消息随后作为候选到达，并带有执行器给出的理由。
- **通过订阅和主会话协调。** 执行器为上下文跟随相关线程。主会话可以用不超过 100 词的说明发送来自其他工单的事实。订阅和说明都不授予权限。
- **只在明确引用时附加。** 消息必须指向某个工单的拉取请求、Issue、来源消息或确认，或指向一次恰好匹配一个工单的近期改动。仅话题相似不够。附加会让工单订阅整个线程；发现无关的执行器使用 `orchestrator_thread_unrelated`，且不在那里回答。
- **具体地确认一次。** 在开始实质工作之前，执行器确认每个新请求，并说明对象和预期结果，形式为原始消息下的回复。
- **通过 `human_action` 提问。** 执行器在它的报告中放入一个具体问题，主会话原样转达。如果问题是由某人的消息引发的，它会指明该消息，宿主在那里提问。否则在工单的原始会话中提问。已获授权的工作可以同时继续。
- **诚实的交付。** 开启拉取请求不等于就绪；草稿标记为未完成；没有真实改动前截图的界面改动不算就绪。
- **人的暂停持续有效，** 跨越跟进、新的执行器会话和重启，直到人解除。自动反馈永不恢复被暂停的工作。
- **示例只展示格式。** 一个示例句子不是智能体可以断言的事实。

- **按结果处理评审。** 已修复并验证的评论静默解决；被拒绝的评论给出一句理由。自动评审者的线程在得到回答后解决，每个自动评审者在每个拉取请求上最多三轮。
- **对重复工作让步。** 当人的分支或拉取请求已经包含工单的改动时，执行器用一句话关闭自己的拉取请求并指向他们的。

版本

- 每次保存都会创建一个不可变版本，包含完整配置、修订号、前驱、时间戳和来源（面板或工具）。恢复某个版本或默认值会创建另一个版本。
- 保存会指明其基础修订。冲突的保存会被拒绝，但不会丢弃草稿。
- 会话在每次模型请求时读取最新版本，包括同一轮中的后续步骤。保存不会重启、取消或唤醒会话。宿主记录每个会话使用过的修订；执行器可以读取它在其中工作过的任何修订。
- 部署设置总大小上限，包含覆盖内容。校验会拒绝未知键、保留的内置技能名、未知的覆盖部分以及缺失的模板占位符。它从不静默截断文本。与默认内容完全相同的覆盖会被移除，以便该部分跟随未来的默认内容。

面板逐页说明

面板在侧边栏「Superagent 多通道」，与官方入口互不覆盖。左侧分栏固定为：**来源 / 评估 / 执行 / 提示词 / 部署**，顶部是**概览**。 下面的截图都是实机截图，敏感信息已打码。



评估与策略

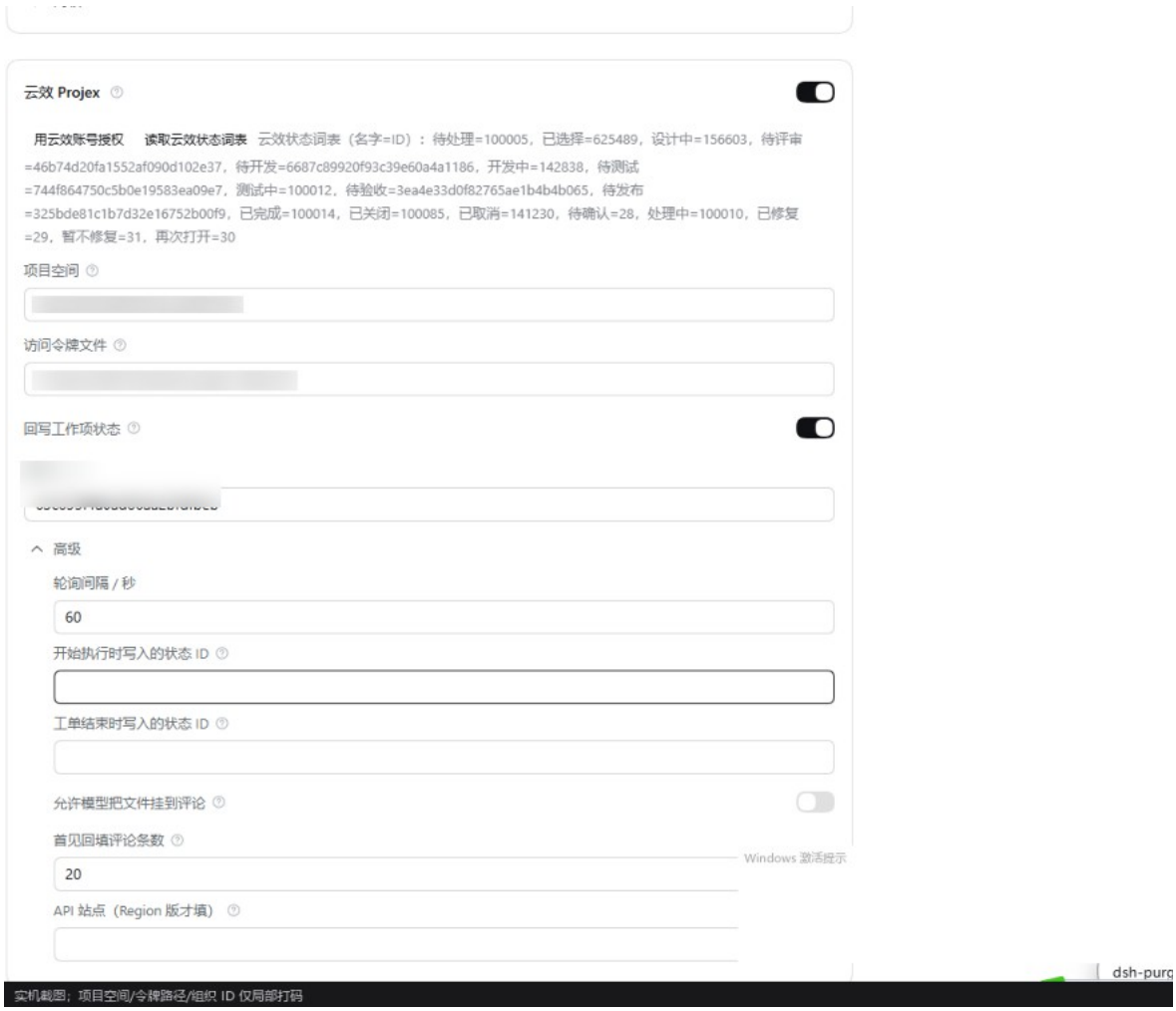
概览

- 顶部统计：待办工单、评估中、被策略拦截的消息、回复失败等；点击数字跳到对应列表。
- 卡片会指出**下一步该做什么**（例如“某来源没有配置” “某条回复连续失败”），并给出可点的修复动作。
- 「打开主 Session」直接进入编排器的对话，便于人工接手。

来源

每个来源是一张卡片：

- 开关决定是否启用；未配置的卡片会说明「来源未启用 / 等待配置」。
- 卡片上有该来源的**动作**：飞书/GitHub 的检查、微信的「启动桥 / 停止桥 / 检查」、云效的「用云效账号授权 / 读取云效状态词表」、企业微信的检查等。
- 字段按需分组，凭据优先用 0600 文件；旧的字段会被标注迁移到新的位置（不会静默丢弃）。



云效 Projex 来源卡片

以云效 Projex 为例：授权、状态词表、项目空间、令牌文件、状态回写与两个**状态下拉**（显示看板名字，存状态 ID）、轮询间隔、附件评论开关、首见回填条数、API 站点。

触发

- 按时间倒序列出收到的每条 Trigger：来自哪个来源/线程、谁发的、判定强度、去了哪条工单或为什么被忽略。
- 可点进线程查看上下文与原始消息；「重新评估」会把这条重新交给评估（不改变授权）。

评估

- 上半部是**策略**：规则开关（如 Copy fixes / Visual fixes，可自定义）、每个来源的「能发起工作的用户」名单、准入表（名单内/外 × 群里 @/未 @/私聊）。
- 下半部是**评估设置**：接单模式（关闭 / 仅观察 / 自动接单）、单次评估时限、研判模型与推理强度。
- 每次评估都会记录：判定、强度（explicit / inferred / quoted）、依据与耗时；「评估未完成」会显示原因而不是空白。

执行

- 进行中的工单、它们的执行器 Session、当前阶段（working / waiting / ended）。
- 每条工单可查看：任务说明、输入消息、执行器对话摘要、最新报告；可中断、可要求补充、可直接回复。
- 执行器只拿工单范围；文件交付在报告里以附件形式回到原线程。

提示词

- 主 Session 与执行器 Session 的每一段都可查看与覆盖；改动按版本保存，可对比历史、恢复默认。
- 六个来源技能各占一段：飞书、GitHub、微信、云效 Projex、企业微信、钉钉。



提示词

部署

- 显示当前 `stateDir`、数据库版本、健康检查结果、后台任务（桥、轮询器）的状态。
- 升级提示与迁移记录在这里；两台 superagent 并存时，各自显示自己的目录与锁。

截图与隐私

- 面板从不打印令牌；截图前仍建议检查姓名/群名/项目 ID，必要时打码后再发布。

- 本仓库 `docs/images/` 里的示例都已打码，替换示例时保持同样的打码标准。

006 部署

使用本指南安装、配置和运行 dsh-superagent。它涵盖账户、启动检查、升级和故障排查，建立在 001 领域模型 到 005 面板 的基础之上。

运行内容

- **一个专用的 DeepSeek Harness profile**，本指南中称为 `superagent`。从 Web profile 模板创建它。它加载 bundle，并在 `127.0.0.1` 上提供其面板。把它与你日常工作的 profile 分开：该插件会改变进程级的 `PATH` 和 git 设置，且其进程需要独立的重启和升级。
- **一个 agent 的 GitHub 账户。** agent 以该账户的身份发表评论、推送分支、发起 pull request，通过一个仅以该账户登录的 GitHub CLI 完成。
- **可选：** 一个用于聊天请求的飞书机器人。

插件配置保存状态目录和机器限额。其余一切通过面板操作：来源（Sources）用于仓库和账户，策略（Policy）用于允许名单，执行（Execution）用于 worker 目录和命令。

权限与隔离

bundle 为执行器定义了 `dsh-superagent-worker`，为主会话定义了 `dsh-superagent-router`——与 2.3 之前的 bundle 使用的 preset id 相同，因此从旧版本安装切换过来时，会恢复其现有的会话，而不是以 `Unknown agent preset` 失败。它们会出现在 DeepSeek Harness 的 preset 列表中，但仅供本插件使用。不要为个人会话选择它们，也不要禁用它们的配置行。应用需要它们来创建和恢复其会话。

在 Web UI 中打开一个空白托管会话时，可能会选择该 profile 的默认 preset。dsh-superagent 会在第一轮之前恢复其角色专用的 preset。

两个 preset 都以操作员的 OS 用户身份、在 `danger-full-access` 下运行，并从订阅的线程读取消息。账户 wrapper 默认选择 agent 的凭据；它们不隔离进程。worker 可以按路径执行程序并读取操作员的主目录。若需隔离，请使用一个仅包含 agent 凭据的专用 OS 账户。

前置条件

- Node.js 22.19+ 或 24+、pnpm、git 2.32+、Python 3.8+ 和 tmux
- GitHub CLI（`gh`）2.48 或更高版本。来源的 CLI 字段必须指向 `gh` 或 `lark-cli` wrapper；host 依赖它们的子命令和输出格式。
- 位于 github.com 上的仓库；不支持 GitHub Enterprise Server。要为每个工单创建 checkout，请保留一个本地 clone，prepare 命令可从中添加 worktree。若该仓库使用 Git LFS，请安装 `git-lfs`。
- 为 DeepSeek Harness 配置的模型提供方。除非评估（Eval）或执行（Execution）设置指定了其他模型，dsh-superagent 使用该 profile 的默认模型。

- 飞书：`lark-cli`（已在 1.0.90 测试）以及一个带机器人的飞书或 Lark 应用

账户

Agent 的 GitHub 账户

21. 为 agent 创建一个 GitHub 用户账户，并授予它对仓库的写权限。GitHub App 不能作为 agent，因为 dsh-superagent 读取该账户的通知。
22. 为它提供一个仅使用自身凭据的 GitHub CLI：

```
mkdir -p ~/.local/bin
cat > ~/.local/bin/gh-agent <<'EOF'
#!/bin/sh
GH_CONFIG_DIR="$HOME/.config/gh-agent" exec gh "$@"
EOF
chmod +x ~/.local/bin/gh-agent
gh-agent auth login
gh-agent api user --jq .login
```

以 agent 账户登录；最后一条命令会打印其 login。dsh-superagent 以相同方式读取该 login。

飞书机器人（可选）

23. 创建一个具备机器人能力的自建应用。授予其接收消息事件、读取和发送聊天消息、读取聊天成员等权限，然后发布应用版本。
24. 为机器人提供其独立的 `lark-cli` 配置（限定为机器人身份）以及一个 wrapper：

```
LARKSUITE_CLI_CONFIG_DIR="$HOME/.lark-cli-dsh-superagent" lark-cli config init --app-id <cli_…> --app-secret-stdin
LARKSUITE_CLI_CONFIG_DIR="$HOME/.lark-cli-dsh-superagent" lark-cli config strict-mode bot --global
cat > ~/.local/bin/lark-bot <<'EOF'
#!/bin/sh
export LARKSUITE_CLI_CONFIG_DIR="$HOME/.lark-cli-dsh-superagent"
exec lark-cli "$@"
EOF
chmod +x ~/.local/bin/lark-bot
```

第一条命令从标准输入读取 app secret。请使用独立的配置目录：`lark-cli` 每个 app id 只允许一个 profile，而你的个人配置可能已经包含该应用的用户登录。

host 会拒绝一个能以用户身份行动的 CLI。它从机器人配置中读取 app id、飞书/Lark 品牌和机器人的 `open_id`。

两份配置共享 app secret 的 keychain 条目。要仅移除机器人配置，请删除其目录；不要运行 `lark-cli config remove`，因为它会影响共享的 secret。

安装

在一台尚无 `superagent` profile 的机器上，运行以下代码块一次：

```
npm install -g @deepseek-ai/dsh@0.2.1-rc.3
git clone <this repository> dsh-superagent && cd dsh-superagent
pnpm install && pnpm build
dsh superagent --from-default-profile web --dump-config > /dev/null
dsh plugin --profile superagent add "link:$PWD"
```

上面固定的版本是公开的；其他版本必须满足 `package.json` 中的 `engines.dsh`。

第四条命令从 DeepSeek Harness Web 模板创建 `~/.dsh/profiles/superagent`。第五条安装本 bundle。设置 `DSH_HOME` 可重定位默认的 `~/.dsh` 目录。

仅在新安装时运行 profile 创建命令。如果 profile 已存在，该命令会替换它并删除其 `cordis.patch.yml`（包括机器配置）。升级时不要使用它。

机器配置

将 examples/profile.cordis.patch.yml 复制为 `~/.dsh/profiles/superagent/cordis.patch.yml`（替换该文件），并替换每一个 `。若使用飞书，还要在那里启用 dsh-superagent-multi-channel-feishu 条目。webserver` 条目设置面板的端口。

键	必填	含义
`stateDir`	否	绝对路径：数据库、锁、`worker-bin` 中的命令 shim、agent 的 git 配置，以及默认情况下 `work` 中的工单目录。默认使用已存在的 `~/.dsh/state/superagent`（2.3 之前的安装）或 `~/.dsh/state/dsh-superagent-multi-channel`。升级前请备份
`promptMaxChars`、`admissionContextMaxPages`、`admissionContextTimeoutMs`	否	保存的提示词配置的最大值，以及一次评估在什么时间内读取多少线程页

首次启动后，在 **设置 → 部署（Settings → Deployment）** 或 DeepSeek Harness 插件管理器中编辑机器限额。更改无需重启即可生效。`stateDir` 无法在那里修改。无效值或未知的配置键会阻止启动；日志会指出是哪个键。

与官方 superagent 并存安装

本 bundle 是一个独立的包，拥有自己的入口 id、agent preset 和面板入口，因此可以**安装**在另一个 superagent bundle 旁边，而不会覆盖它的行、preset 或面板槽位。

一个 Harness Host 上**每个 id 只能有一个来源类型（Source kind）**，而每个 superagent bundle 都注册相同的类型 id（`github`、`feishu`、`projex`……）。因此可以安装两个 superagent bundle，但**同一时刻只能启用一个**；同时启用两个会让第二个 bundle 的来源行报告 `Source kind already registered`。切换就是禁用其中一个 bundle 再启用另一个，且每个 bundle 保留自己的 `stateDir`，所以切回来不会改变其他任何东西。

	官方 superagent	本 bundle
包	官方包	`dsh-superagent-multi-channel`
profile 入口 id	`dsh-superagent`、`dsh-superagent-`	`dsh-superagent-multi-channel`、`dsh-superagent-multi-channel-`

	官方 superagent	本 bundle
agent preset	`dsh-superagent-worker`、`dsh-superagent-router`	**相同的 id**，这是刻意的：同一时刻只有一个 superagent bundle 被启用，切换后会恢复任一 bundle 创建的会话
面板入口	其自身的入口	侧边栏「Superagent 多通道」
状态	一个 `stateDir`	自己的 `stateDir`（默认值见下文）

要迁移本 bundle 旧版本的安装：禁用旧 bundle，启用本 bundle，并将 `stateDir` 留空——默认值会沿用现有的 `~/.dsh/state/superagent`，因此数据库、工单和游标一并带过来。如果两者都保持安装，则各自需要**自己的** `stateDir`（一个目录只容许一个实例，锁也会这样说明）。

从插件市场或 tarball 安装

- 插件管理器或市场：安装 `dsh-superagent-multi-channel@`。市场从包中读取 `dsh.bundle.patch` 和展示元数据（`locale/en.json`、`locale/zh.json`、`icon.svg`），因此条目会以自己的标题、描述和图标出现。
- 自定义插件索引（「Custom plugin indexes」bundle 的 Discover 标签页）或社区市场：将其指向已发布的包，或托管仓库 tarball 并安装该 URL——profile bundle 也可从 `…/archive/refs/tags/v.tar.gz` 安装。
- 本地开发：用绝对 checkout 目录执行 `plugin_manager` 的 `install_bundle` 会链接工作树，因此重建后无需重新安装即可见；`npm pack --dry-run` 精确显示已发布 tarball 会包含什么。

启动与停止

```
scripts/dsh-superagent start
scripts/dsh-superagent status
```

`start` 在 tmux 会话 `dsh-superagent` 中运行 `dsh --profile superagent`，对其他 profile 则为 `dsh-`。它最多等待一分钟，直到 runtime 就绪或报告某项初始化要求。首次运行时，它会打印缺失的初始化信息。它会拒绝重复的实例，包括在本脚本之外启动的实例。

`stop` 和 `restart` 请求优雅关闭，并最多等待执行器五分钟。它们从不强制杀死进程，也不触碰其他 tmux 会话。

`status` 打印进程 id、健康状况和初始化问题。对于不同的 profile 或端口，使用 `--profile` 和 `--url`，或设置 `DSH_SUPERAGENT_PROFILE` 和 `DSH_SUPERAGENT_URL`。

`status`、`start` 和 `restart` 以四种退出码之一退出：

退出码	含义
0	服务正常运行
1	未运行、运行了两个实例、无响应，或其 runtime 报告错误；重启可能修复

退出码	含义
2	正在运行，但除 runtime 之外的某个部分（如某个来源或某个策略同步 `sync:`）报告错误；重启无法修复
3	正在运行并等待初始化；重启无法修复

仍在启动中的来源算作正常运行。初始化无法检查的飞书机器人也算。

`status` 在 `errors` 下返回失败的部分及其细节，在 `setup` 下返回初始化要求。`start` 和 `restart` 将这些以行形式打印；如果服务始终未起来，它们退出码为 1。只有有意的 `stop` 会暂停自动恢复。

每个 profile 在 `.artifacts/` 下存储日志和控制文件：

- 启动日志：`dsh-superagent----.log`，其中 pid 属于该脚本。保留最新的 20 个。
- 控制文件：`control/profiles//`，包含停止标记、锁和进程 id。

打开日志中打印的 URL，然后在侧边栏选择 **Superagent**。

每小时检查

在你日常使用的 DeepSeek Harness profile 中创建每小时检查，而不是在 `superagent` 中。它必须在 agent 服务停止时继续运行。请该 profile 中的 agent 使用以下提示词创建一个每小时的计划任务，并填入 checkout 路径：

```
Check dsh-superagent: run `scripts/dsh-superagent --profile superagent status` in <checkout>. Exit 0: do nothing and say nothing. Exit 2: do not restart; tell me each part under `errors` with its detail, unless you already told me about the same ones. Exit 3: do not restart; tell me what `setup` says setup needs, unless you already told me. If it reports manually_stopped: true, leave it stopped. Otherwise (exit 1) run `scripts/dsh-superagent --profile superagent --respect-stop restart`, which exits the same way: 0 means it recovered, so say nothing; 2 and 3 are reported as above; if it exits 1 or fails otherwise, read the newest .artifacts/dsh-superagent-superagent-*.log and the Troubleshooting section of docs/006-deployment.md, fix what you can, and tell me what was wrong and what you did.
```

`--respect-stop` 保留有意的停止。只有 `stop` 会写入标记；`start` 会清除它。`restart` 不写标记，因此该检查可以恢复一次关闭超时的重启。

设置

初始化必须成功，轮询或工作才会开始。概览（Overview）列出缺失的要求并链接到相应设置。设置变更会重新运行初始化。如果你在应用之外修复了某些东西（如 CLI 认证），请选择 **重新检查（Check again）**。

25. **来源（Sources）。** 设置 GitHub 仓库（`owner/name`）和 agent CLI，例如 `/home/me/.local/bin/gh-agent`。请求通常通过 agent 账户的通知到达。提及该账户的新 Issue 或评论会在下一次轮询时到达；通过编辑现有 Issue 正文添加的提述则不会。请改为发表新评论。对于飞书，设置机器人 CLI、聊天、每个聊天的接收全部或仅提述模式，以及是否接收私聊消息。
26. **策略（Policy）。** 两条示例规则可以编辑或删除。在 **每个来源上（On each Source）** 下，添加可以请求工作的 GitHub 账户和飞书用户；在名单中有人之前，任何请求都无法变成工单。使用飞书时，可以选择一个聊天作为名单的同步来源，在启动时和点击 **立即同步（Sync now）** 时同步。
27. **执行（Execution）。** 目录根和一个 prepare 命令。要为每个工单提供仓库的 checkout：

```
   git -C /home/me/src/widgets fetch --no-tags "https://github.com/$DSH_SUPERAGENT_GITHUB_REPOSITORY.git" "$
{DSH_SUPERAGENT_GITHUB_PR_HEAD:-HEAD}:refs/dsh-superagent/$DSH_SUPERAGENT_TICKET" &&
   git -C /home/me/src/widgets worktree add -b "agent/$DSH_SUPERAGENT_NAME" "$DSH_SUPERAGENT_DIR" "refs/dsh-
superagent/$DSH_SUPERAGENT_TICKET"
```

将个人 CLI（例如登录到你本人账户的 CLI）列为被阻止的命令。

28. **提示词（Prompts）。** 设置团队段落：GitHub 披露行、审查机器人、提交作者和自动合并条件。

管理 API

面板使用一个本地 HTTP API，agent 也可以调用它。`SETUP.md` 用它保存设置，服务脚本读取 `GET state`。

向 `http://127.0.0.1:/orchestrator/api/` 发送请求，并带上 `x-orchestrator-request: 1`。host 必须是 `127.0.0.1:` 或 `localhost:`。缺少该请求头，或 `Origin` 来自其他站点，会收到 403。

API 保存拥有面板的权限。与通过主会话工具所做的保存不同，它们可以更改执行环境和初始化账户字段。

agent 可以依赖以下路由和字段。`src/app/api.ts` 中的其他路由为面板服务，并随面板变化。

请求	应答
`GET state`	每个页面读取的内容：`setup`，含 `problem` 和 `feishuProblem`，每个为 `null` 或一个含 `place`、`code`、`values` 和 `detail` 的初始化问题，以及初始化解析后的 `accounts`；`health`，每个部分一条，见下文，含 `state`（`starting`、`ready`、`paused` 或 `error`）、`detail` 和 `at`；`load`，执行器会话 `running`、工单 `queued` 和 `maxWorkers`；`paused`；以及 `masterId`
`GET settings`	`blocks`：`policy`、`eval` 和 `executor`，各自带其 `value` 和 `revision`；`sources`：来源实例，各带 `id`、`kind`、`installed`、其块的 `value`、`{"enabled": …, "config": {…}}` 和 `revision`、其 kind 的表单，以及其初始化 `problem`（若有）；`policyVersion`；和 `environment`，即执行环境的 `root` 和 `variables`
`GET sources`	每个来源的状态：`state`（`missing`、`off`、`error`、`setup`、`starting`、`paused` 或 `ready`）、其各部分的 `health`、其初始化 `problem`、最近一天的 `stats`、当天它 `filtered`（过滤）掉的触发，以及当天它 `dropped`（丢弃）的消息（最新的丢弃在前），每条带其 `channel`、`preview`、`ticket`（`id` 和 `title`，当它属于某个工单时）、其 `thread`（当存储中有它时），以及 `dropped.at` 和 `dropped.reason`、`channelGone` 或 `channelIgnored`
`PUT settings/` 带 `{"value": …, "revision": n}`	保存一个块：`saved`、该块的新 `revision`、`policy_changed` 和 `restarting`。`value` 是整个块，与 `GET settings` 所持的一致；缺失的键取其默认值
`POST executor/pause` 带 `{"paused": true}` 或 `false`	暂停或恢复处理：`paused`
`POST setup/check`	通过重启 dsh-superagent 重新运行初始化：`restarting: true`。当初始化没有可修复项时会拒绝

请求	应答
POST policy-sync/	立即同步该来源的允许名单：policy，即该来源部分按保存时的状态
GET prompts、PUT prompts 带 {"config", "baseRevision"}	读取提示词配置：current，带其 revision 和 config、defaults、sections（团队段落，按会话读取它们的顺序）、skillName（团队技能名称的模式）、builtin（团队技能不能使用的名称）和 skillPrefix（会话加载技能名称时所处的前缀）。或保存它：返回新的 revision
POST tickets//retry、POST tickets//stop	重试或停止一个工单，与面板一致

健康条目按职责分组：

条目	报告内容
runtime	应用本身
models	正在等待模型预算重试的会话
evals	最近一小时内不可用、之后既无判定也无驳回的评估
assessment、launch、delivery、relay、notice、reply	003 执行 中描述的存储工作；delivery 统计等待中的工作，而 reply 涵盖 Channel 消息
worker	执行器会话的事件处理；在其首次失败后出现
github、feishu、history	来源轮询、事件和历史恢复
sync:	允许名单同步

一次拒绝带有一个状态码和 error（其英文文本）：

- 400：无效值（issues，各带其 path）、账户检查失败（setup）、固定拒绝（code 和 values），或请求体不是 JSON 或过大。
- 404：某物不存在。
- 409：过期的 revision。
- 500：其他任何情况。

各块：

- source:github、source:feishu：{"enabled": true, "config": {…}}。GitHub 的 config 有 repository 和 cli，以及开关 observeIssues、observePulls、observeComments 和 observeCi（默认关闭），它们接收仓库中每个新 Issue、新 pull request、评论以及默认分支的 CI；飞书的 config 有 cli、以 [{"chatId": "oc_…", "name": "…", "listen": "mentions"}]（或 "all"）表示的 chats、directMessages，以及取值为 "none" 或 "mentions" 的 otherChats。
- policy：{"rules": […], "sources": {"github": {"allowed": [{"id": ""}], "bots": [{"id": "dependabot[bot]"}]}, "feishu": {"allowed": [{"id": "ou_…", "name": "…"}], "syncChat": "oc_…", "admission": {"oc_…": {"unlistedAddressed": "explicit"}}}}}。bots 列出其消息可成为推断候选的机器人。admission 行以 chat id 为键，飞书用 other 或 direct，GitHub 用 issues、pulls、comments 或 ci；未设置的单元格取其默认值（见 002 授权）。
- eval：mode、timeoutSeconds 和 model。
- executor：model、maxWorkers、directory、prepare 和 blockedCommands。处理暂停是其自身的设置 paused，由 POST executor/pause 设置。

更改仓库或 CLI 会先运行相应的初始化检查。检查失败会返回 400，详情在 `setup` 中。成功的账户更改，或在初始化待定期间的任何保存，会返回 `restarting: true`。应用重启期间 API 暂时不可用。

```
api=http://127.0.0.1:3082/orchestrator/api
rev=$(curl -s -H 'x-orchestrator-request: 1' $api/settings | jq '.sources[] | select(.id == "github")
| .revision')
curl -s -X PUT -H 'x-orchestrator-request: 1' -H 'content-type: application/json' $api/settings/source:github
\
  -d "{\"value\": {\"enabled\": true, \"config\": {\"repository\": \"acme/widgets\", \"cli\":
\"$HOME/.local/bin/gh-agent\"}}, \"revision\": $rev}"
```

初始化检查

初始化使用 DeepSeek Harness 的子进程服务，与 worker shell 一样。它会移除名称包含 KEY、PASSWORD、SECRET 或 TOKEN 的环境变量。

初始化验证 GitHub 仓库和 agent 的推送权限，写入 worker 环境，并检查 git 对 github.com 仅使用 agent 凭据。飞书初始化失败只会禁用飞书来源。

初始化写入的进程环境：

- **命令 shim** 位于 `/worker-bin`，在 `PATH` 中排在最前：`gh` 运行 agent 的 CLI；`lark-cli` 运行机器人的 CLI；被阻止的命令会失败。shim 在运行其目标前会从 `PATH` 中移除自身所在目录。
- **git 配置** 位于 `/worker-gitconfig`：替换操作员的全局和系统 git 配置。它选择 agent CLI 作为 github.com 的凭据助手，将 GitHub SSH 地址改写为 HTTPS，并定义 Git LFS 过滤器。诸如 HTTP 代理之类的个人 git 设置不适用。此处不设置提交身份，git 从不提示凭据，且 `GH_TOKEN` 和 `GITHUB_TOKEN` 会被移除。

`worker-bin` 或 `worker-gitconfig` 中一个并非由 dsh-superagent 写入的文件会终止初始化。位于某个自身配置添加了凭据的 git 仓库内的工单目录，会使该工单失败。

试一试

用一个被允许的 GitHub 账户，打开一个 Issue 或发表一条包含 `@` 的评论。在一个轮询周期内（默认一分钟），概览应显示一个工单。agent 会确认请求，在其自己的会话和目录中工作，并在原对话中交付。对于飞书，在已配置的聊天中提述机器人。

升级

29. 等到没有执行器正处于一轮的中间。
30. 备份 `/orchestrator.sqlite`。
31. 运行 `git pull && pnpm install && pnpm build`。
32. 运行 `scripts/dsh-superagent restart` 并检查其状态。

重启会结束正在运行的 Assignment，并以恢复说明启动替代者。团队文本和被覆盖的提示词部分会保留；没有覆盖的部分遵循新的默认值。升级后请复查标记为 **默认已更新（Default updated）** 的部分。

由较新版本迁移过的数据库无法被较旧版本打开。因此回滚需要升级前的备份。升级 DeepSeek Harness 本身之后，重启正在运行的实例，以便面板加载匹配的制品。

Agent preset

worker 和 router 这两个 preset 会在 5 秒后将 shell 命令移到后台。会话若没有其 preset 就无法加载。在会话使用时，请在 `cordis.patch.yml` 中保留 `dsh-superagent-worker` 和 `dsh-superagent-router` 两行；否则恢复会以 `agent-preset/not-found` 失败。

故障排查

初始化问题会以面板的语言显示在概览上，并附有修复链接；日志和 API 显示下面的英文文本。

症状	原因与修复
`The agent GitHub CLI is not set.`	在 GitHub 卡片上设置 agent 的 CLI
`The agent GitHub CLI was not found.`	设置 wrapper 的绝对路径，或 `PATH` 上的命令名
`The agent GitHub CLI cannot read its account`	用 `auth login` 登录该 wrapper
`@… cannot read …` 或 `@… cannot push to …`	授予 agent 账户对仓库的写权限
`The GitHub Source names no repository.`	在 GitHub 卡片上设置仓库 `owner/name`
一个工单以 `The prepare command failed: …` 失败	输出会指出原因，例如错误的 clone 路径；修正命令，然后重试该工单
`git does not read the github.com credential configuration dsh-superagent writes`	安装 git 2.32 或更高版本
一个工单以 `The git configuration of the repository in … sets …` 失败	从该 clone 的 `.git/config` 中移除所指名的凭据或 `http.*.extraheader` 设置
`… was not written by dsh-superagent`	将所指名的文件移出 `/worker-bin` 或 `/worker-gitconfig`，或另选一个 `stateDir`
`The bot lark-cli allows user identities`	将飞书来源指向一个仅机器人配置，如上面的飞书机器人一节
`The bot lark-cli does not work.` 或 `The bot identity of app … is …`	检查配置的 app id 和 secret
`The Feishu bot is not a member of the chat.`	在聊天设置中将机器人加入该聊天；下一次尝试会成功
`status` 退出码为 2	某个部分报告错误，重启无法修复：`errors` 指出每个部分（如管理 API 下所列）及其细节；每个部分一旦恢复正常就会清除自己的条目
`status` 退出码为 3	初始化在等待：`setup` 说明缺少什么，概览会链接到修复它的设置
`status` 退出码为 1	服务未运行、运行了两个实例、无响应，或其 runtime 报告错误；输出和最新日志会说明是哪种
`… timed out after 90 s`、`… was stopped by …`、`… exited with code …` 或 `… could not run: …`	某个 CLI 失败但未打印原因；以操作员用户身份手动运行所指名的程序，以查看它自己的消息
面板显示 dsh-superagent 无响应	它正在重启，或因配置未启动；DeepSeek Harness 日志会指出原因
`dsh-superagent configuration has keys it does not use`（日志）	移除所指名的键；消息会说明每个值现在在哪里

症状	原因与修复
日志显示该插件与正在运行的 dsh 不兼容	安装一个处于 `engines.dsh` 范围内的 DeepSeek Harness 版本
DeepSeek Harness 升级后，面板显示 "Failed to load plugins"	重启正在运行的实例
提述后没有任何反应	检查作者是否在该来源的允许名单上、来源是否开启、处理是否未暂停。在 GitHub 上，通过编辑 Issue 正文添加的提述不会被接收；请在评论中提述 agent

007 架构

这是实现层的开发者指南。它把 001 Domain model 到 006 Deployment 中的概念映射到类、接口和文件。请先从运行时和引擎开始，然后把模块表和测试表当作代码索引使用。

核心类与胶水层

核心类以小接口的形式拥有领域概念和外部边界。胶水层把工具参数、HTTP 请求、来源载荷和 DeepSeek Harness 事件转换成对这些类的调用，再把结果转换成卡片、视图和消息。

```
flowchart TB
  plugins[Source plugins] -->|SourceContext| engine
  tools[Session tools] --> engine
  http[Management routes] --> engine
  engine[Engine] -->|Repositories| store[Store]
  engine -->|Policy| policy[Policy]
  engine -->|SourceScope| registry[SourceRegistry]
  engine -->|Assessor| assessor[MasterAssessor]
  engine -->|Executor| executor[DshExecutor]
  assessor -->|decide| engine
  engine -->|Outbox| outbox[Outbox]
  executor -->|assignmentStarted, inputsDelivered, report| engine
  settings[Settings] -->|policyChanged| engine
  assessor & executor & sessions[Sessions] & outbox -->|enqueue| effects[Effects]
  effects --> store
```

`Runtime`（`src/app/runtime.ts`）构造应用、提供依赖，并负责启动和关闭。`Engine` 构造自己的内部部件。执行器和评估器会回调引擎，因此 Runtime 交给它们的是 getter，而不要求按环形顺序完成构造。

Runtime

`Runtime` 是组合根；它不含任何领域规则。其构造函数锁定状态目录、打开存储并构建每个组件。若构建失败，它会关闭存储并释放锁。

启动流程注册管理路由并运行 Setup。setup 完成后，它安装会话钩子和执行器钩子、确保主会话存在、启动策略同步、恢复被中断的派工，并启动轮询和效果。`ready` 标记启动完成。

关闭流程取消轮次、等待轮询、效果和 agent 结束、还原进程环境、关闭存储并释放锁。来源插件通过窄接口 `SourceHost` 把 Runtime 作为 `orchestrator` 服务注入。

部件	模块	作用
Setup	`src/app/setup.ts`	按工作进程 shell 运行 CLI 的方式探测它们，以此解析仓库和账号；写入并应用工作进程的环境和命令 shim，并保留 setup 问题；设置通过它检查变更后的账号，也通过它重启，而重启会再次运行 setup
Passes	`src/app/passes.ts`	一次一轮，每两秒一轮，并在任何命令要求时运行：先处理到期的效果，再执行 `Engine.evaluateDue`；来源的触发通过它抵达引擎

部件	模块	作用
Outbox	src/app/outbox.ts	引擎的 Outbox 端口，也是回执的唯一写入者：send 在同一个事务中保留消息的回执并把它的 reply 效果入队，该效果的处理程序通过 SourceRegistry.send 发送它；来源关闭时消息等待，会话已不存在或无人监听时消息被丢弃
Notices	src/app/notices.ts	把领域事件和工作进程活动转换成主会话消息。eventNotice 处理领域通知；reportRelay 立即转发提问，把普通进度留在其工单上，并转发其他报告；followupRelay 指明拥有人工跟进的那些工单。轮次进行期间到达的转发会在该轮次的下一个步骤通过 DeepSeek Harness 收件箱一起投递

Engine

Engine（src/core/engine.ts）是触发、线程、决策、工单和派工的唯一写入者。它从 src/core/ports.ts 接收这些端口：Repositories、当前策略的 getter、SourceScope、Assessor、Executor 和 Outbox。可选依赖是 prepareTicket、now 和 emit。

引擎的所有方法都是同步的。每个命令在一个事务中修改自己的聚合。evaluateDue 为它修改的每个 Eval 或工单使用单独的事务。端口在事务内把外部工作入队；适配器在提交后执行它。无效请求抛出 UserError。

引擎暴露一个入口，并委派给五个内部部件。它拥有每个命令的事务。跨部件的命令（如 attach、evaluateDue 和 policyChanged）按顺序调用这些部件。

每个部件都继承 EnginePart（src/core/part.ts），后者提供选项和共享读取。部件之间的依赖单向流动：

部件	职责	依赖
Intake	来源的触发：连同其处置一起只存一次，记录到它的线程及其关联上，扇出到已订阅的工单，并且（除非它是已抵达某个工单的人工消息）在其主线程上排队等待 Eval	工单

部件	职责	依赖
Evals	线程的 Eval：批处理，门禁的强度，参与者与依据（连同赋予它们该依据的触发，即 primary），从候选投递起算的截止时间以及暂停带来的顺延，过期 Eval 的重新入队，候选、重放和交还，策略变更时的重新入队，以及每个 Eval 最终形成的决策：接受、停止、忽略或不可用	工单
Tickets	工单生命周期：由一次接受而创建（连同其授权、订阅和首个派工），它跟随的线程及其关闭（followsDone），被附加的线程与仅为上下文而跟随的线程，输入与备注，恢复以及恢复所拒绝的消息，重试、重启、停止和结束，它丢弃的订阅，以及把策略变更通知到受其影响的工单	回复
Assignments	执行器及其工作进程所报告的内容：派工已开始、输入已抵达或未能抵达、一次重启中断了它，以及工作进程的报告及其向人提出的问题	工单、回复
Replies	通过 Outbox 进行的渠道写入：确认、答复、结果和需要关注的消息，每一种都按其允许的规则进行	无

命令使用祈使式名称，如 stopTicket；适配器报告的事实使用过去式名称，如 assignmentStarted。查询 candidate 使用名词。

调用方只拿到引擎中属于自己角色的 Pick：EngineForSources、EngineForMainSession、EngineForPeople、EngineForSettings 或 EngineForExecutor。

角色	方法	效果
来源	receive(trigger)	只存储一次触发，并回答 stored、duplicate，或在没有来源接收它时回答 filtered；把它扇出到订阅了其线程的工单，其中人工消息会立即恢复已结束的工单（除非处理已暂停），并在其主线程上排队等待 Eval，除非它是已抵达某个工单的人工消息——那种情况仍归该工单处理
来源	bindThread(ticketId, thread, own)	让一个未结束的工单跟随该来源的某个线程（例如某个 pull request 的线程），若该工单开启了它则为 own；再次绑定的已关闭线程会重新变为打开。结束工单的是线程的关闭，而不是 ready 报告

角色	方法	效果
来源	`threadClosed(thread, outcome, at)`	一个线程关闭了：每个跟随它的未结束工单都记录该关闭，并在它对自己跟随的线程处理完毕（`followsDone`）后结束：它开启的线程没有仍处于打开状态的，或者，当它没有开启任何线程时，它跟随的每个线程都已关闭。它按那些关闭中最新的一个以 `merged` 或 `closed` 结束，除非：在关闭之后恢复的工作尚未报告 `ready`；或其执行器在其最新报告（`progress` 之外）或最后一轮次结束之后，通过确认或答复承接了某个人工跟进；随后一次 `dropped` 报告会像那次关闭一样结束它
主会话	`candidate(triggerId)`	持有某个触发的那个 Eval，按其评估器所看到的样子
主会话	`candidateDelivered(evalId)`	该 Eval 的候选已抵达主会话：其截止时间从现在起算
主会话	`decide(triggerId, outcome)`	记录一个 Eval 决策，或为显式请求记录一个直接决策。接受会以原子方式创建工单、授权、派工、订阅和确认。从卡片上被忽略的推断候选，会在所给原因之前记录 `ignoredFromCard`。当 `followupOwners` 仍拥有人工跟进时，只允许停止
主会话	`attach(ticketId, triggerId, note?)`	把某个触发路由到它所指向的工单，该工单的执行器随后把它作为跟进决策，并结束持有它的那个 Eval，该 Eval 的其他触发则等待一次新的 Eval；已结束的工单会在其授权下立即恢复（除非处理已暂停）。仍由某个执行器决策的跟进会被拒绝，被路由回交还它的那个工单的跟进同样会被拒绝
主会话	`requestStatus(ticketId)`	向等待中的执行器会话询问其状态，并回答是否发出了询问
主会话	`note(ticketId, text)`	把来自其他工作的事实作为文本输入交给工单的执行器，这不授予任何权限
主会话、执行器	`reply(request)`	为其用途所指名的渠道排队一条消息：`acknowledgement`、`answer`、`result` 或 `attention`
人	`stopTicket(ticketId, reason?)`	人的停止操作，它会撤销授权

角色	方法	效果
人	`retryTicket(ticketId)`	结束某个工单已停滞的派工并启动一个新的派工；该工单是在 `failed` 或 `blocked` 报告之后等待人工介入的，或其输入投递失败（`canRetry`）
人	`reevaluate(triggerId)`	把已存储的触发排队等待一次新的 Eval
人	`unsubscribe(thread)`	丢弃某个线程上的所有订阅，此前跟随它的工单不再跟随它；每个工单继续决策它在那个线程上收到的跟进
人	`dismiss(decisionId)`	忽略 `unavailable` 决策所显示的问题，该决策会记录这次忽略
执行器	`assignmentStarted(ticketId, assignmentId)`	派工开始运行；其等待中的输入被发出
执行器	`inputsDelivered(ticketId, assignmentId, inputIds)`	输入已抵达该派工的执行器会话，在其中各自等待某个步骤来取用它
执行器	`inputsRead(ticketId, inputIds)`	执行器会话把承载这些输入的消息收进自己的历史（一个 `user/message` 会话事件），其模型由此拥有它们；某个步骤曾认领它们但在之前被取消或失败的，不算数。只要其后跟随一次报告或一轮次的结束（重启之后亦然），每个输入就算作已读，且其工单的结束并不会让它失效；在读取记录存在之前投递的输入，只要在其投递之后跟随了一次报告或一轮次的结束，就算作已读
执行器	`turnEnded(ticketId, assignmentId)`	执行器会话的一轮结束了，在报告之后亦然，此时后台已无为其运行的东西，或它正交接给人。某个步骤此前取用的内容即已读（`inputsRead`）
执行器	`deliveryFailed(ticketId, assignmentId, error)`	投递输入失败；工单会一直显示该失败，直到某次投递成功、新的派工开始或工单结束
执行器	`report(ticketId, report)`	记录一份报告及其向人提出的问题；`dropped` 会结束工单，`ready` 会完成一个不跟随任何线程的工单，并按线程结束的方式结束一个已处理完其所跟随线程的工单
执行器	`markUnrelated(ticketId, thread, reason)`	只丢弃该工单对某个被附加线程的订阅（它判定该线程无关），并连同原因交还它在那个线程上决策的跟进
执行器	`followForContext(ticketId, thread)`	为获取上下文，把一个未结束的工单订阅到一个相关线程：该线程的消息会到达工单，但不给它跟进强度，也不会恢复它

角色	方法	效果
执行器	`handback(ticketId, triggerId, reason)`	记录该工单交还了一个抵达它的触发，并带上其候选所携带的原因；若确实由该工单决策，则在不再有其他工单决策它时把它排队等待一次新的 Eval；它回答是否已入队。一个工单对每个触发只交还一次，并保留其订阅
执行器	`assignmentInterrupted(ticketId, assignmentId)`	宿主停止时某个派工正在运行：在工作中途被中断的派工，或在其最新报告之后收到输入的派工，会带着一条恢复备注以及那些已投递给它但执行器会话未取用的输入重新开始，如同一次重试；此后已报告过的派工保留其会话。DshExecutor 在启动时（`resume`）逐个报告它们，最旧的工单优先，并把处于等待中的启动再次入队
轮询	`evaluateDue()`	除非处理已暂停：为那些排队触发在等待的线程启动 Eval，每个线程一个，期间排队的触发作为一个批次；把已过截止时间的 Eval 的触发重新入队，连续两次，第三次则将其记录为 `unavailable`；并恢复那些在暂停期间收到人工消息的已结束工单，前提是其授权仍然有效。候选尚未抵达主会话的 Eval 不会过期。处理暂停期间没有 Eval 过期，一旦恢复，每个截止时间都按暂停时长顺延
设置	`policyChanged(from, to, effects)`	告诉每个受影响的活跃工单该变更如何影响它（`src/policy.ts` 中的 `policyEffects`：它被接受时所依据的规则已变更或不再适用、其请求方可能已无法发起工作，或主动工作已不再被接受），并按新版本重启进行中的 Eval

引擎把批处理、授权、扇出、恢复和关注生命周期这些规则保持私有。几个共享谓词让这些规则保持一致：

- `waitsForPerson`（`src/core/model.ts`）：最新的派工报告了 `blocked` 或 `failed`，且此后没有人工消息到达。只要该条件成立，自动输入和失败的启动就一直被挂起。
- `followupOwners`：仍对其自己线程上收到的人工跟进负责的那些工单。工单可以是活跃的，也可以是未经人工停止而结束、正等待恢复的。交还该消息即结束该归属关系。
- `settled`：某个决策已解决了该请求；停止、不可用、已路由和被拒绝的停止这几类决策不算数。
- `ticketStage`（`src/core/stage.ts`）：实现 003 Execution 中的阶段表。

工单结束时，未读的跟进会回到 Eval，除非另有工单仍拥有它们，或另有处置已解决它们。已确认、已答复和已交还的消息被排除在外。对于没有确认处置的较旧记录，发件箱回执提供该事实。最终报告还涵盖派工开始时正在等待的消息，包括恢复它的那条消息。停止工单的那条消息不会被自动重新评估。

每次变更后，引擎从 `src/core/ports.ts` 的 `DomainEvents` 中发出一个事件：

`trigger.received`、`eval.started`、`decision.recorded`、`ticket.created`、`ticket.resumed`、`ticket.ended`、`assignment.started`、`assignment.ended`、`thread.subscribed`、`thread.unsubscribed`、`trigger.requeued` 或 `policy.changed`。

Runtime 把 `ticket.ended`、`trigger.requeued` 和 `thread.unsubscribed` 转成通知。投递给工单的人工 `trigger.received` 会变成一次跟进转发。`pullEndNotices` 在工单以 merged 或 closed 结束时，用一个稳定的 id 发送一条聊天通知。其他事件目前没有消费者。

Model

`src/core/model.ts` 存放领域类型，`src/core/stage.ts` 存放工单阶段。决策把原因记录为 `reason`，即 `Reason` 的列表（`src/core/reasons.ts`）：每个 `Reason` 是一个 `code` 及其 `values`，并且对于批次中的单个触发，还带其 `triggerId`。

核心自带的代码（如 `timeout`、`policyChanged`、`routed` 和 `startedTicket`）都在 `REASONS` 中，并带有 agent 通过 `reasonText` 读取的英文表述；门禁或来源作用域给出自己的代码，如 `not_a_member`；`said` 承载评估器或人写下的文字。面板按 `reason.` 为每个代码取词，被来源过滤掉的触发也以同样方式记录该来源的代码。

工单对每个事实只保留一份。它没有 `ended` 时即为未结束，`ended` 记录它何时、由谁、为何以及以什么结果结束；它最后说的话是它最新的派工报告（`latestReport`）；它的执行器会话以它命名（`src/executor/dsh.ts` 中的 `workerSessionId`）。

它的字段都是有类型的：`origin`，即赋予其 Eval 批次强度的那个可准入触发以及该批次本身（`triggerId`、`triggerIds`）；`execution`，执行器所选的内容（`name`、`directory` 和 `assignment`）；`delivery`，最后一次失败的输入投递（`error`、`at`），一直保留到某次投递成功、新的派工开始或工单结束；以及 `follows`，来源让它跟随的线程，每个为 `{thread，own，closed?}`：工单开启它时为 `own`，一旦关闭则 `closed` 连同 `at` 和 `outcome`。只有工单会写 `follows`；GitHub 来源用 `pullsOf` 和 `pullOf`（`src/sources/github-pull.ts`）读取它的 pull request。

承载触发的输入会记录它是否由人写成，`waitsForPerson` 会读取这一点。线程还在 `context` 中列出仅为上下文而跟随它的已订阅工单，并在 `expired` 中累计其排队触发的 Eval 连续过期的次数；一个评估记录其 `primary` 触发以及其候选何时被 `delivered`。

触发的处置记录它通过扇出或附加而抵达的工单、某个人发起的重新入队、每个交还它的工单（连同原因），以及每个其执行器确认或答复了它的工单。

授权的 `basis`（`MandateBasis`）记录门禁如何准入它的触发。重新校验读取接收方的 `source`、`scope` 和 `row`，被列出的 `sender`、匹配到的 `rules`，以及 `via`：

via	准入依据
`addressed`	一个被允许的发送方对 agent 发话
`followup`	某条消息到达了一个有工单的线程
`admission`	一个准入表单元格；`observation` 标记推断出的候选
`bot`	一个被列出的 bot 产生了推断出的候选

触发和线程的 `meta` 属于来源。核心不解释它们。

Policy

`src/policy.ts` 定义设置模式、授权规则和评估规则。`settingsPolicy(settings, scope)` 每次使用时都返回一个新的、无状态的 `Policy`：

- `version`、`paused` 和 `evalTimeoutMs` 描述它的当前状态。
- `gate(trigger, thread, tickets)` 返回准入强度和依据。
- `revalidate(mandate)` 检查既有授权是否仍然有效。
- `admitsInferred(rules)` 仅在有启用的匹配规则的自动模式下允许主动接受。

`assessmentOutcome(assessment, settings, strength)` 把模型的评估转换成引擎的结果。它要求对每条启用的规则都有判断、方向已定、存在匹配的规则，以及意图的证据。在自动模式之外，推断出的接受会变成一条“would accept”记录。

`policyEffects(tickets, old, now, scope)` 指出一次变更影响哪些未结束的工单以及如何影响。`Engine.policyChanged` 只通知这些工单。`policyVersion` 对规则、接受模式以及各来源的策略部分（含 bot 列表）做哈希。

Store

`Store`（`src/app/store.ts`）拥有一个 SQLite 数据库。Runtime 在打开它之前先获取目录锁，从而防止第二个进程迁移或写入同一份状态。

它的接口按用途分组：

- **领域仓储：** `atomic`，加上 `triggers`、`threads`、`tickets` 和 `decisions`，每个都有 `get`、`save` 和列表查询。带索引的工作集是 `threads.active()`、`tickets.open()`、`tickets.active()` 和 `read.subscribedThreads`。
- **读取查询：** `read` 通过 `threadTriggers`、`triggerStats`、`search` 和 `droppedReplies` 之类的方法为视图、工具和来源轮询服务。这些方法从不写入。
- **来源元数据：** `threads.describe` 和 `threads.patchMeta` 更新来源拥有的线程元数据。

- **设置：** `settings`、`saveSettings`、`blockRevision` 和 `settingsHistory`。设置和提示词使用带编号的 `Versions`（`src/app/versions.ts`）。
- **发件箱回执：** `replyStatus` 和 `receipts`，只由发件箱写入。回执操作有 `get`、`waiting`、`behind`、`add`、`settle` 和 `withdraw`。`behind` 检查同一工单和同一会话的较早消息是否在失败次数少于三次的情况下仍在等待。被撤回的消息以 `channelGone` 或 `channelIgnored` 结清。读取查询通过 `failingReplies`、`droppedReplies` 和 `droppedSince` 暴露失败和丢弃的回复。
- **带类型的键值记录：** `get`、`set`、`delete` 和 `keys`，使用来自 `KV` 的条目。

`afterCommit` 只在事务提交后运行动作，回滚后绝不运行。Runtime 以记录存储时的形态读取它们；schema 11 迁移在启动时转换较早的受支持记录。

Effects

`Effects`（`src/app/effects.ts`）是持久化的副作用队列。命令在其事务内调用 `enqueue(kind, key, payload)`；执行只在提交之后开始。

每个 kind/key 组合对应一行。对同一个仍在等待的 key 再次入队会替换其载荷并使其立即到期，且不改变队列顺序。`handle(kind, run, options)` 为每个 kind 注册一个处理程序。它按从旧到新的顺序处理到期的行，直至并发上限。

失败会以指数退避重试。抛出 `Later` 会推迟工作且不计为失败，例如在没有可用工作进程槽位时。`start` 在 setup 之后开始处理，`drain` 在关闭期间停止处理但保留待处理的行，`pending/idle` 暴露队列状态。

类型	处理程序	作用	同时并发	重试
`assessment`	MasterAssessor	把一个 Eval 的候选卡片引导进主会话的轮次，然后告诉引擎它已投递，从而启动该 Eval 的截止时间	4，暂停期间为 0	2 秒起，逐次翻倍至 60 秒
`launch`	DshExecutor	启动一个待处理的派工：准备命令、执行器会话、派工消息	空闲的工作进程槽位数，暂停期间为 0	模型提供方或网络的暂时性失败在 5 秒后重试，逐次翻倍至 300 秒，共三次；其他任何失败，以及准备命令失败，都会使其工单失败。因暂停或槽位已满而被挂起的，会在 5 秒后重试
`delivery`	DshExecutor	把工单等待中的输入按顺序交给其正在运行的派工	4，暂停期间为 0	2 秒起，逐次翻倍至 60 秒；等待工作进程槽位的，在 2 秒后重试

类型	处理程序	作用	同时并发	重试
relay	Sessions	把工作进程的报告或提问，或某个人的跟进，引导进主会话的轮次	1，按队列顺序	2 秒起，逐次翻倍至 60 秒
notice	Sessions	向主会话注入一条通知而不唤醒它	1，按队列顺序	2 秒起，逐次翻倍至 60 秒
reply	Outbox	通过其来源为某个渠道发送消息，且在该工单发给该会话的、失败未达三次的较早消息之后；撤回该消息会取消它	1，暂停期间为 0	5 秒起，逐次翻倍至 300 秒；来源关闭或排在较早消息之后的，在 5 秒后重试

每个 kind 都在自己的名字下有一个健康条目。当其某个效果连续失败三次时它会报告错误，但 `delivery` 除外——它的失败改为显示在其工单上。转发保持顺序，通知也是如此；一个失败的会阻塞其后同类型的项。

Sources

来源通过 `SourceKind` 扩展宿主，`SourceKind` 定义在 `src/sources/kind.ts`。一个 kind 声明它的身份、标题、描述、配置 `schema` 和 `fields`、派生的 `facts`，以及 `health` 条目。可选行为包括 `scope`、`channel`（`allowed`、`send`、`link`）、`threadContext`、`find`、`policy`、`environment` 和 `refreshLabels`。

`SourceRegistry`（`registry.ts`）拥有已注册的 kind 和已存储的实例，并实现引擎的 `SourceScope`。注册会在 `HealthBoard` 上认领健康条目；重复认领和保留的 `sync:` 名称会被拒绝。SourceContext 只通过其注册所认领的条目写入，因此注销之后的报告不会产生任何效果。

注册表的配置方法包括 `kind`、`instances`、`instance`、`validate`、`validatePolicy`、`policyPart`、`channelAllowed`、`declarations`、`environments`、`find` 和 `scope`。`find` 会询问每个支持搜索的 kind。

通用代码通过注册表访问来源行为。`send(message, commands)` 调用该 kind 的渠道，并返回 `sent`、`gone`，或在该 kind 未安装时返回 `waiting`。`link` 解析其渠道链接。`threadContext` 把 kind 的读取器与其配置配对，当该 kind 没有读取器时不返回任何内容。存储的配置无效会使该实例被禁用。

插件收到的是 `SourceHost`：注册表以及 `context(kind)`。kind 自身使用更窄的 `SourceContext`，它由 `src/app/source-context.ts` 中的 `sourceContextOf` 构建。

该上下文提供部署与命令、setup 就绪状态（`ready`、`configured`、`problem`）、配置、`receive`、`atomic` 以及带类型的 `state`。它还暴露线程读取与元数据写入、工单读取与绑定/关闭命令、触发读取、经发件箱的 `notify`，以及针对已声明健康条目的 `report`。

DshExecutor

DshExecutor（src/executor/dsh.ts）为每个工单提供一个会话和一个目录。start 和 deliver 分别把 launch 和 delivery 效果入队。stop 通过 Store.afterCommit 取消轮次，因此被回滚的结束不会停掉工作进程。

prepare(ticket, outcome) 提供引擎的 prepareTicket 钩子：短名称、目录和 ticketExecution 派工文本。workerSessionId(ticket.id) 派生出会话 id。

WorkerSlots 在启动之前，或在唤醒空闲工作进程的投递之前预留容量。人工消息可以在 maxWorkers 之外多用一个槽位，唤醒优先于启动。activeCount 为面板负载提供数据。

提供方和网络的暂时性失败会重试三次。启动沿用效果的退避；失败的轮次会在 15、30 和 60 秒后收到续接消息。inputMessage 依据当前工单状态渲染每个输入，mandateInstructions 加上推断性工作的匹配规则限制。

install 订阅会话事件；dispose 取消订阅。轮次结束通常报告等待中，但在就绪之后或在需要人工操作期间除外。错误则报告 failed。启动时，resume 通过 assignmentInterrupted 报告被中断的派工，并重新入队那些不等待人工的待处理启动，最旧的工单优先。

MasterAssessor

MasterAssessor（src/app/assessment.ts）让主会话评估主动候选。request 把评估效果入队；release 在其 Eval 结束时丢弃缓存状态。

它的 context、read 和 decide 方法服务于 orchestrator_eval、orchestrator_eval_read 和 orchestrator_eval_decide。它们通过 SourceRegistry.threadContext 提供有界的线程上下文、仓储上下文，以及决策提交。没有上下文读取器的 kind 只提供触发和批次。

assessmentOutcome 把模型的回答转换为 Engine.decide 所需的形式。Evals 会再次用当前策略校验推断出的接受。

Settings

Settings（src/app/settings.ts）读写带修订号的设置块。current 和 view 读取它们；save(block, value, revision, origin) 校验一次保存；pause(paused, origin) 修改独立的 paused 设置。

每次保存都声明其来源：panel、tool 或 sync。只有 panel 能修改执行环境或 SETUP_FIELDS（src/deployment.ts）中列出的字段。

策略版本变更会计算 policyEffects、记录哪些工单被通知，并调用 Engine.policyChanged。syncable 和 sync(id) 刷新某个来源的策略部分。refreshLabels(id) 刷新显示名称。

Prompts

Prompts（src/app/prompts.ts）把配置存储为 Versions。current、version 和 history 读取它们；validate、preview 和 save(value，baseRevision，source) 检查并保存它们。compose 和 skills 构建会话文本。

builtInParts 列出覆盖键、角色、默认文本和必需的模板占位符。每个覆盖项存储团队 text 和一个 base：其原始默认值的 builtInBase 键。保存会把该默认值保留在 KV.prompts.builtIn 之下。defaultChanged 随后报告已变更的默认值、可用时它们的旧文本，以及已移除的部件。

校验会从已保存的覆盖项继承省略的 base，丢弃已保存数据中仍然存在的已移除部件，并拒绝空白文本或其他未知键。这使保存工具能对它读取到的配置做往返保存。environmentSection 和 policySection 只使用自有键，一次性替换模板值。没有覆盖项的配置使用默认值。

Sessions

Sessions（src/app/sessions.ts）为两种角色管理 DeepSeek Harness 会话。ensure 用其角色的模型创建一个会话。install 跟踪 agent 事件，并为受管会话注册提示词、技能和工具。

prompt 每个请求 id 只投递一次。relay 和 notify 把主会话消息入队，由 relay 和 notice 效果处理。src/app/notices.ts 定义它们的内容；delivery 检查消息是否已抵达。

每次模型请求都会把会话绑定到当前的提示词修订版。它用过的修订版被持久保存：promptVersion 标识生效的那一个，mayRead 限制工作进程可以查看哪些版本。注册情况按 agent 分别记录在一张映射表中。

status 和 promptVersion 为提示词页面服务。保存经由 Prompts.save 并通知主会话。modelRetry 暴露模型预算的重试状态。

胶水层

胶水层	转换内容
src/app/runtime.ts	组合根：用各部件所使用的接口构建它们，注册路由和来源宿主，并启动和停止 dsh-superagent
src/app/notices.ts	把领域事件和工作进程报告转换成给主会话的通知和转发
src/app/tools/main.ts、src/app/tools/executor.ts	把主会话和执行器会话的工具参数转换成核心类的调用，并把它们的结果转换成卡片
src/app/http.ts、src/app/views/、src/app/api.ts	把 HTTP 请求转换成调用，把存储读取转换成面板的视图；api.ts 为面板给每条路由定类型
src/app/resources.ts、src/app/links.ts、src/app/cards.ts	把资源名转换成读取、把来源消息转换成链接、把记录转换成给模型的紧凑卡片
src/sources/github*.ts、src/sources/feishu*.ts、src/sources/poll.ts	把 GitHub 和 Feishu 载荷转换成触发和线程事实，把消息转换成 CLI 调用
src/deployment.ts、src/setup.ts、src/infra/	setup 探测与问题、CLI 传输、agent 的进程环境

依赖规则

- **显式、窄的依赖。** 每个类都收到一个接口，该接口指明它用到的东西，大多表现为其他类的 `Pick` 类型：核心类用 `ExecutorHost`、`AssessorHost`、`SessionsHost`、`SettingsHost`、`SetupHost`、`PassesHost` 和 `OutboxHost`，胶水层用 `HttpHost`、`MainToolsHost`、`ExecutorToolsHost`、`ResourceHost`、`LinkHost` 和 `SourceContextDeps`，而引擎则以调用它的那个角色的 `Pick` 形式出现。Runtime 只在这种接口之后传递自身。来源插件注入 `orchestrator` 服务，其类型把它命名为 `SourceHost`：注册表以及 `context(kind)`；其 kind 的代码只看到自己的 `SourceContext`。
- **谁能 import 谁。** `src/core` 除 Node 的哈希外不 import 自身之外的任何东西，也不做任何 I/O。`src/sources` 不从 `src/app` import 任何东西。面板只从宿主侧 import 类型，以及 `src/admission.ts`、`src/limits.ts`、`src/setup.ts`、`src/navigation.ts`、`src/sources/host.ts`、`src/core/errors.ts`、`src/core/reasons.ts` 的纯值和 `src/app/api.ts` 的过滤列表。
- **每种状态只有一个写入者。** 每种状态只有一个写入者，其他人读取它或向该写入者询问。

状态	写入者
触发、线程、决策、工单、派工	引擎
来源的读取位置，以及它在其线程上保留的 meta（`label`、`url`、`facts` 及其自有键）	该来源，通过 `SourceContext`
设置及其历史	设置
提示词版本	提示词
效果行	效果
发件箱回执	发件箱：引擎的消息和来源的 `notify` 都经过它的 `send`，并由它的 `reply` 效果标记
健康条目	在 `HealthBoard`（`src/infra/health.ts`）上认领它的那个部件，该板拒绝第二次认领：`runtime` 是 Runtime，轮询通过它上报；`evals` 是每轮巡检之后的 Runtime；`models` 是会话；每个效果 kind 是效果；`worker` 是 DshExecutor；某个来源 kind 的条目是该 kind 对应来源的 SourceRegistry；`sync`：是设置

来源只通过 `SourceContext.tickets.bindThread` 和 `threadClosed` 修改工单，这两个方法会调用引擎。

- **请求与事实。** 请求某事的方法以祈使动词加宾语命名，如 `stopTicket` 或 `bindThread`，并且可能被拒绝；记录适配器做了什么的方法用过去式命名，如 `assignmentStarted` 或 `deliveryFailed`。每个宿主都以自己需要的角色类型接收引擎。
- **带类型的字段，来源数据放在来源之下。** 代码读取工单和授权的带类型字段。来源不在工单上保留任何东西：工单跟随它的哪些线程是工单的 `follows`，而它关于某个触发或线程所保留的内容在它们的 `meta` 中。
- **固定的拒绝。** 人或 agent 可以触发的拒绝是来自 `src/core/errors.ts` 中 `USER_ERRORS` 的 `UserError(code, values)`。管理 API 对 `missing` 代码返回 404，对其他代码返回 400，面板用其语言为每个代码取词。setup 问题是 `SetupError` 代码（`src/setup.ts`），固定的健康文本是 `HealthCode`（`src/sources/host.ts`），取词方式相同。

来源特定的依赖

有些通用模块仍然直接引用 GitHub 或 Feishu：

- 发件箱应用 Feishu 聊天字数上限。
- `src/app/links.ts` 解析 Feishu 消息链接和 `repliedTo`。它和 Runtime 用 `feishuSettings` 读取设置。
- `src/app/resources.ts` 读取 `feishu/` 和 `github/` 的原始资源；`src/app/cards.ts` 为工单卡片使用 `pullOf`。
- Runtime 为 merged 或 closed 的工单发送 `pullEndNotices`。
- 执行器工具用 `readPull` 和 `claimPull` 绑定被报告的 pull request，使用 Runtime 暴露的 GitHub 上下文。
- MasterAssessor 用 `readRepositoryPath` 读取仓库文件。
- Sessions 和 Prompts 只为 GitHub 和 Feishu 注册内置技能。迁移会转换它们的 schema-10 记录。
- Setup 只探测 GitHub 和 Feishu 的账号字段（`SETUP_FIELDS`）。只有这些来源通过 `SourceContext.problem` 收到账号 setup 问题。

模块

路径	职责
`src/index.ts`	主插件入口：读取机器设置，启动 Runtime，并为其提供保存某个来源账户时所用的重启
`src/deployment.ts`	插件的 `Config` 与 `Machine`；`resolveDeployment` 和 `resolveFeishuBot` 从来源设置中读取仓库和账户，每次 CLI 均按执行器 shell 的方式运行；`SETUP_FIELDS` 列出设置过程所读取的来源字段。代码中不指定任何个人、账户或组织
`src/setup.ts`	设置问题：`SetupError`，带有错误码、修复它的位置，以及日志、主会话和面板共用的英文文本
`src/policy.ts`	策略：按块划分的设置 schema、策略版本、门禁以及 Mandate 复核
`src/admission.ts`	准入表的值、每个单元格的选项和默认值，与面板共用
`src/limits.ts`	用户所设值的限制，由设置 schema 校验，并由面板的字段提供
`src/kv.ts`	`KV`：每一族 `kv` 记录及其键、值类型和所有者，以及存储和 `SourceContext.state` 使用的 `Key` 和 `Family` 类型
`src/app/lock.ts`	状态目录的锁：每个状态树一个 dsh-superagent，stale 窗口足够短，使重启时不会认为上一进程的锁仍然有效
`src/navigation.ts`	一旦宿主目录列出某个 DeepSeek Harness 会话，便从面板打开它

路径	职责
src/core/model.ts	领域类型，其中包括带类型的工单和 MandateBasis，以及 threadKey、ticketOutcome、reached、followupOwners、settled、waitsForPerson、liveTicket、runningAssignment、latestAssignment 和 latestReport
src/core/ports.ts	端口：Repositories、Policy、SourceScope、Assessor、Executor、Outbox，以及领域事件
src/core/engine.ts	引擎，核心的入口点，以及角色接口 EngineForSources、EngineForMainSession、EngineForPeople、EngineForSettings 和 EngineForExecutor
src/core/part.ts	EnginePart，引擎各部分的基类，以及 EngineOptions
src/core/intake.ts	Intake：将来自来源的触发归入其工单，并在某人的消息未到达工单时将其归入 Eval
src/core/evals.ts	Evals：线程的 Eval 及其决策
src/core/tickets.ts	Tickets：工单生命周期、工单所跟随的线程、订阅和输入
src/core/assignments.ts	Assignments：执行器的事实及其工作者的报告
src/core/replies.ts	Replies：引擎对 Channel 的写入
src/core/stage.ts	ticketStage 和 canRetry：003 Execution 中的工单阶段
src/core/errors.ts	UserError 和 USER_ERRORS
src/core/reasons.ts	Reason、REASONS 和 reasonText：核心记录某个决策或搁置某个触发的原因，按错误码给出
src/core/ids.ts	key：由输入派生的 id，相同输入得到相同 id
src/core/memory.ts	MemoryRepositories：内存中的存储仓库，按存储的顺序排列并具备其嵌套回滚，用于领域测试
src/core/index.ts	核心的导出
src/app/runtime.ts	Runtime，组合根，及其启动与停止
src/app/setup.ts	设置，以及 setupChanged——某次保存是否改变了设置过程所读取的字段
src/app/passes.ts	Passes，pass 调度器
src/app/outbox.ts	Outbox：引擎的 Outbox 端口、回执，以及发送这些回执的 reply 效果
src/app/notices.ts	主会话的通知与转发：eventNotice、reportRelay、resultRelay 和 followupRelay
src/app/store.ts	存储，以及 Receipt——outbox 中的一条消息
src/app/store-read.ts	storeReads：供视图、工具和来源轮询使用的存储读取查询，从不写入
src/app/migration.ts	schema 11 迁移：从 schema 10 一步迁移，将每条存储记录写为其当前形态
src/app/effects.ts	效果
src/app/versions.ts	Versions：以带编号的快照保存的记录，并指向当前快照，用于设置和提示词
src/app/cache.ts	TextCache：在内存中按大小限制保存的文本，最先丢弃最久未使用的
src/app/settings.ts	设置

路径	职责
src/app/settings-history.ts	policyHistory：每次保存的设置修订，以及它相对上一修订的改动
src/app/prompts.ts	提示词、builtInParts（团队可覆盖的内置部分），以及 promptParts（某个角色的提示词各部分的顺序）
src/app/context.ts	内置提示词文本（从 prompts/ 读取），以及会话所读取的记录条目
src/app/skill-files.ts	打包文本：promptFile（将文件中的单句行合并为段落），以及随包附带的技能
src/app/sessions.ts	会话
src/app/delivery.ts	AgentDelivery：按请求 id 向 DeepSeek Harness 代理投递一次消息，跨恢复的会话亦然
src/app/retry-health.ts	RetryHealth：哪些会话在等待模型预算重试，供 models 健康条目使用
src/app/role-models.ts	解析 Eval 和执行器的模型，并为新会话以其角色对应的模型进行初始化
src/app/assessment.ts	MasterAssessor、评估 schema，以及 orchestrator_eval 所回答的窗口
src/app/execution-environment.ts	执行环境：其变量、工单的值、目录根，以及提示词部分的各行，由 src/app/prompts.ts 填入其模板
src/app/source-context.ts	sourceContextOf：每种来源类型的 SourceContext
src/app/links.ts	sourceLink（触发消息所在的页面），以及 candidateCard
src/app/cards.ts	供模型使用的触发、工单、线程和决策的紧凑卡片、工单卡片状态的封闭集合 TICKET_STATES，以及 textPage
src/app/resources.ts	RESOURCES 和 readResource：orchestrator_read 为各角色读取的内容
src/app/tools/tool.ts	Tool 类型、readTool、两种角色的 orchestrator_read，以及 AcknowledgementSchema
src/app/tools/main.ts	主会话的工具
src/app/tools/executor.ts	执行器会话的工具
src/app/http.ts	/orchestrator/api 路由：同源检查、请求限制，以及每种失败对应一个状态码
src/app/api.ts	Api，每条路由及其参数、查询、请求体和响应，以及列表的过滤器；它仅导入类型，因此面板可将其打包
src/app/views/text.ts	视图文本的两种形式：preview（单行纯文本）和 markdown（不含 HTML）；plain 和 readable
src/app/views/records.ts	视图共用的行：触发、工单、线程、决策和 Channel 链接，以及 ownersOf——在一次请求读取时后续跟进的所有者
src/app/views/tickets.ts	按阶段划分的工单列表、工单页面、Mandate 依据事实以及执行器的负载
src/app/views/threads.ts	线程列表、线程时间线、搜索以及决策列表
src/app/views/sources.ts	来源状态：每个来源实例的状态、其最后一天的计数，以及当天它过滤和丢弃的内容

路径	职责
src/app/views/settings.ts	设置页面的视图：每个块及其修订、每个来源的设置及其类型的表单，以及执行环境
src/app/views/overview.ts	概览：需要人处理的工单、问题、最后一天的流转以及执行器的负载
src/executor/dsh.ts	DshExecutor、WorkerSlots、ticketExecution、inputMessage 和 mandateInstructions
src/executor/background.ts	工作者在 DeepSeek Harness 中的后台工作、其会话拥有的 shell 作业（ctx.jobs）及其子代理（ctx.subagents）：runningJobs、untilQuiet、killJobs、collectJobs、stopSubagents、stopBackground、until，以及 cancelled——因停止其子代理而取消轮次的工作者
src/infra/commands.ts	Commands，CLI 传输：run；api 和 pages（限于已配置的仓库并经过校验）；lark（作为机器人）；git。CliError 携带命令输出所报告的错误码，例如某个 GitHub HTTP 状态；无任何输出的命令以其结束方式失败，绝不以它的参数失败；validItems 跳过并统计列表中的格式错误项
src/infra/environment.ts	代理的进程环境：命令 shim、git 配置和凭据助手、runAsWorker，以及对环境和工单目录的检查
src/infra/paths.ts	包的根目录，即打包文件所在位置
src/infra/text.ts	字数统计和聊天消息限制、fill 和 sameName
src/infra/health.ts	HealthBoard，每个部分的健康条目，各由其唯一写入者认领
src/infra/secret-file.ts	机密文件的唯一"仅所有者"规则：readSecretFile 和 requireOwnerOnly，在平台具备 POSIX 位时执行 0600 检查，报出被拒文件实际拥有的模式，并将 Windows 交由 ACL 处理
src/infra/hosting-api.ts	代码托管 HTTP 客户端：HostingApi 和 createHostingApi，令牌从其 0600 文件读取，PRIVATE-TOKEN，白名单化的 /projects 路径，以及 30 秒超时
src/infra/yunxiao-api.ts	云效 HTTP 客户端：YunxiaoApi、createYunxiaoApi 和 YunxiaoHttpError，每次调用都读取其令牌文件，/oapi/v1/ 路径限制和组织白名单，JSON 与 multipart 请求体，以及遇到 401 时先续订一次 OAuth 令牌再重试
src/infra/yunxiao-oauth.ts	云效 OAuth 2.0 登录：授权服务器元数据发现、动态客户端注册、PKCE S256、127.0.0.1 回环回调、令牌交换与刷新，以及插件其余部分已经读取的"仅所有者"令牌文件（其旁附有刷新材料）
src/sources/kind.ts	SourceKind 协议
src/sources/instance.ts	实例 id 规范化：instanceId、kindOf 和 storesKind
src/sources/registry.ts	SourceRegistry
src/sources/host.ts	SourceHost、SourceContext、Health、health，以及固定的健康文本

路径	职责
src/sources/poll.ts	GitHub 和飞书轮询的共用部分：POLL_HORIZON_MS、polled、cursorAt，以及 pollThreads——它把失败作为 pollError 保留在其线程上
src/sources/github.ts	GitHub 类型和插件入口，导出为 ./github-watch：一轮通知轮询、处于打开状态的工单的拉取请求及其 CI，以及已订阅的线程
src/sources/github-trigger.ts	将 GitHub 评论、评审、Issue 正文和 CI 变更作为触发，以及 GitHub 地址
src/sources/github-pull.ts	拉取请求规则：工单跟随哪些拉取请求（claimPull、observePull）、它向引擎报告的关闭，以及 pullEndNotices——工单以合并或关闭结束时发出的聊天通知
src/sources/github-channel.ts	GitHub Channel：评论和评审线程回复，sent 或 gone
src/sources/github-read.ts	CI 快照、仓库读取、github/ 资源，以及 githubThreadContext：按页读取 Issue 或拉取请求的对话
src/sources/feishu.ts	飞书类型和插件入口，导出为 ./feishu-watch
src/sources/feishu-trigger.ts	将飞书消息作为触发，对实时事件、历史页和话题页使用同一规则
src/sources/feishu-loops.ts	飞书循环：事件消费者、历史恢复、聊天名称，以及已订阅话题的轮询
src/sources/feishu-channel.ts	飞书 Channel：话题回复，sent 或 gone
src/sources/feishu-read.ts	对触发消息及其根、父级和话题的 feishu/ 读取，以及 feishuThreadContext——它将它们连同其图片一并提供给评估
src/sources/admission-context.ts	评估起始所依据的有界窗口：触发所在线程的前几页（通过其类型读取），以及触发周围的消息
src/sources/feishu-image.ts	将飞书消息的图片作为附件读取，供执行评估的会话使用
src/client/index.tsx	客户端插件入口：侧边栏入口、主面板，以及 DeepSeek Harness 插件管理器中的页面
src/client/app.tsx	面板：头部、搜索、词表、标签页，以及当前路由的页面
src/client/provider.tsx	每个页面从其宿主读取的内容：文案、调用、导航、通知、状态轮询，以及 restart——在 dsh-superagent 重启期间快速读取状态
src/client/api.ts	管理 API 的调用和轮询（由 src/app/api.ts 定型），以及 Eval 状态的决策过滤器
src/client/route.ts	标签页、设置分组、parseRoute 所读取的带类型路由，以及记录最后打开的设置分组的路由记忆
src/client/overview.tsx	概览
src/client/source.tsx, trigger.tsx, eval.tsx, execution.tsx	每个阶段一个文件：其状态，以及它若有的设置
src/client/thread.tsx, ticket.tsx	记录页面：带时间线的线程，以及带授权、输入和派工的工单
src/client/settings.tsx	设置标签页：分组导航和屏幕上的分组

路径	职责
src/client/settings-kit.tsx	设置块的构建部件：各阶段分组共用的设置、useBlockDraft、字段（其中包括来源类型的字段），以及保存历史
src/client/prompts.tsx	提示词设置：页面持有其读取的视图，其编辑器持有草稿，编辑、完整提示词和历史标签页据此渲染；内置部分的编辑器展示已更改的默认值，并保留或放弃覆盖
src/client/prompt-config.ts	以纯函数形式给出的提示词配置：按部分的配置、团队部分、技能名称、内置部分（part/，带 withOverride）以及提示词视图给出的部分标题，还有完整提示词的行级差异
src/client/deployment.tsx	部署设置，也显示在 DeepSeek Harness 插件管理器中
src/client/draft.tsx	设置草稿：draftReducer、其状态及其与更新保存修订的合并、saveBarText（保存栏的那行文字），以及保存栏
src/client/controls.tsx	DeepSeek Harness 控件的本地副本，遵循其标记、键盘行为和设计令牌，因为该 bundle 并不导入 @deepseek-ai/dsh-client-ui-primitives；它们不读取任何上下文
src/client/widgets.tsx	页面共用的小组件，读取面板的上下文：区块、页面标题、加载状态、时间、链接、已确认操作、设置检查以及 Markdown
src/client/rows.tsx	页面共用的行：工单、线程和决策，线程的 chip，以及自动化消息连续段落的折叠
src/client/markdown.ts	报告和问题的 Markdown 子集，以及消息中的裸地址，以块和行内片段形式呈现
src/client/text.ts	按键查阅的文案，以及领域值、原因、发送者和时间的措辞
src/client/marks.ts	视图所分类状态的标记：工单或线程的阶段、来源的状态和决策的结果，各自以一个圆点、图标、色调和标签表示
src/client/locales.ts	两种语言的全部文案，以及 001 Domain model 的词表
src/client/client.css, css.d.ts	面板的样式表，以文本形式加载
src/sources/chat/port.ts	聊天端口：ChatEvent、InboundTransport 和 ChatAdapter，即聊天后端所实现的接口
src/sources/chat/dedupe.ts	DeliveredRing，传输层的有界已投递消息 id 环形缓冲，用于丢弃重复投递
src/sources/chat/kind.ts	chatKind，将 ChatAdapter 转换为聊天 SourceKind 的工厂
src/sources/chat/weixin/backend.ts	BridgeEndpoints 以及 WxRoom/WxPerson 类型：桥接客户端的常量和类型
src/sources/chat/weixin/bridge.ts	createBridgeClient，sidecar 桥接的 HTTP 客户端：健康、状态、房间、成员、消息和发送，及其令牌文件、超时和可重试的失败
src/sources/chat/weixin/kind.ts	weixinKind / createWeixinKind，微信聊天的 SourceKind：其 schema 和字段、结构化 @ 范围规则、由桥接支持的 Channel 及其环境
src/sources/chat/weixin/media.ts	readWeixinImage 和 sniffImageMediaType：从 sidecar 下载消息图片，仅信任其魔数，并将其作为附件提交

路径	职责
src/sources/chat/weixin/thread.ts	weixinThreadContext，微信的 ThreadContext：会话页、被回复的消息及其携带的图片，通过桥接客户端读取；root 返回 no_root，因为微信没有话题根
src/sources/chat/weixin/read.ts	面向工作者的微信读取：weixinRef/weixinReferences/weixinResource 以及针对 weixin//message、/thread 和 /around 的 readWeixin，形态与飞书资源语法相同
src/sources/hosting/port.ts	代码托管端口：HostAdapter、ChangeRequest 和 ChangeNote
src/sources/hosting/kind.ts	hostingKind，将 HostAdapter 转换为托管 SourceKind 的工厂
src/sources/hosting/gitlab/adapter.ts	GitLab 的 HostAdapter：createGitlabAdapter、合并请求、note 和流水线的映射，以及其 CI 快照
src/sources/hosting/gitlab/kind.ts	gitlabKind，基于 GitLab 适配器的 GitLab 托管 SourceKind，带有 organizationId、hostUrl 及其环境
src/sources/work/port.ts	工作项端口：WorkAdapter 和 WorkItem
src/sources/work/images.ts	工作项图片：解析行内 ` 引用，通过 files API 下载，在 /projex-images` 下按 sha256 去重，并将其中一个作为附件回送
src/sources/work/kind.ts	workKind，将 WorkAdapter 转换为工作 SourceKind 的工厂
src/sources/work/context.ts	workThreadContext：将工作项作为一个对话读取——描述作为根消息，评论作为带分页的线程，代理自身的评论标记为 bot，已归档的图片作为附件回送
src/sources/work/environment.ts	workEnvironment：按工单命名的变量，指明工作者所处的工作项（_PROJECT、_WORKITEM、_SERIAL、_CATEGORY、_STATUS、_URL），外加宿主添加的任何内容，例如 Projex 组织
src/sources/work/feedback.ts	reflectWorkTickets：工作项在工单处理期间及工单结束后所显示的状态，通过 KV 台账按阶段写入一次，使看板跟随工单而非保持不动
src/sources/work/projex/adapter.ts	Projex 的 WorkAdapter：createProjexAdapter、按类别的工作项和 note 及其 nextToken 分页，以及其标记幂等的回复
src/sources/work/projex/kind.ts	projexKind，基于 Projex 适配器的云效 Projex 工作 SourceKind，带有 organizationId、其环境，以及填充状态下拉框的 refreshLabels
src/sources/work/projex/read.ts	面向工作者的 Projex 读取：projexRef/projexReferences/projexResource 以及针对 projex//item 和 /comments 的 readProjex
src/sources/hosting/stream.ts	pollHosting，托管轮询的一轮：游标、发生变化的变更请求、它们的 note 和 CI，以及溢出计数
src/sources/work/self-writes.ts	自写台账：本进程所写入的工作项修订，轮询不得将其读回为新请求，因为一次评论或状态写入会推高用于派生触发 ID 的修改时间

路径	职责
src/sources/work/stream.ts	pollWorkItems、workTrigger、noteTrigger 和 observeNotes：工作项轮询的一轮，外加按所跟工作项的评论轮询，带有按项的水位线以及有界的 backfillComments 首次见到时的回填
src/sources/chat/dingtalk/adapter.ts	钉钉适配器：parseStreamEvent 和 sendMessage，经由 sessionWebhook 和 oapi 路径，带有其幂等标记
src/sources/chat/dingtalk/stream.ts	startStream 和 createLongPollingStream：消费注入的事件流，报告其健康状态并重连直至中止
src/sources/chat/dingtalk/kind.ts	dingtalkKind / createDingTalkKind，钉钉聊天的 SourceKind：其 schema 和字段、基于 @ 的范围、由流支持的 Channel，以及仅读取回调消息的 ThreadContext
src/sources/chat/dingtalk/read.ts	面向工作者的钉钉读取：dingtalkRef/ dingtalkReferences/dingtalkResource 以及针对 dingtalk//message 的 readDingtalk
src/sources/chat/wecom/crypto.ts	企业微信回调加解密：signatureOf/verifySignature，以及对 AES-256-CBC 载荷的 encrypt/decrypt
src/sources/chat/wecom/adapter.ts	企业微信回调适配器：parseXml，以及 createWecomSender（带有其缓存的 access_token）和消息发送
src/sources/chat/wecom/webhook.ts	handleWebhook，纯回调处理器：校验、解密、解析、去重，以及 URL 校验回显
src/sources/chat/wecom/kind.ts	wecomKind / createWecomKind，企业微信聊天的 SourceKind：其 schema 和字段、基于 @ 的范围、回调 Channel 及其事实、wecomTriggerOf，以及仅读取回调自身消息的 ThreadContext
src/sources/chat/wecom/read.ts	面向工作者的企业微信读取：wecomRef/wecomReferences/ wecomResource 以及针对 wecom//message 的 readWecom，即回调所携带的那一条消息
src/sources/dingtalk-watch.ts	钉钉插件入口，导出为 ./dingtalk-watch：注册钉钉类型并运行其注入的流
src/sources/wecom-watch.ts	企业微信插件入口，导出为 ./wecom-watch：注册企业微信回调类型，并在 /wecom/ 挂载其公开的签名回调路由，将每个回调转换为触发
src/sources/weixin-watch.ts	微信插件入口，导出为 ./weixin-watch：注册微信类型并轮询其 sidecar 桥接，以获取已配置聊天中的新消息
src/sources/gitlab-watch.ts	GitLab 插件入口，导出为 ./gitlab-watch：注册 GitLab 类型，其 note 通道通过该实例所绑定的适配器发送
src/sources/projex-watch.ts	Projex 插件入口，导出为 ./projex-watch：注册 Projex 类型，其评论通道通过该实例所绑定的适配器发送

src/sources/github.ts 和 src/sources/feishu.ts 也是两个来源插件的入口，由 tsdown.config.ts 构建，并在 package.json 中导出为 ./github-watch 和 ./feishu-watch。bundle 的 cordis.patch.yml 插入这三个插件，其中飞书为禁用状态。

测试

测试	覆盖
tests/core-engine.spec.ts	内存仓库上的领域规则：接收、Eval、扇出、恢复与停止、被中断的派工、暂停、策略变更、报告与结果、附加、回复、失败的投递以及被忽略的问题
tests/stage.spec.ts	工单阶段，逐行
tests/policy.spec.ts	来源范围、门禁、准入表、Mandate 复核、策略版本、设置迁移和评估结果
tests/registry.spec.ts	SourceRegistry：实例、校验、策略部分、Channel 和线程上下文
tests/instance.spec.ts	实例 id：instanceId、kindOf 和 storesKind，以及带与不带 kind 的来源实例设置
tests/sources.spec.ts	SourceContext 上的飞书和 GitHub 来源：转换、轮询和读取位置、拉取请求规则、健康
tests/commands.spec.ts	以部署账户身份进行的 CLI 调用、各 Channel 的回答，以及仓库范围内的 GitHub 调用
tests/feishu-read.spec.ts, tests/github-read.spec.ts	飞书和 GitHub 读取
tests/admission-context.spec.ts, tests/feishu-image.spec.ts	评估的线程窗口，以及飞书图片读取
tests/assessment.spec.ts	MasterAssessor 每次 Eval 保留什么，以及何时放手
tests/background.spec.ts	等待工作者的 shell 作业和子代理：将其唤醒的结束，以及停止的宿主
tests/bundle-patch.spec.ts	bundle 的补丁（按 DeepSeek Harness 读取它的方式），以及它所定义执行器会话的代理预设
tests/effects.spec.ts	效果队列：合并、限制、带退避的重试、健康与关闭
tests/passes.spec.ts	Passes：一次一个，在某个 pass 运行期间请求的那些在其后作为一个运行，以及失败的 pass 所报告的运行时健康
tests/outbox.spec.ts	运行中 Runtime 上的 Outbox：在暂停处理或其来源未安装期间，消息会等待
tests/cache.spec.ts	TextCache
tests/queue.spec.ts	Runtime 通过效果队列进行的作业，以及早期版本所存储的内容
tests/followups.spec.ts	某人在工单线程上的后续跟进：其发往主会话的通知、对其采取行动的工具、突发的代价；工作者的交还和上下文跟进，以及主会话的记录
tests/health.spec.ts	每个部分一个健康条目，各由其自身的下一次成功清除
tests/delivery.spec.ts	AgentDelivery
tests/retry-health.spec.ts, tests/role-models.spec.ts	模型预算重试，以及两种角色的模型
tests/runtime.spec.ts	在模拟的 DeepSeek Harness 宿主上，通过工具和 HTTP 运行的整个应用
tests/tools.spec.ts	会话的工具：orchestrator_read 为各角色读取的内容，以及其他主会话工具的结构和检查
tests/runtime-concurrency.spec.ts	相互重叠的作业：pass 期间的触发、在投递和启动期间发生变化的工单

测试	覆盖
tests/http.spec.ts	同源和回环检查、每种失败的状态、请求体限制、路径参数和查询
tests/settings.spec.ts	与模型或账户检查竞争的设置保存、被某次保存赶超的策略同步，以及暂停、同步和某个块的拒绝
tests/views.spec.ts	管理视图从存储中派生出什么，例如被丢弃消息所属的线程
tests/timeline.spec.ts	线程的各页，以及触发按时间再按 id 的相邻项，每个接收到的触发的时间采用统一形式
tests/migration.spec.ts	schema 11 迁移：每条存储记录为其当前形态，以及对旧 schema 的拒绝
tests/store.spec.ts	仓库及其工作集（在 SQLite 中与在内存中一致）、kv 区域，其中 KV 的每个键仅在 src/kv.ts 和迁移中拼写，还有设置历史与查询
tests/secret-file.spec.ts	两个平台上的机密文件规则：POSIX 文件被拒时带上它实际拥有的模式，缺失或空的文件始终被拒，而 Windows 接受它无法表达的情况
tests/prompts.spec.ts	提示词校验、组装和版本
tests/cards.spec.ts	内置文本必须陈述的规则，逐文件列出
tests/model-surface.spec.ts	模型所读取的一切：工具、提示词、技能和输入，对照工具 schema、执行环境和词表进行检查，并在每个角色的大小预算之内
tests/deployment.spec.ts, tests/environment.spec.ts, tests/index.spec.ts	插件配置和设置、代理的进程环境、插件入口
tests/service-script.spec.ts	scripts/dsh-superagent 针对假的 ps、tmux 和 dsh
tests/client.spec.tsx, tests/navigation.spec.ts, tests/controls.spec.tsx	客户端插件的注册和轮询、会话导航和路由记忆、本地控件，以及面板的导入（不存在循环）
tests/panel-overview.spec.tsx, panel-source.spec.tsx, panel-trigger.spec.tsx, panel-eval.spec.tsx, panel-execution.spec.tsx	面板的每个页面，对照按路由应答的假管理 API
tests/panel-settings.spec.tsx, panel-prompts.spec.tsx, panel-deployment.spec.tsx, panel-panel.spec.tsx	设置分组、草稿和历史、提示词、部署页面，以及面板的搜索、词表和路由记忆
tests/draft.spec.ts, tests/prompt-config.spec.ts, tests/markdown.spec.ts, tests/marks.spec.ts	草稿状态、合并和保存栏的那行文字；按部分的提示词配置和行级差异；Markdown 子集及其链接；视图所分类的每个状态的标记
tests/locales.spec.ts, tests/styles.spec.ts	两种语言的文案、样式表
tests/class-docs.spec.ts	src 中的每个类在其前面都有一个 JSDoc 块
tests/docs.spec.ts	编号文档仅链接到更早的文档、每个本地链接都能解析、任何文档都没有合并冲突标记，且本文档的表格列出每个模块和每个测试
tests/kv-instances.spec.ts	KV 各系列和健康键的实例维度：每个实例一个逻辑键，位于其自身的存储键中，以及 healthEntry 的 实例:条目 形式
tests/registry-instances.spec.ts	从同一设置映射解析出的同一类型的多个实例，各自拥有自己的健康条目和各自的出站 ChannelHost

测试	覆盖
tests/hosting-port.spec.ts	hostingKind 的 schema、scope 和通道能力，基于 stub HostAdapter
tests/chat-dedupe.spec.ts	DeliveredRing 的有界去重，以及基于 stub InboundTransport 构建的 chatKind
tests/work-port.spec.ts	workKind 的类别到行映射及其基于 stub WorkAdapter 的出站回复
tests/work-context.spec.ts	工作线程上下文：描述作为根、评论作为跨页的线程、工作项每次对话只读取一次、失败的读取会重试、代理自身的评论标记为 bot，以及已归档图片的回送
tests/work-environment.spec.ts	共用的工作环境：按宿主的变量名、来自触发通道地址或其 meta 的工作项值、对在别处开始的工单的空值，以及宿主自身的添加
tests/work-feedback.spec.ts	reflectWorkTickets：起始状态一次、结束后的完成状态、已是最新的状态被跳过但记录、失败的写入会重试、被遗忘的工单被丢弃，以及只读宿主保持不动
tests/weixin-bridge.spec.ts	基于 stub fetch 的微信桥接客户端：其六个端点、bearer 令牌文件的模式、事件映射、幂等键和失败码
tests/weixin-watch.spec.ts	微信轮询循环的监听集合：它每轮推送的已配置 id、后端无法解析的聊天，以及它必须不发送任何内容的两种情况
tests/execution-directory.spec.ts	工单目录根：两个平台上的绝对路径（/x、C:\x、C:/x、\\server\share），相对路径则被拒，在 schema 和输入自身的模式中都如此
tests/state-lock.spec.ts	acquireStateLock：获取空闲状态目录、接管持有者已消失的锁、等待被短暂持有的锁，并在确实被持有时指出补救办法
tests/state-lock.spec.ts	acquireStateLock：获取空闲状态目录、接管持有者已消失的锁、等待被短暂持有的锁，并在确实被持有时指出补救办法
tests/person-notice.spec.ts	personNotice：当某人的消息被拒时其收到的那一行，按原因码及回退给出
tests/outbound-file.spec.ts	resolveTicketFile，工单模型与机器文件之间的门禁：越界爬升的路径、离开目录的符号链接、可执行文件、空文件、超大文件，以及没有目录的工单
tests/weixin-kind.spec.ts	微信的 SourceKind：其 schema 和字段、结构化 @ 范围规则、Channel 和策略、环境及其事实、其搜索词和回复映射
tests/weixin-read.spec.ts	微信工作者读取：引用和资源语法、message/thread/around 形态及其 next_resource、no_room/not_found 回答，以及 root/parent/thread 的 ThreadContext 部分
tests/hosting-gitlab.spec.ts	GitLab 适配器和托管 HTTP 客户端：合并请求、note 和流水线映射、分页、幂等回复，以及令牌和路径检查
tests/hosting-gitlab-kind.spec.ts	GitLab 托管的 SourceKind：其标识、schema 和字段、其环境及其范围行

测试	覆盖
`tests/source-actions.spec.ts`	来源卡片操作：微信类型提供 start/stop/restart/check，各自以固定的 argv 对已配置任务调用 `schtasks`，就绪状态跟随任务名，且非任务名的名称会被 schema 和运行器双双拒绝
`tests/client-bundle-id.spec.ts`	构建出的客户端 bundle 以包名注册，构建横幅从 package.json 派生该名称，而非重复一个字面量
`tests/panel-copy-and-order.spec.ts`	两条页面规则：任何来源的文案或任一 UI 字典中都没有 Markdown 标记；来源卡片先按启用在前、再按英文名排序，且不改动输入
`tests/admission-tables.spec.ts`	每个来源的准入表读法一致：聊天按每个会话一行，外加其他聊天和私信（其范围报告这些 id），微信对其群组做同样的事，GitLab 声明一个包含四个接收区间的策略
`tests/work-admission-rows.spec.ts`	Projex 行由一个同时持有宿主键的实例配置构建：完整配置能构建它们，不在列表中的类别被标记为未接收且没有单元格，面板发送的策略块和来源块通过校验
`tests/work-policy.spec.ts`	Projex 准入：派工即直接请求、未指派的项保持环境性、所列成员的已指派项为显式、未列出的项被拒，并提供三个类别行
`tests/work-projex.spec.ts`	Projex 适配器：按类别的工作项和 note、`workitems:search` 分页、幂等回复、项目名册、附件上传及其 `embedHtml`/链接回退、RICHTEXT 评论体、工作流状态列表，以及云效客户端的检查（含 JSON 和 multipart 请求体）
`tests/work-projex-kind.spec.ts`	Projex 工作的 `SourceKind`：其标识和字段、其 schema、其范围行、其环境和 Channel 能力、其搜索解析、其成员同步，以及其状态列表操作
`tests/yunxiao-oauth.spec.ts`	云效 OAuth：PKCE、元数据发现、对照注入的 fetch 和真实回环回调的完整浏览器登录、伪造 state 的拒绝、令牌文件，以及续订
`tests/dingtalk-adapter.spec.ts`	基于注入的 `fetch`/`connect` 的钉钉适配器：流事件解析、sessionWebhook 和 oapi 发送、幂等标记、失败码以及流的中止
`tests/dingtalk-kind.spec.ts`	钉钉的 `SourceKind`：其标识和 schema、其字段、其策略、其能力、其基于 `@` 的范围以及 ThreadContext 部分
`tests/dingtalk-read.spec.ts`	钉钉工作者读取：引用和资源语法、那一条消息，以及 `no_root`/`no_parent` 部分
`tests/projex-read.spec.ts`	Projex 工作者读取：带状态和图片的工作项、带标记代理自身评论的分页评论线程，以及引用语法
`tests/wecom-crypto.spec.ts`	企业微信回调加解密：排序后的签名、在不指名密钥的情况下校验密钥，以及加密/解密的往返
`tests/wecom-webhook.spec.ts`	企业微信回调处理器和发送器：XML 解析、URL 校验、签名事件、去重以及缓存的 access token
`tests/wecom-kind.spec.ts`	企业微信的 `SourceKind`：其标识和 schema、其字段、其两分支范围、其至少一次投递的能力和事实、回调消息以及 ThreadContext 部分

测试	覆盖
tests/wecom-read.spec.ts	企业微信工作者读取和触发映射：引用和资源语法、带图片标记的消息、通过 chatid 的群回复和通过 touser 的私信回复，以及 no_root/no_parent 的 ThreadContext 部分
tests/hosting-stream.spec.ts	真实存储和 stub 适配器上的 pollHosting：游标、发生变化的变更请求和 CI、溢出以及一次失败的变更
tests/notice-channel.spec.ts	拒绝通知只到达对话、别无他处：工作项的通道在注册处拒绝它、聊天的通道接受它、从未选择加入的通道拒绝它，且 acceptsNotice 在行入队之前就作出回答
tests/work-self-write.spec.ts	台账及其外围的适配器：每个键一个修订、有界，以及一个丢弃自身写入但保留某人之后编辑的轮询
tests/work-stream.spec.ts	真实存储和 stub 适配器上的 pollWorkItems：初始化后的游标、三个类别、幂等、中止和一次失败的类别，以及评论轮次——某人的评论在其工作项的准入行中移交、代理的和机器人的被跳过，以及工单未打开的线程完全不读取
tests/work-images.spec.ts	工作项图片：`` 解析（HTML 和 jsonML）、sha256 去重、类型/大小限制、失败降级，以及将已归档图片读回为附件
tests/panel-instances.spec.ts	面板针对同一类型多个实例的数据层：两个聊天实例的行和健康、删除其中一个，以及一个不带 kind 的旧版来源
tests/live/weixin-bridge.live.spec.ts	对照真实本地 sidecar 的微信桥接（仅在 LIVE=1 时）：健康、状态、房间、成员、消息、幂等发送，以及令牌和权限失败
tests/live/gitlab-contract.live.spec.ts	对 GitLab.com 契约的只读验证（仅在 LIVE=1 时）：合并请求和 note 映射，以及未授权令牌的错误（从不指名该令牌）
tests/live/projex-contract.live.spec.ts	对一个真实组织只读验证 Projex 契约（仅在 LIVE=1 时）：每个被轮询的类别都通过 workitems:search 列出、工作项可同时按其 id 和序列号读取，且评论以文本返回而非 API 存储的 JSON 封装
tests/live/thirdparty-errors.live.spec.ts	企业微信、钉钉和云效的认证失败（仅在 LIVE=1 时）：各提供方的错误码，以及消息中不出现任何机密
tests/weixin-media.spec.ts	基于注入的 fetch 和假附件的微信媒体路径：魔数嗅探、大小和部署的拒绝、附件提交，以及 ThreadContext 的消息和图片读取

tests/fixtures/ 存放若干测试共用的内容：host.ts，一个模拟的 DeepSeek Harness 宿主；panel-server.ts 和 panel.tsx，假管理 API 及包住某个页面的面板；deployment.ts，一个已解析的部署；git.ts，测试所启动的每个 git 的环境（不指名任何仓库）；以及 async.ts。

持久化

一个 SQLite 文件，/orchestrator.sqlite，仅由存储打开。

表	存放	写入者	
trigger	每个触发，其时间 at 采用 Date#toISOString 给出的统一形式，在引擎接收它时设置，因此时间按文本比较	引擎	
disposition	每个触发的去向：带来源原因码的 filtered、其 Eval 和决策、它到达的工单（通过扇出或附加）、它是否被重新入队、将它交还的工单及其原因，以及其执行器确认或回答了它的那些工单	引擎	
trigger_thread	每个触发位于哪些线程上，连同其时间以及是否由某人书写，使线程的触发无需扫描每个触发即可读取，按时间再按 id 排序：线程时间线的某一页从 `	` 游标继续，共享同一时间的触发不会被跳过	存储，随每个触发
thread	线程：订阅、为上下文而跟随它的工单、已排队的触发、进行中的 Eval、其过期计数以及 meta；生成列 active（在有触发排队或 Eval 运行时置位）和 subscribed（当某工单订阅它时为其来源）	引擎；来源写 meta	
ticket	工单及其 Mandate、来源、执行、派工、Channel、输入以及它们所跟随的线程；生成列 open（在其尚未结束时置位）和 active（在输入未投递或最新派工处于待处理或运行中时置位）	引擎	
decision	决策，按线程	引擎	
effect	效果队列：按类型和键、载荷、其顺序、带最后错误的失败尝试，以及其到期时间	效果；schema 11 迁移将早期版本为主会话保留的内容入队	

表	存放	写入者	
outbox	每条发往某个 Channel 的消息的回执，按 id，连同它所针对的工单及其 status（pending、sent 或 dropped），保留使某个 id 绝不被入队两次；对于被丢弃的消息，何时与为何，体现为生成列 dropped、其时间和一个原因码。等待中的消息有一个 reply 效果，持有其尝试次数	Outbox	
kv	所有其他记录，按键	命名它的部分	

生成列由其行的 JSON 计算得出，在其集合之外为空，每个生成列在其集合上都有一个索引，因此一次 pass 只读取活跃的作业。早于某列的数据库会在存储打开它时获得该列。

KV（src/kv.ts）定义每条键值记录的所有者、键和类型。所有读取者和写入者都使用这些条目；只有迁移代码会指名更旧的键。

记录	键或内容
schema	当前 schema 版本
设置版本	settings-revision、settings-version:、settings-block-revision:；快照包含来源、策略版本和受影响的工单
提示词版本	prompt-revision、prompt-version:
主会话	master-id
会话提示词历史	session-prompts:
来源状态	读取位置，如 github-cursor:、PR CI 指纹和已见的飞书话题消息；键以类型或 thread- 加上一个线程键开头
显示名称	source-labels:

忽略操作存储在其决策上。输入文本在读取时从工单渲染，因此两者都不需要单独的键值记录。

读取缓存使用有界的内存 TextCache 实例，重启后丢失。会话在翻页浏览某个原始来源记录或提示词预览时，重启后必须从偏移 0 重新读取。在准入之后解析的飞书链接以同样方式缓存；基于链接的搜索只有在当前进程生命周期内该链接被读取过之后才能找到它们。

目前没有任何东西会删除旧记录，因此以下内容会无界增长：trigger、disposition 和 decision 行；已结束的工单及其线程；outbox 的回执；以及 kv 中每次保存的设置修订和提示词版本、每个会话所依据的提示词修订，以及来源按线程和按 Issue 或拉取请求的记录。

SCHEMA_VERSION 只增不减。较新版本的数据库拒绝启动；src/app/migration.ts 在启动时一次性升级预发布 schema 10 的数据库，并拒绝更旧的数据库。

面向模型的接口

主会话工具

主会话有 14 个应用工具：

用途	工具
读取与路由	orchestrator_read, orchestrator_route
管理工单	orchestrator_ticket_retry, orchestrator_ticket_request_status, orchestrator_ticket_note
重新评估	orchestrator_trigger_requeue
评估主动作业	orchestrator_eval, orchestrator_eval_read, orchestrator_eval_decide
通信与退订	orchestrator_reply, orchestrator_thread_unsubscribe
配置	orchestrator_settings_save, orchestrator_policy_sync, orchestrator_prompts_save

orchestrator_route 按顺序接受 1–20 个决策。每个接受、附加、停止或忽略都作为一条单独的引擎命令运行。它还允许直接从推断出的候选卡片上忽略该候选。orchestrator_ticket_note 将事实作为上下文而非权限发送。

设置保存选择 policy、eval、executor、pause 或 source；最后一个还会指名其来源 id。带 preview: true 的提示词保存会组合文本而不保存。

主会话还拥有 DeepSeek Harness 的 skill、bash、read、read_image、job_output、job_list 和 job_kill。避免代码读取是一条提示词规则，而非工具限制。

预设与等待限制

新的主会话使用 dsh-superagent-router（ROUTER_PRESET）。shell 命令和作业等待在 5 秒后返回。如果 Web UI 在首轮之前选择了另一个预设，Sessions 会恢复此预设，或对工作者恢复 WORKER_PRESET。

src/app/sessions.ts 中的 noWait 守卫拒绝在命令位置调用 sleep 的主会话 shell 命令、超过 30 秒的 shell timeoutMs，以及超过 5 秒的 job_output 等待。报告和作业结果会自动到达，因此等待只会延迟到达的候选。

处于 standard 的现有主会话同样需要为前台命令和作业等待设置显式超时；否则该预设的默认值会分别将步骤挂起 60 秒和 30 秒。宿主通过 presetForSession 确定当前预设，回退到会话投影或头部，若无法读取该值则假定为 standard。

执行器工具

每个执行器有六个应用工具：

工具	用途
orchestrator_read	读取其自身工单可用的资源

工具	用途
orchestrator_ack	确认一个请求
orchestrator_report	报告进度或结果，按拉取请求规则绑定拉取请求
orchestrator_thread_unrelated	丢弃一个无关的已附加线程，交还其拥有的后续跟进，并告知主会话原因
orchestrator_thread_follow	为上下文订阅一个相关线程，并告知主会话
orchestrator_trigger_handback	将一条消息交还一次，并附带其候选所携带的原因

如果另一张工单仍拥有该消息，或本工单从未拥有它，交还会返回 queued: false。例如，一条机器人消息独立地进入了 Eval。

工作者预设在 cordis.patch.yml 中定义，并通过 WORKER_PRESET 选用。src/app/sessions.ts 中的 WORK_TOOLS 保留可用的文件、搜索、shell、作业、web、技能、todo 和子代理工具。工作者不会获得向人提问或呈现、更改 profile、在工作者槽位之外调度作业、启动或向其他会话发消息、或编辑外部文档的工具。工单结束后工具调用会被拒绝。

结果形态与评估上下文

路由结果使用 {decision?, reason, ticket?}。orchestrator_route 返回 {results}，保持条目顺序，并为每个条目添加 trigger_id 和 kind。无效或被拒的条目返回 refused 和一个原因，而不会中断整批。

设置和提示词保存返回已保存的配置。策略同步返回来源、允许列表大小，以及 added 和 removed 名称。

orchestrator_eval 返回上下文而非指令。评估指令位于 runtime/admission（prompts/admission.md），规则位于 runtime/policy。其上下文包括：

- 触发文本，最多 4,000 个字符，连同图片 id 和该批中的其他触发。
- 最多八个 related_tickets（打开的在先），以及 tickets_total。
- 该 Eval 的 policy_version。
- 一个线程窗口：前五条消息，以及触发两侧各五条。消息使用 {id, from, at, text}，通常限制为 400 个字符，而两侧各两条时为 1,200。被截断的消息包含 chars；图片引用使用 images。
- 其他已加载的消息以 50 字符的 {id, from, preview} 桩形式给出。orchestrator_eval_read 按 id 展开它们。

工单卡片使用来自 src/app/cards.ts 的封闭集合 TICKET_STATES：一个打开的阶段，如 person/review 或 working/queued，或一个已结束的结果。无论通过接受、评估还是 orchestrator_ack 提交，确认都限制在 100 词以内。

读取资源

src/app/resources.ts 中的 RESOURCES 定义资源并生成工具的 resource 描述。

资源	内容
tickets, triggers, threads, decisions	分页列表，形式为 {items, next_offset}；活动列表最新的在前
decision/	一个决策
thread/	标签、工单、最近的决策以及一页触发
trigger/	消息文本、被回复的触发和相邻消息，不含 CI 运行
ticket/	相关 id、来源批次的触发、线程，以及派工和输入的前几页
工单子资源	/assignments, /inputs, /assignment, /result, /report/, /transcript, /message/, /input/
settings, settings/	修订和摘要，或一个可保存的完整块
prompts, prompts/, prompts/current, prompts/defaults, prompt-preview/	提示词配置和组合文本
models, models/	可用模型
feishu//…, github/…	原始来源记录，在各来源技能中描述

工单线程对仅上下文跟进标记为 context，对来源绑定标记为 bound。派工页面最新的在前，并包含报告预览。用 next_offset 继续列表。/input/ 接受输入的名称或 id。settings 完整包含小块，并摘要策略块和来源块。

来源资源包含消息邻域、github/pulls//thread/ 和 github/commits//checks。

原始记录的消息以一种紧凑形态读取，{id, from, at, text, reply_to?, path?, line?, url?}，而非 GitHub 或飞书记录它们的方式。

执行器会话只读取其自身工单、其线程、到达它的或由其请求所持有的触发、它所依据的提示词版本以及原始记录，其页面略去它无法读取之物的 id、决策和其他工单。

页面所显示的每个 id 都通过这些之一读取，因此会话可从一对象走到下一个；不用于读取的哈希，例如 Eval 的 id 或策略版本，会从页面上略去。

文本页默认 4,000 个 UTF-16 代码单元，最多 8,000；列表默认 20 项，最多 50 项。

早期版本的名称 event/、task/ 和 observation/ 仍然可以解析。

投递给会话的消息

这些消息被引导进主会话的轮次：

消息	内容
Candidate	批量触发卡片、线程、related_tickets、tickets_total、delivered_to、来自 Issue/PR 链接的引用工单、handed_back 原因以及下一步操作
follow_up	一条消息在其执行器眼中的样子、replies_to、接收工单，以及提醒：由执行器决定它，而主会话只能停止作业
worker_report	工单卡片、原始触发、状态和最多 600 字符的摘要；被截断时有 chars 和一个读取指针
worker_attention	携带该问题的工单卡片

其他通知在不唤醒主会话的情况下注入：ticket_ended、settings_saved、提示词保存、触发重新入队、被丢弃的订阅和上下文跟进。设置通知会指名块、变更的路径和 read: settings/。对于主会话自身执行的保存、重新入队、退订或附加，主会话不会收到多余的通知。

工作者以内联方式接收派工和输入。第一条消息包含标题、最多 1,500 字符的派工文本和来源触发卡片。输入携带一张最多 600 字符的触发卡片和 replies_to，或纯输入文本。读取指针仅对截断的内容出现。

仅面板操作

没有用于忽略不可用的决策、重新运行设置、刷新来源显示名称，或将执行器拥有的个人后续跟进发送到 Eval 的工具。最后一个操作被 orchestrator_trigger_requeue 有意拒绝。其他面板操作都有等效的工具。

大多数内置指令位于 prompts/*.md，每行一句，由 promptFile 连接。来源技能位于 skills/dsh-superagent-*/SKILL.md。工具描述和固定通知（如重启恢复说明）保留在代码中。评估指令仅位于 prompts/admission.md。

tests/model-surface.spec.ts 检查这些接口是否有漂移。004 Prompts 说明了团队可编辑的部分。

配置重载

除 stateDir 外，每个插件配置键都是易变的。面板的部署设置和 DeepSeek Harness 的插件管理器通过 DeepSeek Harness 的配置表单编辑它们；宿主写入 cordis.patch.yml 并就地更新这些值，dsh-superagent 在用到每个限制时读取它，因此变更无需重启即可生效。

一次更改了设置过程所读取的来源字段（例如某账户的 CLI）的设置保存，以及设置等待期间的任何保存，都会重启 dsh-superagent。DeepSeek Harness 的设置服务在该重启期间不提供该条目，因此在自身重启后，dsh-superagent 会请求它再次读取该条目，已打开的表单也重新变为可编辑。

对照真实数据审查变更

将 DSH_HOME 指向一个临时目录，并在那里创建一个新的 profile。从备份复制数据库，禁用每个来源，暂停处理并移除主会话 id。为 GitHub 和飞书 CLI 提供只读的替身。

待解问题

- 信任：哪些触发字段受信任，例如来源的参与者（actor）和线程成员资格，哪些不受信任，例如正文和引用。
- 部署隔离：将代理的 profile 放在其自身固定版本（pinned）的 DeepSeek Harness 安装上，这样在别处升级 DeepSeek Harness 不会破坏它。

常见做法与排障

只让白名单里的人能发起工作

33. 面板 → 评估 → 在各来源上选中来源（钉钉 / 飞书 / GitHub / GitLab / 云效 Projex / 企业微信 / 微信）。
34. 在「能发起工作的用户」里加人。微信/企业微信可以点「立即同步 / 刷新名单」从群成员或通讯录带入。
35. 准入表按行定义：名单内/名单外 × 群里 @/未 @/私聊。名单外的人即使 @ 机器人也不会变成工单。

群里只在被 @ 时响应

- 微信 / 钉钉 / 企业微信：把该群/来源的「接收」设为「仅 @」（准入表的「群里未 @」一行设为「忽略」）。
- 飞书：群的「接收」选「仅 @」；工作项/GitHub 没有这个概念。

关掉看板状态回写

面板 → 来源 → 云效 Projex → 关「回写工作项状态」。默认就是关的：智能体只在评论里交付，不改看板。打开后要选两个状态（下拉里是看板上的名字，实际写的是状态 ID）。

让执行器读到原始上下文

来源	资源
飞书	`feishu//message`、`/root`、`/parent`、`/thread`、`/around`
GitHub	`github//…`
微信	`weixin//message`、`/thread`、`/around`（图片 `[image:]`，可用 `GET /media` 下载）
企业微信	`wecom//message`
钉钉	`dingtalk//message`
云效 Projex	`projex//item`、`/comments`（可翻页）

资源是只读的：读不会创建触发、不会回复、不会改远程状态。

排障

现象	先看什么
面板显示「dsh-superagent 没有响应」	多半是插件没挂载：看 DSH 日志有没有 `dsh-superagent ready`；常见原因是状态目录里旧会话的 preset 名不存在（2.3.0 已修，preset 用共享 id），或 `stateDir` 配错。

现象	先看什么
云效报 操作失败，当前工作项已进回收站	工作项被人删了。插件会把它当「已消失」：线程标 gone，不再读、不再报错；恢复工作项后请删掉旧线程让它重新出现。
云效授权页 `invalid client_id or redirect_uri`	2.4.0 以前是 Windows `cmd start` 把 URL 在 `&` 处截断；已改为 `rundll32 url.dll,FileProtocolHandler`。升级后重启 DSH 再授权。
企业微信收不到消息	应用「接收消息」URL 必须是 `https:///wecom/` 且反向代理到本服务；回调 `Token` 与 `EncodingAESKey` 必须和后台一致，缺 Token 时全部拒绝（403）。
钉钉不监听	本 bundle 不内置 Stream SDK：部署要注入 `connect()`（`dingtalk-stream` 客户端）；`facts`/健康会说明「未注入」。
微信桥掉线	面板「检查」读 `/health`；长期无事件会自愈重挂；换号/重登录后桥会重新 attach，不需要重启插件。
状态名提示「从未见过」	先在来源卡片点「读取云效状态词表」（或字段旁刷新），再把下拉里的名字选中。
升级后两套插件都启用	同一种 Source kind 只能注册一次：先停旧 `dsh-superagent-channels`，再启本插件；`stateDir` 留空即沿用原目录。

自己加一个来源

36. 实现 `ChatAdapter`（轮询/流/回调）或 `WorkAdapter`，定义 `SourceKind`（scope / policy / channel / threadContext / find / refreshLabels）。
37. 在 `cordis.patch.yml` 增加一行 `- id: ... name: dsh-superagent-multi-channel/-watch`。
38. 给 worker 加资源：`src/app/resources.ts` 的 `RESOURCES` 与路由，并在 `skills/dsh-superagent-/SKILL.md` 说明。
39. 加单测：读取、引用语法、上下文三段、排障字段；跑 `pnpm check`。

扩展：多通道层

English · **简体中文**

本仓库是一个基于 DeepSeek Harness 的研究原型的**下游发行版**（见 REPOSITORY.zh-CN.md）。原型监听一个 GitHub 仓库、可选飞书。 本版保留原型全部行为，补上国内团队真正在用的通道，并让同一种通道可以跑多个实例。

上游的编号文档仍然描述 Agent 本身。本文是「加了什么」的地图，也是后来者继续扩展时的参考。

1. 加了什么，放在哪

缺口	本版补上的	位置
只有 GitHub	**GitLab**（自建或 gitlab.com）：合并请求、评论、流水线	`src/sources/hosting/gitlab/*`、`src/infra/hosting-api.ts`
没有工作项	**云效 Projex**：需求 / 缺陷 / 任务、评论、状态	`src/sources/work/projex/*`、`src/infra/yunxiao-api.ts`
只有飞书	**微信个人号**：群 @、消息图片、语音转写	`src/sources/chat/weixin/*` 与 sidecar
—	**钉钉** Stream、**企业微信**加密回调	`src/sources/chat/{dingtalk,wecom}/*`
一种通道一个实例	**同种通道多实例**，配置 / 健康 / 游标按实例隔离	`src/sources/instance.ts`、`src/sources/registry.ts`

每个来源插件都会加载（`weixin`、`gitlab`、`projex`、`dingtalk`、`wecom`、`feishu`、`github`），并各自构建成独立入口（`lib/*-watch.js`），所以面板能把它们全列出来，跑不跑由卡片上的开关决定、不由 profile 决定。 没配置的来源报 `sourceOff` 后空转，因此启用一个通道 = 打开开关 + 填它自己的配置，不用重新打包、也不用改 profile。

2. 唯一不得不改的假设

原型只有一个扩展点 `SourceKind`，但它把**通道**当成了**实例**：`registry.resolve()` 写死 `id: kind.kind`，于是「两个 GitLab 项目」「两个微信账号」不可能。本版把**设置键**（实例 id）与 **行为 id**（通道 id）分开，并保留一条兼容规则：

> *默认实例 id 仍等于通道 id，因此已存的 trigger、thread、mandate 与 policy 无需迁移即可解析。*

其余一切都由三份窄契约推出来，而不是三套一次性集成：

```
src/sources/hosting/   HostAdapter  →  变更请求、评论、CI
src/sources/work/      WorkAdapter  →  工作项、评论、状态
src/sources/chat/      ChatAdapter  →  消息、线程、媒体
                       InboundTransport (poll | stream | webhook) 、DeliveredRing
```

新增一个 provider 就是写一个几百行的适配器。trigger 形状、变更请求生命周期、CI 摘要、评论幂等、 卡片与提示词都原样复用；只有身份与 scope 是 provider 特有的。第 6 节是配方。

3. 逐通道的功能点

3.1 GitLab — `hosting/gitlab`

方面	细节
监听什么	配置项目的合并请求、其评论、以及流水线状态
传输	轮询（`pollSeconds`，默认 30）
身份	`hostUrl` + `project`（group/name）；自建实例是一等公民
凭据	个人访问令牌（`api` 作用域）放在**文件**里，每次调用读取，owner-only（`0600`，Windows 用 ACL）
路径安全	每个请求路径都被限制在配置的项目内；越界的路径在任何请求发出之前就被拒
错误映射	非 JSON 响应变成状态错误，绝不回显令牌；401 映射为 `unauthorized`
出站	评论与评审线程回复，**标记幂等**（回复带标记，重试时已落地的能被识别）
已知缺口	对真实实例的写操作在本机未验证（没有 PAT）

3.2 云效 Projex — `work/projex`

方面	细节
监听什么	配置项目空间里的工作项（按类别）及其评论
类别	默认轮询需求 / 缺陷 / 任务；其它类别直接丢弃，不猜
传输	轮询（`pollSeconds`）
身份	`organizationId` + `project`；`assignedTo` 是阿里云 **accountID**，所以配置里必须存机器人自己的 accountID
凭据	`pt-…` 个人令牌放在文件里，owner-only，每次调用重读；也可以用来源卡片上的**「用云效账号授权」**走 OAuth 2.0（授权码 + PKCE，浏览器登录后由本地回环回调接回，令牌写进同一个文件，24 小时有效并用刷新令牌自动续期，刷新令牌 90 天）
路径安全	限制在 `/oapi/v1/`，并有组织白名单；点段逃逸（明文或百分号编码）在任何请求之前就被拒
出站	评论，标记幂等
状态回写	两个状态字段现在是**下拉框**：先点来源卡片上的**「读取云效状态词表」**（或字段旁的刷新按钮），它按类别列出工作项类型（`ListWorkitemTypes`），再读每个类型的工作流（`GetWorkitemWorkflow` 中心版），把状态以 `ID → 名字` 存下来，供下拉直接选——显示的是看板上的名字，存的仍是状态 ID；填名字或 ID 都行，轮询读到的状态作为回退。本实例两处都没见过的名字会被拒绝并说明去哪取，不会当成状态发给云效

方面	细节
评论附件	打开「允许模型把文件挂到评论」后，worker 报告里的文件（执行目录内、≤10 MiB、非可执行）先上传到工作项附件，再以 **RICHTEXT** 评论发出：正文转义成段落，图片用云效返回的 `embedHtml`（原生 ``）并附一条文件名链接，其它文件用链接；链接一律用**永久代理地址** `embedUrl`（不用会过期的 `url`），标记以 HTML 注释同行；没有附件时仍是原来的纯文本评论（保留 Markdown）。默认关闭
成员名单	打开「从项目成员同步名单」后，策略卡片上的「同步项目成员」会把名单换成项目成员（`userId`，即工作项「指派给」里那个 id），保留你排好的顺序、同步名字、追加新成员；启动时同步一次。同步与飞书一样只作用于**默认实例**（实例 id 就是 `projex`）
面板检索	输入编号（`ESFX-6228`）或工作项页面地址，就能定位到该工作项的触发与工单（工作项页面用编号寻址，线程用内部 id，所以链接到线程仍按会话 id 走）
已删的工作项	工作项进回收站后，云效的评论/状态接口答 400「已进回收站」；插件把它当成**已消失**：把线程标记 `gone`、不再读它，也不再把它算作轮询错误（删东西是人的操作，不是插件失败）
站点	留空用中心站 `https://openapi-rdc.aliyuncs.com`；Region 版在「API 站点」填实例域名，授权与调用都走同一站点
已知缺口	对真实实例的写路径（状态 PUT、附件上传与评论正文）未在本机验证（没有 token）；`GetWorkitemWorkflow` 支持中心版与 Region 版，但需要项目里有可用的工作项类型

真机自检有两条现成路径：面板上点「用云效账号授权」；或在终端跑 `python verify/oauth-check.py --token-file`（同一套 OAuth 流程，打印授权地址、写同一个令牌文件，不打印令牌）。拿到令牌后 `LIVE=1 YUNXIAO_ORG=… YUNXIAO_PROJECT=… YUNXIAO_TOKEN_FILE=… npx vitest run tests/live/projex-contract.live.spec.ts` 会只读校验工作项、评论、成员名册和状态词表。

3.2.1 三段闭环：评估 / 执行 / 反馈

工作项通道此前只做到「收得到、回得去」，三段里各缺一块，协作就卡在这三处。现在三段各有明确的落点。

段	缺什么	现在怎么做	位置
评估	触发器只带自己那段文字。评论触发的评估既看不到标题、也看不到描述，更看不到之前的回答——模型只能对着「这个怎么样了？」猜整件事，于是要么乱接、要么全程 `ignore`；而且评论触发不带类别，准入表按默认格子判，操作员对某个类别的接收决定对评论不生效	**把工作项读成一条会话**：描述是根消息（`[ESFX-6228]` 标题 + 正文），评论按时间排在其后，按宿主的页继续读；agent 自己带标记的评论标成 `bot`，不会被当成人的请求；内嵌图片按本地归档读回成附件。评论触发同时带上它所在工作项的**类别**，于是评论和它的工作项落在准入表的同一行	`src/sources/work/context.ts`、`src/sources/work/stream.ts`
执行	工单里的 worker 不知道自己在哪个工作项上：写提交信息、开 PR、回评论都没法点名，只能泛泛说「这个改动」	**每个工单带上工作项身份**：`DSH_SUPERAGENT_PROJEX_PROJECT` / `_WORKITEM` / `_SERIAL` / `_CATEGORY` / `_STATUS` / `_URL`（外加 `_ORG`）。它们出现在执行环境提示词里，和 GitHub 的 `DSH_SUPERAGENT_GITHUB_REPOSITORY` / `_NUMBER` 同一个位置	`src/sources/work/environment.ts`
反馈	评论之外看板一片死水：工作项从接单到交付一直停在原状态，别人只能翻评论才知道机器人是否在做	**状态跟着工单走**：`writeBack` 打开并填「开始执行时写入的状态」「工单结束时写入的状态」后，工单一起步就写开始状态，工单结束（完成 / 合并 / 放弃）再写结束状态。两个字段**填状态名或 ID 都行**——名字按云效工作流词表解析（来源卡片的「读取云效状态词表」用 `ListWorkitemTypes` + `GetWorkitemWorkflow` 读出来并存下），轮询读到的状态作回退。每条工单每个阶段只写一次（KV 账本），写前先读一次、已经是该状态就不写；失败只记进轮询结果，下一轮重试	`src/sources/work/feedback.ts`

三点各自的理由：

- **评估要的是会话，不是句子。** 云效没有 `@`，准入表由负责人（没有则创建人）是否在名单里决定，所以评估是唯一读得到「他到底要什么」的地方。给它一条只有半句话的触发，准入表再准也没用。
- **执行要能点名。** 工作项编号（`ESFX-6228`）是人搜索、评审、写提交信息用的名字；没有它，交付物和看板对不上号，人只能自己拼。

- **反馈要落在看板上。** 评论是写给一个人的，状态是写给一个团队的：站会上看的是列，不是评论。回写因此是**可关**的（`writeBack` 默认 false），填了状态才生效，且**人工停止的工单不写**——停它的人自己知道。

3.2.2 与 GitHub、飞书通道的对照

能力	GitHub	飞书	云效 Projex
评估上下文	`threadContext`：Issue / PR 的整条讨论	`threadContext`：话题内消息与图片	**本次补齐**：描述 + 全部评论 + 内嵌图片
谁算发起人	名单里的账号	名单里的成员	负责人（没有则创建人）的 accountID
执行环境变量	仓库、账号、提交邮箱、编号、PR head	—（会话通道靠提示词里的来源上下文）	**本次补齐**：项目、工作项、编号、类别、状态、URL、组织
反馈·接单	评论	消息	评论
反馈·结论	评论（`announceEnd`，结束自动补一条）	消息	同左：结束自动补一条「已完成 / 已合并 / 已放弃」
反馈·状态	由 PR 合并 / 关闭体现	—	**本次补齐**：看板状态回写，名字来自云效工作流词表
反馈·附件	PR 里的文件	消息附件	**本次补齐**：文件上传到工作项附件，以 RICHTEXT 评论嵌入（开关默认关）
面板名单	「能发起工作的账号」+「评论可成为候选的机器人」	「能发起工作的成员」（可同步群成员）	「能发起工作的云效成员」，**可同步项目成员**（开关默认关）
面板检索	`#12` / Issue、PR 地址	—（聊天按消息）	`ESFX-6228` / 工作项页面地址
拒绝回执	不发（公开记录）	发（会话）	不发（公开记录）

仍然不一样的、且刻意不一样的地方：工作项没有「@」，所以准入表里「名单外·点名」那一格永远是「—」；工作项也拿不到成员名册以外的项目信息，所以「同步项目成员」只同步项目成员，不会把整个组织的人拉进来。为什么状态按云效工作流解析、附件以 RICHTEXT 嵌入、同步为什么只作用于默认实例，以及登录与插件独立性的取舍，见 ADR-009 与 ADR-010。

3.3 微信 — `chat/weixin`

桌面客户端没法从 Node 驱动，所以它放在一个**回环 HTTP sidecar** 后面，契约是冻结的。 Node 侧只认契约，后端因此可替换。第 4、5 节是细节。

方面	细节	
监听什么	面板里配置的会话：群（按结构化 @ 判定）与单聊（按会话即可，私信里没有 @ 这一说）	
传输	轮询 sidecar（`pollSeconds`，最小 5），而 sidecar 本身被客户端的**监听器推送**喂养	

方面	细节	
身份	`selfWxid` 是「谁的 `@` 算数」；会话与发送人用 sidecar 给的 id	
凭据	sidecar 的 bearer token 放在文件里，owner-only	
群 `@`	只看 `ats`，绝不看消息正文 —— 见 §5.2	
出站（文件）	`/send` 的 `file` 可带一个文件/图片（base64，≤20 MiB，文件名只留 basename）；真机验过发出后是图片消息。核心侧已有附件模型（`OutboundMessage.files`）。**worker 在 ready 报告里用 `files` 声明产出的文件**（相对工单目录），主 Session 的结果回复**默认自动附上**它们（也可在 `orchestrator_reply` 显式给 `files`）：**只允许工单目录内的文件**、≤20 MiB、拒绝可执行后缀，且受面板开关 `sendFiles`（默认关）约束——见 ADR-008	
出站	文本，**keyed 幂等**（OutboundMessage 的 id 作为 `Idempotency-Key` 发出），进程内串行且间隔 ≥1 秒。群里可按开关带 @（`mentionBack`，按发送人 id 解析成显示名）；**没有原生引用**——客户端库的 `SendMsg` 没有 quote 参数，见 ADR-007	
入站媒体	图片，经 sidecar 下载，按魔数判定；群与单聊走同一条 `/media` 链路。面板可关（关掉只在正文留 `[图片]`，不下载）	
入站语音	转写为文字（`to_text`，每条最多占用客户端 UI 8 秒，失败降级为占位）。面板可关，关掉正文是 `[语音消息]`	
自己的写入不算新请求	云效的工单触发器按 `gmtModified` 派生，而 agent **回评、回写状态**都会改这个时间——于是自己触发自己（ESFX-6228 实测：同一工作项 2 个触发器，第二个是 6 分钟后的状态回写导致的重复派发）。所以适配器写完会回读一次该条、把产生的版本记进**进程内账本**，轮询到同一版本就跳过；人之后真正编辑会得到新版本，照常触发。账本按「主机 + 组织 + 工作项 + 版本」记录，上限 512 条、24 小时过期	

方面	细节	
请求写在描述里	云效的**列表**接口把 `description` 返回成空串（`formatType: null`），**详情**接口才有内容，还是 RICHTEXT 的 JSON（`{"htmlValue":"…"}`）。所以轮询时若某条工作项没有正文，会按 id 读一次详情、并把 HTML 剥成纯文本——否则智能体只看得见标题，看不到你要它做的事（ESFX-6228 就是这么把「告诉我网址」弄丢的）	
完成也要回评	接单回执由引擎自动发；**完成结论以前只靠主 Session 主动调 `orchestrator_reply`**，漏了就只剩接单回执、来源永远等不到结果。现在工单**结束**时会在它发起的那条会话里补一条「已完成/已合并/已放弃：」，主 Session 已经回过就不再发第二条；机器发起的工单、以及被人工停止的工单不发。频道有长度上限时先试摘要、再退到一句短的，两边都放不下就静默并在事件里报 `result.announceFailed`	
设置页的容错	某个来源的准入行由它自己的配置推导，**推导失败不能拖垮整个设置页**——否则面板打不开、什么都存不了（2026-11-09 事故：云效的行用严格 schema 校验整份实例配置，`organizationId` 被判为未知键，于是「该设置页保存报 `Invalid settings:`」）。现在设置视图对行的构建做了兜底：失败就不显示这张表，其余照常可存	
核准表的行与 scope 一致	钉钉、企业微信没有「群列表」配置，它们的准入行就是 scope 报的那三行：群里 @ 机器人、群里未 @、私聊。行 id 与 scope 不同的话，人配了也没有代码去读，所以以前这两家整张表都是「—」，配不动。现在按 atOnly 决定「群里未 @」那行是否可配	

方面	细节	
来源与评估的分工	来源只管「看不看得见」：收哪些群、收不收私聊、每个群交多少（只交 @ 机器人的消息，或全部消息）。评估只管「算不算工作」：谁能发起、在哪个范围里、按什么强度接。所以「接收的群 / 接收私聊 / 每个群的接收方式」在来源；「能发起工作的用户 + 准入表」在评估。群的「接收」是**每行一个**（与飞书的群列表一致），全局的「仅接收 @」降为「新群的默认接收」，只在群自己没设时生效。来源卡片里没有、也不该有「允许发起工作」开关——那是第三处授权，已删除	
微信名单不标来源	名单只有「用户 ID + 名字」，与飞书一致。群消息还是私聊消息由消息本身决定，准入表据此自动分行；名单里不设「群/私聊」列。同步用的归属标记是内部数据（同步写入的行会被下次同步替换，手加的行保留，按 id 去重、手加优先），不占界面	
微信桥谁启停	全部在面板上：来源卡片「高级 → 桥启动器」填本机启动脚本的绝对路径，「启动桥」由插件以**后台、无窗口、脱离 DSH** 的方式拉起它（所以桥能活过 DSH 重启），「停止桥」按桥自己的端口找到监听进程、核对命令行确属 sidecar 后再结束，「重启桥」= 停再起，「检查桥」读一次 `/health` 说明能力或不可用原因。启动脚本的内容留在本机（客户端库与 shim 都不进仓库）。没填启动器时退回旧的计划任务方式	
微信桥谁启停（旧）	sidecar 是独立的常驻进程：要跑在已登录的**交互桌面会话**里、要活过 DSH 重启、本机 shim 不能进仓库，所以插件**不自己 spawn 它**。来源卡片上的「启动桥 / 停止桥 / 重启桥」只对本机**计划任务**（默认 `DSH-WeChat-Bridge`，可在「高级」改）执行 `schtasks /run	/end /tn；任务名在 `schema` 和动作里各校验一次，按钮永远不会变成任意命令。「检查桥」读一次 /health`，把后端能力和不可用原因直接说出来

方面	细节	
微信「立即同步」同步什么	它把所选群里**桥能给出的人**加进「能发起工作的微信用户」名单。能给出多少取决于后端：wxautox4 没有群成员表接口（`GetAllRecentGroups()` 只有群名和成员数），`capabilities.members` 是 **observed**，只能返回**在这个群里发过言的人**；因此同步是**并集**（只加不删），安静成员不会被误删，也不会被同步进来。后端返回完整名册（members=full）时才用替换语义。拿不到任何人时同步会明确报错，而不是静默成功。它不读聊天内容、不改「接收的群」	
准入表统一形状	五个来源同一套：**行 = 接收范围**（每个已配置的群/私聊/其他群；云效是需求/缺陷/任务；GitLab 是新 Issue/新 MR/评论/CI），带「仅 @ / 全部消息 / 接收 / 不接收」小注解；**列只开该范围真实可用的格子**，其余显示「—」。会话类现在**每个会话一行**（`scope` 返回会话 id 作行名，以前所有群共用一个 `chats` 行）；GitLab 之前**没有任何 Policy**（`hostingKind` 未声明），既无准入表也无法授权，现已补上「能发起工作的账号」与「评论可成为候选的机器人」两个名单	
云效的准入表	与飞书、GitHub 同一套交互：**行 = 接收范围**（需求/缺陷/任务，按实例的「类别」显示「接收」或「不接收」），**列 = 真实可用的那几格。**工作项没有任何字段在点名智能体，所以「名单外·点名」那一列在云效里显示为「—」；只有「名单内·未@」和「名单外·未@」可配。**负责人（没有则创建人）是否在名单里决定落到哪一列，两列各配什么就按什么处理**——「忽略」不接，「允许推断」成为候选并在「自动接单」模式下接单。指派本身不再等于明确请求	

方面	细节	
云效怎么才算接单	工作项没有「@」，所以准入表说了算：负责人（没有则创建人）在名单里就落到「名单内」那一列，不在就落到「名单外」；两列各配什么就按什么处理（忽略 / 允许推断，后者需「自动接单」）。指派只决定落到哪一列，本身不等于接单	
云效 Projex 的接口	轮询走 `POST /oapi/v1/projex/organizations/{org}/workitems:search`：项目以 `spaceId` 传入，裸数组响应，`page`/`perPage` 分页；时间窗用 `conditionGroups`（数组套数组 + `className: dateTime`/`format: input`）。旧路径 `GET /listWorkitems` 已在网关上下线，会一直返回 404。评论正文是 `content` 里的 `htmlValue` JSON，作者字段是 `user`；状态回写是 `PUT /workitems/{id}` 带 `{status: }`（`POST .../status` 同样不存在）	
执行目录的根目录	必须是**绝对路径**：`/srv/work`、`D:\Work`、`D:/Work`、`\\server\share` 都行；相对路径会被拒；留空则用 `\work`。2026-11-09 之前只接受以 `/` 开头的路径（schema 与输入框 pattern 两处），Windows 盘符路径填不进去	
state 目录锁	一个 state 目录只允许一个 dsh-superagent。锁超过 `LOCK_STALE_MS`（5 秒，持有者每 2.5 秒刷新）就算陈旧，会被先清掉再取；真被占用时最多等 6 秒，失败信息写明「关掉另一个实例」或「删掉 `.lock`」。**重启时最常见的故障就在这里**：旧进程被杀后锁还在，新进程在 30 秒内会被判「已被持有」，整个插件不生效（面板显示「没有响应」）	

方面	细节	
state 目录锁	一个 state 目录只允许一个 dsh-superagent。锁超过 `LOCK_STALE_MS`（5 秒，持有者每 2.5 秒刷新）就算陈旧，会被先清掉再取；真被占用时最多等 6 秒，失败信息写明「关掉另一个实例」或「删掉 `.lock`」。**重启时最常见的故障就在这里**：旧进程被杀后锁还在，新进程在 30 秒内会被判「已被持有」，整个插件不生效（面板显示「没有响应」）	
重挂补读	监听断开期间的消息本来会永远丢失（无回调 → 无队列 → 无引擎记录）。现在每次重挂后读回被监听会话的最近 20 条并补进队列（最多 3 个会话、每分钟一次，`/health` 写「重挂后补读 N 条」）	
监听集合	sidecar 的事件队列只由**监听回调**填充，而它只为 `/listen` 里列出的会话触发。所以监听集合 = 「接收的群」∪「能发起工作的用户」——名单里的人会被自动监听，他们的私聊才送得到。这是 2026-11-09 07:18 那条消息没有任何记录的原因：群列表空了、名单里的人也没被监听	
私聊	微信的私聊是**一条流**（不是每人一个会话，`/messages?room=` 与 `room=` 返回同一批），所以人不需要出现在「接收的群」里：来源里「接收私聊」开着就会读这条流，谁能发起工作由评估的名单决定——**人只配一次**	
一个名单	**群**在「来源 → 接收的群」里配（决定收哪些群）；**人**只在「评估 → 策略 → 在各来源上 → 能发起工作的微信用户」里配，和其他通道一致。私聊不需要在来源里配：它本来就收得到，回复也不需要额外条目。名单为空时策略卡片直接警告	
面板配置	`POST /listen` 集合替换，同一次请求还带 `options.voice` / `options.images`；增删会话与翻转开关都不用重启	
关键词	`keywords` 里的词出现在群消息正文里，就**不需要 @** 也算找机器人（大小写不敏感）；是否成为可接的工作仍由准入模型判断	

方面	细节	
关键词自动回复	`autoReplies` 命中即由轮询循环直接回固定文字，不经过模型；**这条消息仍交给模型**，不用模板吞掉对方剩下的内容	
游标	`/messages` 的 `head` 让客户端发现「桥重启过、序号回退」并从头再读；序号由时钟播种，跨重启单调	
拒绝回执只发给会话	「这条我没有接：…」只发给**和机器人对话的人**：微信、飞书这类会话频道在 `channel.acceptsNotice` 上显式开启；**工作项（云效/GitHub Issue 等）一律不发**——那是公开记录，评论一句「你不在名单里」既不是回答，还会把配置播出去。一次事故就是这么来的：16 条拒绝评论落在别人的需求、任务、缺陷上。现在两处拦：引擎在入队前问 `SourceScope.acceptsNotice`，registry 在发送前再拦一次（返回 `gone`，收据记为 dropped）	
拒绝回执	消息被**判为忽略**、且是 @ 机器人或私聊机器人时，回一句「这条我没有接：」（名单外、没看出请求、已有工单…）。群里的普通消息不回，免得刷屏	
收到回执	有人 @ 或私聊机器人时，**在评估之前**先回一句（`acknowledge` / `ackText`，同一会话 30 秒内只回一次）。实测评估要 53 秒～2 分 09 秒，回执不缩短它，只让你不用干等	
处理中提醒	回执发出后机器人在该会话一直没开口，就每隔 `progressAfterSeconds` 补一句「还在处理中」，最多 3 次；机器人一说话就停	

同一台机器上只能有一个 UI 自动化客户端。 *微信渠道是靠驱动微信客户端界面工作的，若同时还跑着另一个同类工具（例如 SiverWXBot），两边会抢同一个窗口：后驱动的一方会让先挂上的监听器失效，表现为「界面在动，*

但再也收不到消息」。真机验证过：关掉另一个客户端后重启 sidecar，监听立刻恢复。

*sidecar 现在会自己发现这件事：连续 10 分钟没有任何真实事件就判定挂载哑掉，**先摘后挂**并重调 `StartListening`（每 15 分钟最多一次），`/health` 的 `detail` 里写「最近事件 K 秒前」和「已自动重挂 M 次」。*

M 在涨就是界面被抢过——排查下一个会动微信窗口的程序。

***换号/重新登录**：客户端会换窗口。sidecar 每 30 秒确认一次「我挂的那个窗口还在吗」，不在就清掉所有按账号缓存*

*（会话、群、成员、队列、监听）并重新 attach；attach 失败时**不退出**，`/health` 报 `available:false` 并带原因，*

每 30 秒重试——所以在微信里登录完成后不需要重启桥。

***客户端版本是有前提的**：`wxautox4 41.2.3` 面向 4.1.9 这一代。客户端被升级到 4.1.15.56 后，即使窗口可见、账号*

*已登录，attach 也会一直报「未找到已登录的客户端主窗口」。`/status` 的 `client` 字段就是为此加的——**升级前先看它**；*

出现这个报错时，先确认客户端版本，而不是怀疑配置。

3.3.1 与飞书通道的对齐（本轮补齐）

微信此前已经有的：结构化 @ 判定、群与私聊、入站图片、语音转写、出站文本与文件、keyed 幂等、即时回执、处理中提醒、关键词/自动回复、回复时 @ 发言人、按实例隔离、监听自愈。下面是与飞书逐项对照后补齐的：

能力	**飞书**	**微信（现在）**						
工作线程读会话	`feishu//message	root	parent	thread	around`，worker 可读	**新增** `weixin//message	thread	around，`worker` 可读；next_resource`翻页
会话技能	`dsh-superagent-feishu`	**新增** `dsh-superagent-weixin`（说明资源语法与图片标记；微信没有 CLI，也没有消息深链）						

能力	**飞书**	**微信（现在）**						
评估用的会话上下文	thread/ root/ parent/ around	thread/ **parent**/ root；微信没有话题根，`root` 明确返回 `no_root` 而不是编一个						
面板按消息检索	`om_…` → 检索词	**新增** 平台消息 ID → 检索词；`meta.nativeId` 精确命中						
回复映射（谁回答谁）	`repliesTo` 由 parent 推导	**新增** `repliesTo`，工单线程里显示"这条回答的是哪条"						
每页消息数	`readPageSize`	**新增** `readPageSize`（1–50，桥的上限 50），评估与资源读取共用						
成员名册同步	`syncChat` + 立即同步	已有（`syncChat` + 立即同步，按桥能看到的成员）						
面板事实	机器人身份	已有（登录账号 + `@` 证据强度）						

能力	飞书	微信（现在）						
入站图片	有	已有；评估侧可把 `[image:]` 取成附件；worker 按技能里的一条 `curl/Invoke-WebRequest` 从 `GET /media/` 下载后 `read_image`						
历史回补	加机器人回补最近历史	**已有**：重挂/首次监听时桥读回最近 20 条（每轮最多 3 个会话），现在标记 `historical`，只进评估、不回执、不自动回复						
消息深链	AppLink	**做不到**：sidecar 没有深链接口，`link` 返回空地址（ADR-007 已记）						
机器人名单	可把机器人消息当候选	**做不到**：微信里没有"机器人账号"这个概念，桥一律报 `bot:false`，不假装能识别						

仍有两项是微信后端物理限制，不是没做：**没有消息深链**（sidecar 没有深链接口），**没有机器人名单**（微信没有"机器人账号"概念，桥一律 `bot:false`）。这两条写进了 ADR-011。

3.4 钉钉 — `chat/dingtalk`

方面	细节
监听什么	群里发给机器人的消息，以及私聊
传输	**Stream**（长连接），重连可被中断
身份	`clientId` + `robotCode`
凭据	`clientSecret` 可内联或放文件，owner-only
能力	`idempotency: none`、`maxChars: 5000`、`markdown: true`、`proactive: false`
诚实的弱点	出站是**至少一次**：没有服务端幂等键，sessionWebhook 也无法去重，重试可能重复发送。这个 kind 声明了它，而不是假装没有
已知缺口	Stream 传输需要注入 `dingtalk-stream` SDK 的 `connect`；没有它，kind 只注册不启动

3.5 企业微信 — `chat/wecom`

方面	细节
监听什么	应用的消息：按 `chatid` 的群聊，以及私聊
传输	**webhook**（签名 + 加密回调）
身份	`corpId` + `agentId`
凭据	`corpSecret` 可内联或放文件；回调 `token` 与 43 字符 `encodingAesKey`（或放 `encodingAesKeyFile`）用于验签解密
回调地址	`/wecom/`：插件自己挂这条公共路由，运维把它填到应用「接收消息」的 URL
会话上下文	评估读回调自带的那条消息（`thread`）；`root/parent` 明确回答 `no_root/no_parent`，因为回调既没有话题也没有引用，更没有历史可翻
入站图片	`MsgType=image` 的 `MediaId` 变成 `[image:]`，评估按 `GET /cgi-bin/media/get` 下载成附件
面板检索	十进制 `MsgId` 变成检索词；`repliesTo` 存在，但回调不带被引用消息，通常为空
worker 读原文	`wecom//message`（同飞书的资源语法，一条消息、无下一页）
加密	验签、AES-CBC 解密、重放拒绝（重投的消息不会产生第二个事件）
能力	`idempotency: none`、`maxChars: 2048`、`markdown: false`、`proactive: true`
诚实的弱点	出站至少一次；消息 API 没有幂等键
已知缺口	没有历史回补、没有消息深链、没有群成员名册；worker 只能读回调带来的那一条消息

3.6 提示词技能 — 六个来源一套

每个 Session（主 Session 与执行器）都注册六个内置来源技能，和飞书、GitHub 一样可以在「提示词 → 团队约定」里逐段查看与覆盖（逐项能力对照见 capability-matrix.md）：

技能	文件	讲什么				
飞书技能 dsh-superagent-feishu	skills/dsh-superagent-feishu/SKILL.md	`feishu// message	root	parent	thread	around`、lark-cli 用法、附件与文档的追读
GitHub 技能 dsh-superagent-github	skills/dsh-superagent-github/SKILL.md	gh 的读法、Issue/PR 上下文、发布前检查				
微信技能 dsh-superagent-weixin	skills/dsh-superagent-weixin/SKILL.md	`weixin// message	thread	around、[image:] 与 GET /media` 下载、无深链的诚实边界		
云效 Projex 技能 dsh-superagent-projex	skills/dsh-superagent-projex/SKILL.md	工作项即线程根、评论即回复；DSH_SUPERAGENT_PROJEX_* 身份变量；用 orchestrator_reply 交付，状态回写由插件负责				
企业微信技能 dsh-superagent-wecom	skills/dsh-superagent-wecom/SKILL.md	回调只带来当条消息、没有历史与深链；纯文本 2048 上限；发送 API 无幂等键，重试会重复				
钉钉技能 dsh-superagent-dingtalk	skills/dsh-superagent-dingtalk/SKILL.md	流回调只带当条消息、无历史无深链；Markdown 5000；投递至少一次；@ 按 staffId 结构化判定				

实现上，`composeSkill` 从内置分段 `skill/` 取文本（面板据此展示与覆盖），团队自定义技能仍用自己的文本；`tests/model-surface.spec.ts` 把所有来源声明的环境变量并成一个集合来校验技能里点名的变量，新增来源不会再因为"变量未登记"而误报。

4. 微信 sidecar

一个独立的 Python 进程（只用标准库），监听 `127.0.0.1`，用 owner-only 文件里的 bearer token 鉴权。 契约冻结并带版本；Node 侧是它的参考消费者。

端点	作用
`GET /health`	sidecar 与后端是否可用，以及 `capabilities`
`GET /status`	已登录账号，以及当前用于判定 `@` 的显示名
`GET /rooms`	后端**能看到的**会话（下拉候选），不是正在听的
`GET /members?room=`	该会话里见过的联系人（或按能力给完整成员表）
`GET /messages?room=&cursor=&limit=`	游标之后的事件，带 `next`
`POST /send`	发文本，可选引用，可选 `Idempotency-Key`
`GET /media/`	某条消息图片的字节；失效 `404`，构建不支持下载 `501`
`POST /listen`	集合替换式设置监听；幂等，只对差异动作

契约守的不变量：

- **`ats` 绝不从消息正文推导。** 拿不到 mention 列表的后端要如实声明，不得退化为匹配正文里的 `@名字`。
- **`/media` 绝不返回「200 + 零字节」。** 空载荷就是 `404`。
- **`/send` 进程内串行**，两次发送间隔 ≥1 秒，`/media` 共用同一把锁——它们驱动同一个客户端 UI。
- **`/rooms` 只读。** 绝不因为被调用而顺手启动监听。
- **bearer token 不出现在任何日志行里。**

5. 微信后端阶梯

5.1 阶梯

层	后端	适用
L0	`wxauto` / `wxauto4` / `wxautox4`（UI 自动化）	默认；需要一个交互式 Windows 桌面会话
L1	`wxhelper` 一类注入插件	**已划掉**：只适配微信 3.9.x，且消息载荷里没有 mention 列表，装了也拿不到 `@`（ADR-005 §7）
L2	`WeChatPadPro` 一类 iPad 协议服务	没有 Windows 机器时
L3	托管 API	其它全挂且业务不能停

只实现了 **L0**。L2 是桩：`available: false`，数据端点 `501`，所以将来切过去是改配置 + 重启，而不是改代码。

5.2 群 @，以及为什么它分档

微信 4.x **没有结构化 mention 列表**：消息气泡是扁平的无障碍叶子节点 （`childCount == 0`，没有 text/value/legacy pattern），消息对象上也没有 mention 字段。 所以群 @ 读的是**客户端自己的标记**——从 mention 列表选出的名字后面跟 `U+2005`（四分宽空格）， 手打 @名字 用的是 `U+0020`——外加 @所有人。

这比子串匹配强、比结构化列表弱，所以能力自述要说清是哪种：

值	含义
`structured`	客户端给了 mention 列表（`fake` 参考后端，以及将来的 L1/L2）
`markup`	客户端自己的标记，对本账号解析（当前 L0）
`none`	完全判不出

判据用的本账号显示名按 **60 秒 TTL** 重读、每次 `/listen` 强制重读、轮询路径顺带刷新， 所以改昵称不用重启就能自愈。读失败保留旧名而不是清空——清空会让判定永远不命中，比旧值更糟。

5.3 L0 给不出的东西

契约字段	L0 现实
`room`（chatroomId）	拿不到；会话命名为 `wxname_`
`sender.id`	显示名，同样命名空间化；`sender_info()` 给的是微信号，且滞后一条消息、有时为 null
`at`（消息时间）	有客户端时间分隔符就用它，否则用接收时间
`/members`	只有见过的发送人（`members: observed`）

命名空间化是有意的：它保住了设置里的 id 正则的严格性，而不是为了迁就显示名去放宽；`refreshLabels` 再把 id 解析回名字给面板的下拉用。代价是改群名会换 id。

6. 新增一个 provider

40. 实现那份契约。聊天类需要：列会话、按游标读消息、发送、（可选）下载媒体。工作项：列/读/评论。托管：变更请求、评论、CI。
41. 如实声明 `capabilities` —— `idempotency`（`keyed` / `marker` / `none`）、`maxChars`、`markdown`、`proactive`。

面板会显示它们；写错就是给出一个错的承诺。

42. 写 kind：schema、字段（schema 每个键都要有对应字段）、`scope` 规则、环境变量。
43. 加一个 `*-watch.ts` 插件入口，以及 `cordis.patch.yml` 里的 insert。**不要**给它写 `disabled: true`：kind 只有在插件注册之后才会出现在面板里，而「跑不跑」由面板的开关决定。
44. 把文件行补进 `docs/007-architecture.md` —— 模块或测试文件缺行，`tests/docs.spec.ts` 会红。
45. 补 spec。真实行为放 `tests/live/*.live.spec.ts`，由 `LIVE=1` 门控。

7. 验证矩阵

对象	怎么证明
全量	`vitest run --maxWorkers=2`，macOS arm64 与 Windows 11 双平台绿
微信契约	真实 sidecar 进程 + 回环：9 条 live 用例（端点、`ats`、幂等发送、图片字节、`/listen`）
微信真机	真实微信 4.x：监听回调、本账号身份、发送往返、图片下载、五条标记用例证明手打 `@名字` 不算
GitLab	真实 `gitlab.com` API：MR 映射、伪 token 映射为 401 且不回显
企微 / 钉钉 / 云效	真实鉴权失败被映射成可诊断错误，消息里没有密钥
面板数据层	同通道两实例、删除其一、无通道 id 的旧设置
Windows	首次 clone 时失败的 16 个文件与实测基线，见 HANDOVER-WIN11.md

未验证、也不声称：对真实代码托管/工作项的任何写操作；企微回调走公网反代；钉钉 Stream 植入 SDK； 微信的 `structured` mention（目前没有任何一层提供它）；企微/钉钉的消息图片。

8. 改代码时要守的约定

- **`@` 绝不按文本判定。** `scope` 只读 `ats`。后端可以由客户端自己的标记得出 `ats`，但强度必须声明在 `capabilities.mentions` 里，让人看得出一个群 `@` 是证据还是线索。
- **一份状态一个写者。** Source 只能经 `SourceContext.tickets` 改工单；健康条目按实例 claim；回执只由 Outbox 写。
- **幂等是声明的。** `keyed` / `marker` / `none`，面板显示代码的真实行为。
- **密钥来自文件**，每次调用读取，owner-only，且绝不出现在消息或日志行里。
- **后端差异留在 sidecar。** 身份归一属于那里，不属于设置 schema。
- **`src/sources` 不 import `src/app`。** 双方都要用的类型留在 `src/sources`。
- **不放个人数据、不放机器专属路径。** 文档与夹具用占位符；路径走参数或环境变量。

9. 许可

MIT，与本版所生长的原型一致；上游版权声明保留。微信的 UI 自动化是第三方工具： 未经腾讯授权，可能随时被客户端或平台限制，必须守在 ADR-005 记录的边界内——只在被 `@` 时回复、不群发、 不批量加好友、同一时刻只有一个账号在线。

能力矩阵：四个新增来源对照飞书与 GitHub

关于「对齐」意味着什么的参照。飞书与 GitHub 是上游 superagent 自带的两个来源；另外四个是本 bundle 新增的。✅ 具备 · — 该平台不适用 · ❌ 后端不可能（如实说明，绝不伪造）。

能力	飞书（官方）	GitHub（官方）	微信	企业微信	钉钉	云效 Projex
接收消息	✅ 轮询历史 + 事件	✅ 轮询通知	✅ sidecar 轮询	✅ 签名回调（插件挂载 `/wecom/`）	✅ 流（需注入 `connect`；未随附 SDK）	✅ 轮询工作项/评论
评估 / 判定 / 执行 / 回复循环	✅	✅	✅	✅	✅	✅
结构化 @ 判定	✅ open_id	—	✅ `ats`（markup 回退，强度见 facts）	✅ `mentionsAgent`	✅ `ats staffId`	—
单聊	✅	—	✅	✅	✅	—
出站文本	✅ markdown	✅ 评论	✅ + 回复时 @ 回去	✅ 纯文本 2048	✅ markdown 5000	✅ RICHTEXT/Markdown 评论
出站文件	— （CLI）	—	✅ `sendFiles`	❌ API 仅文本	— （webhook）	✅ 附件
幂等发送	✅ 带键	✅ 标记	✅ 带键	❌ 无（已说明）	❌ 无（已说明）	✅ 标记
供评估使用的线程上下文	✅ thread/root/parent	✅ PR/issue	✅ thread/parent；`root→no_root`	✅ thread；`root/parent→no_*`	✅ thread；`root/parent→no_*`	✅ 工作项正文 + 评论
worker 读取原文	✅ `feishu//…`	✅ `github//…`	✅ `weixin//…`	✅ `wecom//message`	✅ `dingtalk//message`	✅ `projex//item、/comments`
供评估使用的入站图片	✅	—	✅ `[image:]、/media`	✅ `MediaId、/cgi-bin/media/get`	— （文本流）	✅ 工作项/评论图片已归档
面板检索	✅ 消息 id	✅ issue/PR 编号	✅ 原生 id	✅ 十进制 `MsgId`	✅ `msgId`	✅ 流水号或页面 URL
回复→父消息映射	✅	✅	✅ （引用 id）	— 回调不带引用	—	—
显示名刷新	✅ 会话名	✅ 仓库名	✅ 房间/成员	✅ 部门用户（`refreshLabels`）	❌ 需要通讯录权限 API，未提供	✅ 状态词表
成员/名册同步进策略	✅ 会话成员	✅ 协作者	✅ 群成员	❌ 群成员不可读；用户列表只填名字	❌ 同上	✅ 项目成员

能力	飞书（官方）	GitHub（官方）	微信	企业微信	钉钉	云效 Projex
健康条目	`feishu + history`	`github`	`weixin`	`wecom`	`dingtalk`	`projex`
历史回填	✅ 会话历史	✅ 通知	✅ 重挂后追赶（标记 `historical`）	❌ 仅回调	❌ 仅流	✅ 首见评论
消息深链	✅ AppLink	✅ issue/PR URL	❌	❌	❌	✅ 工作项页面
多实例	✅	✅	✅	✅	✅	✅

平台限制，以及为什么它们不是 bug

- 微信、企业微信与钉钉**没有深链**：三者都不暴露按消息的 URL；面板只报出来源。
- 企业微信与钉钉**没有历史**：两者都以推送方式投递（回调 / 流），且不提供回读端点，所以没有回填、也没有 `/thread` 分页。微信通过它的 sidecar 读取历史；飞书通过历史 API；Projex 通过工作项 API。
- **钉钉传输**：WebSocket 客户端是 `dingtalk-stream` SDK，本 bundle 有意不内置它。部署时注入 `connect()`（见 `src/sources/wecom…` 不——见 `src/sources/dingtalk-watch.ts`）；没有它，kind 会被注册但没有任何东西在监听，健康状态会说明这一点。流下游的一切（触发、ThreadContext、worker 读取、回复、策略）都已对齐。
- **企业微信 / 钉钉成员同步**：自建应用无法枚举一个会话的成员；企业微信从它被允许读取的一个部门填充策略里的*名字*，而两者都拒绝把一份可见用户列表当作会话名册。
- **企业微信 / 钉钉 / 微信图片**：微信与企业微信投递图片时带一个部署方可下载的媒体 id；钉钉的流事件携带文本，所以不声称有图片。

本轮新增了什么

46. `projex//item` 与 `/comments` 的 worker 读取（`src/sources/work/projex/read.ts`），带分页，且 agent 自己的评论被标记为 `bot`。
47. `dingtalk//message` 的 worker 读取与一个钉钉 ThreadContext（`thread` = 那一条消息，`no_root`、`no_parent`）——`src/sources/chat/dingtalk/{read,kind}.ts`。
48. 企业微信在通讯录用户列表上的 `refreshLabels`，外加一个 `departmentId` 设置，于是策略的行能显示名字。
49. 一个 `dsh-superagent-dingtalk` 技能，且 Projex 技能现在点名它的两个资源。
50. 测试：`tests/projex-read.spec.ts`、`tests/dingtalk-read.spec.ts`，外加既有的企业微信/微信/Projex 套件。

ADR-005 微信个人号群消息的后端选型与切换策略

- **状态**：Accepted（2026-11-06，方案 owner 已确认）
- **决策范围**：`src/sources/chat/weixin/*` 的后端形态、主备策略、部署前提
- **关联**：设计文档 §6.7、§6.7.9、§6.7.10；本文不重复背景，只固化决定

1. 背景（一句话）

微信群 @ 机器人是本项目的**必须项**，但官方 ClawBot 通道只覆盖单聊，且开源社区的主力后端在过去三年连续退场（itchat → gewechat → wcf）。因此**任何"选定一个后端长期依赖"的方案都不可接受。**

2. 决定

2.1 形态：Windows 常驻 sidecar + loopback HTTP

WeChat 客户端自动化跑在一台 **Windows 常驻机**上，以独立进程（`sidecar/wxauto-bridge`）暴露 `127.0.0.1` 的 HTTP 契约（见 `sidecar/wxauto-bridge/contract.md`），dsh-superagent-multi-channel 通过 `src/sources/chat/weixin/pc-uia/*` 适配器调用。

理由：

51. 客户端自动化与 Agent 运行时**故障域隔离**——微信掉线不该拖垮工单执行；
52. sidecar 可用任意语言/任意库实现，后端切换不动 DSH 侧代码；
53. 契约端点少（读、发、媒体、监听集合各一组），反序列化面小，安全面小。

2.2 主备阶梯（按优先级，不是并列候选）

层	后端	触发启用的条件
L0	`pc-uia`（wxauto 一类，UIAutomation，不注入进程）	默认
L1	`pc-hook`（wxhelper 一类，DLL 注入 + HTTP）	**已划掉**：只适配 3.9.x，且 hook 出的消息没有 `atuserlist`，装了也拿不到 @（见 §7 审计）
L2	`pad`（WeChatPadPro / 社区 fork，iPad 协议）	无 Windows 机、只有 Linux；业务书面接受协议风控
L3	`hosted`（付费 API）	L0–L2 全挂且业务不能停
旁路	企微客户群 / 官方 ClawBot 单聊	客户愿意迁群 / 只需单聊（属需求变更，不是技术降级）

同一时刻只有一个实例持有同一微信号的登录态（双发/风控红线）。

2.3 硬约束（写进 DoD，不满足则"能换"不成立）

54. `WeixinBackend` ≤ 8 个方法；单后端实现 ≤ 400 行。
55. dsh 侧对微信实例 `replyConcurrency = 1`，`/send` 由 bridge 串行 + 1s 间隔 + 抖动。

56. 切换后端 = 改配置 + 重启该实例，**不改 DSH 代码**；目标 ≤ 2 人日。

57. 群 `@` 判定只认 `ats`，**禁止**用文本 `@昵称` 反解。`ats` 可以由客户端自己的 mention 标记得出（见 §7 的 `markup` 档），但必须在 `/health.capabilities.mentions` 里声明强度，且不得退化为「正文里出现了 `@名字` 这个子串」。

58. 权限边界：只被动回复；不群发、不拉群、不加好友。

3. 后果

正面

- 任一后端退场，业务在 60s（主备接管）到 2 人日（跨后端迁移）内恢复。
- 群消息、@、群成员三件事在契约层被固定，Node 侧只依赖契约。

负面 / 代价

- 必须有一台 Windows 机器（或安卓真机）常驻；这是**运维成本**，不是代码成本。
- wxauto 作者免责声明禁止生产/商用 → 需业务 + 法务书面确认，否则 L0 不可用，直接落 L1。
- bridge 侧多一个进程 + 一套凭据（token + 登录态）要管。

不做

- 不做"多后端同时在线双写"（同一群双发）。
- 不把客户端自动化内嵌进 dsh 进程（故障域混同）。
- 不承诺"无风控"，只承诺"被限时能换"。

4. 验收（本 ADR 完成的标准）

59. `poc/probe_wxauto.py` 在目标 Windows 机产出 `probe-report.json`，且 **Q1（群消息字段 + 是否有结构化 ats）** 与 **Q2（群成员表）** 均有证据。

60. `sidecar/wxauto-bridge` 用 `--backend fake` 通过契约自测（本仓已可跑，见 README）。

61. 真实后端接入后，跑通设计文档 §6.7.8 六条剧本（含"改昵称后 `@` 仍生效"与"重试不重复发送"）。

62. 主备切换演练：L0 停进程 → L1 在 60s 内接管，群里无重复消息。

5. 回滚

- 单实例回滚：实例置 `enabled:false` → 该群不再接入，其余通道不受影响。
- 跨后端回滚：把 `backend` 配置改回上一层，重启实例；登录态需要重新扫码（微信不允许两处同时在线）。
- 彻底回滚：删除 `weixin` 实例配置，插件 `disabled`，行为等同 v1.0 方案（无微信接入）。

6. 未决前置（阻塞项，必须由业务方给出）

#	前置	谁给	不给的后果
1	Windows 常驻机（或安卓真机）承诺	业务/运维	L0/L1 全部不可用，只能落 L2（风险最高）或旁路
2	wxauto 生产/商用许可确认（法务）	业务 + 法务	L0 不可用，直接落 L1（注入风险）
3	目标微信 PC 客户端版本冻结策略（不允许自动升级）	运维	客户端升级即可能失效，切换成本回到"按周"
4	**探针与客户端必须在同一交互式桌面会话**（物理控制台或未锁屏的 RDP；不能从 SSH/服务/CI 会话跑）	运维	`wxauto` 通过 UIAutomation 读客户端窗口，跨会话一律看不到窗口，表现为 `WindowDiscoveryError`

7. 真机探针实测记录

2026-11-06 首次运行（Windows 11 / Python 3.13.17）

- 环境：`Windows-11-10.0.26200-SP0`，wxauto **3.9.11.17+dsh1**（`wxauto-fork/wxauto-main` 构建）
- 结果：`import wxauto` ✅；`attach client` ❌

```
WindowDiscoveryError: No visible Weixin/WeChat top-level window is accessible in Windows session 21. Discovered client window sessions: (none).
```

- 判定：**不是代码缺陷，是会话/窗口不可见问题。**探针进程在 session 21（典型 RDP/远程会话），UIAutomation 只能枚举本会话顶层窗口；若客户端在别的会话、或未启动、或只在托盘，均表现为 `(none)`。
- 因此新增 `poc/win_diag.py`：`python win_diag.py` 直接给出当前进程 session、微信窗口所在 session、是否管理员、以及下一步动作；`probe_wxauto.py --diag` 同效。attach 失败时会自动打印 next steps。
- **准入状态：仍为未通过**（Q1/Q2 均无证据）。下一次运行必须满足 §6 前置 4，并要求客户端版本与 wxauto 构建匹配（3.9.x 与 4.x 是两套 UI）。

2026-11-06 第二次运行（同机，会话问题已解决）

- 探针已能**看到并 attach 到窗口**：`Weixin.exe`，pid 46576，session 21，class `Qt51514QWindowIcon`，title 微信；

但 `RuntimeError: Attached to Weixin/WeChat HWND 0x1F0C2A, but the wxauto 3.9 accessibility tree was not found,` root children = `MMUIRenderSubWindowHW`（微信 4.x 的新 UI 容器）。

- **判定：客户端是微信 4.x，`wxauto`（3.9 系）无法驱动。**这是版本代际问题，不是会话问题（窗口这次可见）。
- 可行路径（二选一，成本/风险不同）：

路径	做法	代价	风险
4.x 路线（不动客户端）	`pip install wxauto4`（免费，Python 3.9–3.12，客户端 ≤ 4.1.8.107）或 `pip install wxautox4`（付费，Python ≤ 3.13）	需装 Python 3.12 或购买激活码；客户端可能需要降到 ≤4.1.8.107	免费版对客户端版本敏感
3.9 路线（不动工具链）	装微信 3.9.11.17 + 关闭自动更新，沿用现有 `wxauto` fork	需在客户端上做版本锁定	官方可能强推升级

- **Q1 未决风险（重要）**：wxauto4 文档的 `Message` 只有 `attr/type/id/sender/content/info`，**没有文档化的结构化 @ 列表**（`at` 只出现在发送侧 `quote(text, at=[...])`）。因此群 @ 是否可结构化判定，取决于运行时 `info` 里是否带 at 列表——必须由探针 dump 后决定。若确实没有，按 §6 前置规则**不得改用文本 @昵称 反解**，应转 L2（pad）或另选后端。
- 工具已升级：`poc/win_diag.py` 识别 `Weixin.exe/Qt51514QWindowIcon` 即判为 4.x 并给出包建议；`poc/probe_wxauto.py` 依次尝试 `import wxauto4` 再回退 `wxauto`，并把包名写进报告。

2026-11-07 探针 v3：先保证"一定出报告"

- 现象：Windows 上跑旧版 probe **没有生成 `probe-report.json`**。三个原因（均已修）：

1) 旧版顶部 `import win_diag` 是硬依赖，只拷了 `probe_wxauto.py` 就 `ModuleNotFoundError`，一行都不写； 2) 旧版只尝试 `wxauto4` / `wxauto`，**没有 `wxautox4`（Plus）**，装了商业版也判为"无包"； 3) `--out` 默认是相对 cwd 的文件名，报告可能落在别的目录。

- v3 起：`win_diag` 变可选；包按 `wxauto4 → wxautox4 → wxauto` 依次尝试；报告在 `finally` 里必写并打印**绝对路径**；新增 `--selftest`（不碰微信，只验证写文件链路）。
- 同时收紧 @ 探测：把 `attr`（消息来源 system/self/friend）从候选里剔除，并要求 `atListCandidates` 里的值真像列表——否则旧逻辑会把 `attr` 当成 @ 列表，**直接骗过 Q1**。
- 另据使用者反馈：**商业版授权后微信消息收发成功**。这说明客户端与 Plus 库在该机器上可用，但"产品能收消息"不等于"我们能拿到结构化 @ 列表"，Q1 仍需 `atListCandidates` 证据。

2026-11-07 群消息图片（Q4）：能力已确认存在，落地前先量三项指标

- 依据 v4.0 文档：消息类表里 ✨图片消息 `ImageMessage` 带 ✨（Plus 构建专属），该类提供 `download(dir_path, original)` 与 `ocr(timeout)`；群里的图片就是 `type='image'` 的普通消息，`sender` 为群成员， `chat_info()` 返回 `chat_type='group'`。因此「识别 → 下载 → OCR」链路成立，但**需要商业版**。

- 三个必须先量的约束：① 免费构建可能只给 `type='image'` 而无 `download()`；

② `GetAllMessage()` 只覆盖当前窗口已渲染的消息，更早的图要先 `roll_into_view()`，所以轮询间隔过大时图片可能已滚出视野； ③ 下载会占用客户端 UI，必须与切会话/发消息串行，用户同时操作时会变慢。

- 探针新增 `--media`：报告 `media.imageCount / hasDownload / download.bytes / download.ms / ocr`，

在无 `download()` 的构建上明确输出 `image messages need the Plus build`，不会假绿。

- 落地设计（待数据确认参数后实施）：`ChatEvent.media` + sidecar `GET /media/`（下载缓存 + TTL）+ `weixinKind.threadContext.image()` 取图作为模型附件（复用 `feishu-image.ts` 的既有模式）；出站发图暂不做。

2026-11-07 Q1 定论：L0（pc-uia）没有结构化 @；但「L0 不可用」这个结论是错的

*这一节保留了两层：先说清**哪一部分结论成立**，再说清**哪一部分是我判断错误**。后者更重要，因为按错误结论会把一个能用的后端关掉。*

成立的部分：没有结构化 @ 列表

环境：Windows 11（session 1，console，Active）/ Python 3.12.10 / `wxautox4` 41.2.3 / 微信 4.x（`C:\Program Files\Tencent\Weixin\Weixin.exe, Qt51514QWindowIcon`）/ 已登录账号 `。 探针：poc/win_diag.py → a WeChat window is visible in this session； poc/probe_wxauto.py --group "" --media → poc/probe-report.json`。

***隐私**：`probe-report.json` 与消息 dump 里是**真实聊天内容**。它们只用于本地取证，不提交、不进文档。本文引用的一律是**值的形状**（`、、、、`、`），不是具体是谁。*

结论：`atListCandidates` 为空 → 后端按 §4.1 应切 L2（pad）；群 @ 不得文本反解。

三条独立证据（任一条都足以否掉 L0 的结构化 @）：

63. **UI 树是扁平叶子。** 消息控件 `mmui::ChatTextItemView`（`AutomationId chat_message_list.qt_scrollarea_viewport.chat_bubble_item_view`）的 `childCount == 0`，`TextPattern` / `ValuePattern` / `LegacyIAccessiblePattern` / `SelectionPattern` **全部不存在。** @昵称 与正文是同一个 `Name` 字符串，客户端没有把 mention 渲染成独立元素。

64. **消息对象没有 at 列表。** `FriendTextMessage` 的全部字段是 `id / content / hash_text / hash / sender / chat_info / type / attr`（`attr` 是 `friend|self|system` 来源标记）。

65. **`FriendMessage.at` 是动作不是字段。** 它是 bound method，执行即「拍一拍」，返回 `WxResponse{"status":"成功"}`。把它当取值方法会**每轮询一次就拍群成员一次**——真实副作用。

*唯一带 @ 的结构线索是正文里的 `U+2005`（四分宽空格）分隔符，例如 `@\u2005` 妈。它是**文本**特征，可被复制粘贴伪造，因此只作为证据记录在报告的 `textSeparator`（`textDerived: true, usable: false`），**不构成判定依据**（§2.3 硬规则 4）。*

顺带查清的另外四个 L0 缺口（同样进契约，避免以后被误判为 bug）：

契约字段	L0 实际情况	后果
`room`（chatroomId）	拿不到。`ChatInfo()` 只有 `chat_type/chat_name/group_member_count`；`SessionElement` 只有 `name/time/content/isnew/new_count/ismute`	`rooms[].id` 只能用显示名，而 Node schema 的 `^[A-Za-z0-9_-]+$` 会拒绝中文群名 → **配置层根本表达不了**
`sender.id`	只有显示名。`sender_info()` 的 `id` 是微信号（如 `）不是 wxid_`，且**观测到滞后一条消息、有时返回 `null`**	Policy 的 wxid 名单在 L0 上不可执行
`at`（消息时间）	普通消息无时间字段，但 `TimeMessage` 分隔符带 `time`；按房间记住最近的分隔符即可分钟对齐，只有监听启动前的历史消息回落服务端接收时间	已按此实现，见 §3.2
`/members`	无群成员 API（`_api` 只有 `add_group_members`）	成员表退化为空

因此 L0 的准入门槛没有通过，而且不只是 @ 一项：即使只做单聊，身份与时序也达不到契约。 Node 侧严格的 `WXID_PATTERN` 在这里不是缺陷而是**准入闸门**——它挡住了不达标的部署。

我判断错的部分（2026-11-07 晚修正）

上面的分析**证据都对，结论错了。**错在两个地方：

错误一：我用 `GetAllMessage()` 轮询收消息。 它只返回当前窗口**已渲染**的消息， 而 `ChatWith()` 在目标已是当前会话时不会重绘，于是周期性读到 0 条。我把这个现象 当成了「L0 收不到群消息」。owner 指出本机有一个能跑的生产实现 （`D:\\\wxbot_core.py`），它用的是**监听模式**：

```
self.wx.AddListenChat(nickname=..., callback=self.message_handle_callback)
self.wx.StartListening()
```

真机复测（只用「文件传输助手」自测，不往群里发东西）确认：`AddListenChat` 返回会话名，`StartListening()` 之后 `SendMsg` 一条，回调**实时触发两次**（`TimeMessage` + `SelfTextMessage`）， `chat` 对象给出 `{'who': '文件传输助手', 'chat_type': 'friend'}`，`msg.chat_info` 给出 `{'chat_type': 'friend', 'chat_name': '文件传输助手'}`。**推送式，不依赖窗口渲染。**

错误二：我把「没有结构化列表」直接推成「不准入」。 生产实现用的是文本子串匹配：

```
self.config.AtMe = "@" + self.wx.nickname      # 2714 行
if self.config.AtMe in message.content: ...    # 3381 / 3405 / 3729 / 3748 行
```

它确实工作。但子串匹配连「他刚才说了 @ 这句话」都会误判。**更好的判据是客户端自己的标记**： 真的从 mention 列表里选人时，微信插入的是 `@` + **`U+2005`（四分宽空格）**； 人工手打用的是 `U+0020`。真机 dump 的两条证据：

```
"@<成员>\u2005 一会儿带两个豆浆回来，我们吃过了"
"@<成员>\u2005 妈"
```

所以「@本账号显示名 + U+2005」或「以 @本账号显示名 结尾」才是判据，另外 @所有人\u2005 也算对本账号说话。这不是文本反解——它要求客户端自己生成的标记， 手打不出来（但可以粘贴，见下）。

修正后的决定

capabilities.mentions 从布尔改成三态，把「证明有多强」写进契约而不是藏在代码里：

值	含义	谁
structured	客户端给的 mention 列表	fake（参考实现）、未来的 L2/L1
markup	客户端自己的 mention 标记 + 本账号显示名	**pc-uia（L0）**
none	完全判不出	pad 桩

markup 的代价必须写明，因为它影响「能不能把群 @ 当作授权」：

66. **判据名会变**：判据含本账号显示名，所以显示名带 **60 秒 TTL 重读**、/listen 时**强制重读**、/messages 轮询顺带刷新；改昵称**不用重启**，/health 与 /status 的 detail 里能直接看到当前判据名。读失败保留旧名并记 note（清空会让判定永远不命中，比旧值更糟）；

67. **可伪造**：分隔符能被粘贴，所以它比子串匹配强，但**不是不可伪造**；
68. 因此 §2.3 硬规则 4 的表述改为：**不得按 @昵称 文本反解；允许按客户端标记判定，且必须在能力自述里声明强度**。§8 的不变量与之一致。

身份问题用**命名空间**解决，而不是放宽 Node 侧的 id 正则：sidecar 把没有稳定 ID 的会话与 发送人命名为 wxname_，name 字段带显示名；Node 侧的 ^[A-Za-z0-9_-]+$ 保持不变，weixinKind.refreshLabels 把 id 解析回显示名供面板显示。 代价同样要写明：**显示名改了 id 就变**，配置需要跟着改。

L2 / L1 仍然是更强的路，但不再是「唯一能用的路」：

- **L2（pad）**：能同时给出 chatroomId、wxid、结构化 ats、真实时间。需要业务方提供 WeChatPadPro 一类的 iPad 协议服务。sidecar 的 --backend pad 已留桩： available:false + 数据端点 501，切换只需改配置 + 重启，不动 Node 代码。
- **L1（DB 读取器）**：本机已确认消息库位置与加密形态，是**不注入进程也能拿到全部结构化字段**的路线。证据与做法见下。它解决的是 structured 那一档，而不是「能不能用」。

阶梯可行性审计：L1 与 L2 对真实文档的核查（2026-11-07）

§2.2 的阶梯假设「L1（注入）能拿到结构化 @」「L2（pad）能解决一切」。对**真实文档**核过之后， 这两个假设都要修正——这一步是为了不让人照着阶梯去做无用功。

L1（wxhelper 一类注入插件）= 对本机不可用，而且即使可用也不解决 @。

- 它只适配 **微信 3.9.x**（WeChat.exe）：仓库 README 明列 3.8.0.41 / 3.8.1.26 / 3.9.0.28 /

3.9.2.23 / 3.9.2.26 / 3.9.5.81，doc/ 下只有 3.9.2.23 与 3.9.5.81 两份，jsDelivr 上最高 tag 是 3.9.10.19-v1，全仓库**没有** 4.x / Weixin.exe 的任何痕迹。本机是 4.x。

- 更关键：它 hook 出来的消息 JSON 是 **11 个字段**（pid/fromUser/toUser/content/signature/msgId/msgSequence/createTime/displayFullContent/type/base64Img），**没有 atuserlist / atList**，也没有任何返回结构化 @ 列表的端点。也就是说：换到 3.9 客户端、 接受注入风险之后，我们想要的那个字段**依然不存在。**
- 它默认监听 0.0.0.0:19088 且**无任何鉴权**（源码 StartHttpServer），等于把整个微信号暴露给局域网。

作者自己在 README 写「可能会造成封号等后果」。

L2（WeChatPadPro 一类 pad 服务）= 版本这关过了，@ 这关文档没给。

- 它不驱动本地客户端，是 iPad 协议端（iOS 18.61 / build 861），所以「本机装的是 4.x」与它无关。
- 公开 Cloud swagger（gwpad.com/Cloudswagger/swagger.json）可核对到 **401 个端点**：basePath=/v1，响应包 {Code，Data，Message，Success，request_id}， 鉴权头 X-Cloud-Access-Token。
- **但全文检索 atuserlist 命中 0**，at_list / AtList 全是 chat_list 的误命中；唯一与 @ 相关的字段是**发送侧**的 Msg.SendNewMsgParamDoc.At。入站 sync_message 事件**没有字段级 schema。** 也就是说：结构化 @ 列表**是否存在于入站事件里，公开文档无法确认。**
- 群 id 与发送者 wxid 有文档：Msg.QuoteContextDoc.chat_user_id（群聊为 room@chatroom）、from_user_id（群聊为成员 wxid）。所以 L2 在**身份**上确实能补齐 L0 的缺口。
- 闭源二进制 + 授权码计费，最新版本只在赞助群；仍在维护（2025-08-22 发布 v2.01）。

文档有坑：README 指向的本地 swagger 文件已不在仓库，旧文档站已 404，只有上面那个 swagger 可信。

结论：阶梯从「L1 优先、L2 次选」改为——**L1 划掉**（对 4.x 不可用，且不提供 @ 列表）； **L2 是唯一候选**，但它能否给出结构化 @ 只能由持证方实测确认。

给持证方的一次性验证（10 分钟，决定 L2 值不值得接）：用 pad 服务登录一个测试号，拉一个群、让群友 @ 该号一次，然后看 sync_message（或同义入站事件）里这条消息的原始 JSON：

- 若出现形如 atuserlist / at_list / msgsource 里带 wxid 的字段 → L2 成立，mentions 可升级为 structured；
- 若没有 → L2 只能补身份与时间，mentions 仍是 markup 档，那就**没有必要为 @ 换后端。**

L1 可行性证据（已确认，未实施）

```
%USERPROFILE%\Documents\xwechat_files\<account>_<hash>\db_storage\
  message\message_0.db   11,882,496 B   ← 群/单聊消息
  contact\contact.db     23,150,592 B   ← 联系人、群成员
  ...
```

`message_0.db` 前 64 字节是随机字节（`79 4b ad 98 12 22 ba 23 …`），**不是** `SQLite format 3` → 确认 SQLCipher 加密（微信 4.x 用 SQLCipher v4：4096 字节页、AES-256-CBC、页尾 HMAC-SHA512）。 文件在客户端运行时被独占打开，读取需 `FileShare.ReadWrite`。

一旦拿到密钥，协议层自带的结构化字段即可直接映射契约：会话 `username`（= chatroomId）、 发送者 `sender`（= wxid）、`create_time`（= 真实时间）、以及消息 proto 里的 at 列表 （= `ats`）。**这一条同时解决 Q1 与 Q2**，也是唯一不违反「禁止文本反解」的 L0 替代方案。 密钥需要从 `Weixin.exe` 进程内存取出（本机是管理员会话，具备条件），属于 L1 的 「注入/内存读取」风险档，需业务方书面接受后才能做。

L1 突击：已经试到哪里，为什么停（2026-11-07 实测，未拿到密钥）

思路：SQLCipher 接受**裸密钥**（`x''`），此时 KDF 被跳过，密钥字节同时用作加密密钥与 HMAC 密钥，于是**一次 HMAC-SHA512 就能验证一个候选**（本机实测约 31 µs/候选）。这让「扫」变得可行，不像 PBKDF2（256000 轮 ≈ 0.1 s/候选）只能靠已知偏移。

已经排除的：

69. **`login\\key_info.db` 是明文 SQLite**（`SQLite format 3`），只有一张表 `LoginKeyInfoTable(user_name_md5, key_md5, key_info_md5, key_info_data BLOB)`，5 行，`key_info_data` 每行 180 字节，与 `%APPDATA%\Tencent\xwechat\login\\key_info.dat` 同构。 → 说明这里存的是**被包裹**的密钥，不是明文。

70. **对该 blob 的穷举全部失败**：把 180 字节按 16/24/32/40/48 五种长度、每个偏移都当候选（27660 次裸密钥尝试，0.22 s），再用 `user_name_md5` 的 ASCII 与原始字节、blob 首/中/尾切片、 整个 blob 试 PBKDF2（64000 与 256000 轮 × SHA512/SHA256 两种页布局，810 次，143 s）——**全部不命中。** 跨 5 行 + `.dat` 也没有共享的 32 字节窗口。

71. **进程内存扫描：做了但不足以下结论。** `Weixin.exe` 有 6 个实例，180 秒预算内只扫到 46 MB（5,857,300 个候选，8 字节对齐），未命中。按 31 µs/候选折算，把这些进程按 stride 8 全扫完约需 30–60 分钟，本次没有跑完。

为什么停在这里：负结果的可信度取决于校验器本身，而我**没有一份已知可用的 SQLCipher 库**可以 校准它（页布局 4096/reserve 80 与 48、页号是否参与 HMAC、salt 是否跳过 16 字节，都是按文档推断的）。 在未校准的校验器上跑 40 分钟得到「没找到」，不能写成「不存在」。继续做的正确顺序是：

72. 先用 sqlcipher CLI 造一个已知密钥的库，把校验器对齐到它（这是 10 分钟的事，但必须先做）；

73. 再用 C/Rust 写扫描器（Python 的 31 µs/候选是下限，C 能到 1 µs 量级），stride 1 全扫；
74. 拿到密钥后才谈 DB 读取器：`message/message_*.db` 的 schema、群消息的 mention 字段、`contact.db` 的成员表——那是另一个独立工作量。

结论：**L1 仍然是最强的路，但它是一个需要先校准再实现的工程任务，不是一次会话能收口的。** 在此之前 `markup` 档是可用的，且它的三条代价已经降到两条（改名自愈、id 会换、可伪造）。

2026-11-07 群消息图片：已落地（commit cfa971e）

链路（每一段都有测试）：sidecar `/messages` 只为**能下载**的构建列 `media:[{id:'#0',kind:'image'}]` → `bridge.toChatEvent` 映射进 `ChatEvent.media` → `weixin-watch` 把 id 放进 `Trigger.meta.media` → `weixinThreadContext.read()` 在图片消息正文尾追加不可见 `[image:]` → `images(text)` 用同一正则提取 → `image()` 走 `GET /media/` → 魔数判定 → `attachments.saveImage` → 模型可见。

不可让步的约定：只按魔数（PNG/JPEG/WebP/GIF）判定，**不信**远端 Content-Type；超 `imageLimits.maxImageBytes` 或格式不接受时**在 saveImage 之前**抛错（不落盘、不进上下文）；`/media` 与 `/send` 共用 UI 串行锁； 无下载能力的构建 `/media` 返回 **501**，定位失败 **404**，绝不返回空字节。

遗留 Minor（评审 t54，均低危，记录在案）：

- IMG-1 仅按单图 `maxImageBytes`，未叠加 `maxMessageImageBytes`/每消息图片数上限
- IMG-2 bridge 先整块 `arrayBuffer` 再查限，未做流式上限
- IMG-3 sidecar `/media` 成功分支未显式拒零字节
- IMG-4 `imageIds()` 从正文解析，用户可自造 `[image:x]` 触发一次请求（sidecar 404 兜底；建议后续只接受 `meta.media` 里的 id）

ADR-006 健康条目的键兼容：默认实例保留裸名

- **状态**：Accepted（2026-11-06，P0 实施中确认）
- **决策范围**：`SourceRegistry.healthEntries` / `sourceContextOf` / `views/sources.ts` 里健康条目的 key 形态
- **关联**：设计文档 §6.6、§4.3；P0 落地工单 §1.5；实施任务 t3

1. 背景

设计文档 §6.6/§4.3 要求"健康条目一律按 `instance:entry` 命名"：一个 kind 可有多个实例，只有把实例 ID 拼进 key，同一 kind 的两个实例才能各报各的健康、互不冲突。P0 工单 §1.5 与 §4 风险表 R2 的原始方案也写成"key 一律 `instance:entry`"。

但 P0 的硬约束是"行为零变化"：旧数据库里的设置、旧面板、既有运维脚本都按裸名 `github` / `feishu` / `history` 读取健康条目。若一律改成 `github:github`，旧面板升级瞬间会读不到健康，回滚旧版本也会读不到新形态的键。

2. 决定

- **默认实例（`id === kind`，如 `github`、`feishu`）沿用注册时 claim 的裸名条目**：`github`、`feishu`、`history`。
- **具名实例（`id !== kind`，如 `github-2`）惰性 claim `${instance}:${entry}`**：`github-2:github`。
- 理由：

75. 旧数据库与旧面板读取零变化，升级无需迁移任何持久化数据；
76. 回滚安全——旧版本仍能按裸名读到默认实例的健康；
77. 具名实例是新增概念，没有历史包袱，直接采用规范形态。

3. 后果

- **键形态不对称**：同一 kind 的默认实例是 `github`，第二实例是 `github-2:github`。所有新增读取方都必须知道这条规则。
- **views 读取必须"先看裸名再看前缀"**：`views/sources.ts` 的 `entryOf` 用 `instance.id === instance.kind ? name : healthEntry(instance.id, name)`；`sourceContextOf` 把 `kind.id ?? kind.kind` 传给 `healthEntries`，由 `healthEntries` 在 `instance === kind` 时走注册期裸名条目、否则惰性 claim 前缀条目。
- **不要"顺手统一"**：把默认实例也改成 `github:github` 会立刻破坏旧面板与回滚兼容，属于需要单独评审的迁移。
- **未来若要统一的迁移步骤**：

78. 新代码先双写双读：同时维护裸名与 `${instance}:${entry}` 两份键；

79. 面板与视图先切到前缀形态，裸名保留一个版本作为回退；
80. 健康是运行期内存态、本身不落库，故持久层通常无需变更；若有监控/脚本另存快照，需同步改读取路径；
81. 观察一个发布周期无异常后，再删除裸名分支。

- **风险**：双写期间两份键可能短暂不一致（旧代码只写裸名），必须在同一次上报内一起更新；外部消费者若直接读裸名，迁移时会静默丢失告警。

4. 验收（测试覆盖的两侧）

- `tests/kv-instances.spec.ts`：覆盖 **key 生成侧**——`healthEntry('wecom-hr','wecom') === 'wecom-hr:wecom'`、与 `wecom-ops` 的不同，及新 KV 家族按实例分键。
- `tests/registry-instances.spec.ts`：覆盖 **claim/上报侧**——两个具名实例（`wecom-ops`/`wecom-hr`）各自 claim `${id}:wecom` 且互不覆盖，`report` 仍只允许 kind 声明的条目；旧 `github` 设置按裸名解析为默认实例（`id === kind` 侧由该文件第 2 条用例守住）。

ADR-007 微信渠道能力对齐：SiverWXBot 的功能面 vs 本仓库

- **状态**：Accepted，三期均已实施（2026-11-08）：P1 `7b4e316`、P2 `c4350a8`、P3 本提交
- **决策范围**：微信来源（`src/sources/chat/weixin/`）的面板字段、sidecar 出站能力、按关键词分流的归属
- **关联**：ADR-005（L0 后端与 @ 判据）、ADR-006（健康条目键）、`sidecar/wxauto-bridge/contract.md`

1. 结论摘要

82. **私聊**：监听、文本、图片都已可用（真机验证）。面板上"添加联系人"就是把他从会话下拉里选中——桥只会把**真见过的会话**列出来，没有手填 wxid 的入口（显示名派生的 `wxname_…` 不是真 wxid）。
83. **群聊**：监听、仅 @ 回复、群图片都可用；@ 是 `markup` 档（客户端标记），不是结构化提及。
84. **两处库限制**，不是配置问题：
    - **回复时引用消息：做不了。** `wxautox4` 的 `SendMsg(msg: str, who, clear, at, exact)` 与 `Chat` 的全部公开成员里**没有任何 quote/ref 参数**（`Chat` 成员：`SendMsg SendFiles SendAudio AtAll GetNewMessage …`）。微信 4.x 的"引用"只能靠右键菜单的 UI 交互，脆弱且未暴露。替代：把引用写进正文前缀（「引用 @张三」原文…）。
    - **回复时 @ 发言人：能做。** `SendMsg(..., at: Union[str, List[str]])` 支持，且 `at` 收显示名；本仓库之前**故意**没接（`send()` 的 docstring 写明"绝不放 `at=`"），因为入站的 `FriendMessage.at` 是"拍一拍"动作，曾被误用成群内 @。出站 `at=` 是另一条路径，接它需要单独的真机验证。
85. **语音识别**：入站已实现（`to_text`，8 秒上限、只试一次、失败降级为 （语音转写失败）），但**还没在真实语音消息上验证过**；它每次都会占用客户端 UI 最多 8 秒，所以需要一个开关而不该无脑常开。
86. **关键词**：本仓库完全没有，属于新能力，不是勾选项。可分两层做，见 §5。

2. 逐项对齐

SiverWXBot 的能力	本仓库现状	面板可配置?	结论
启用群聊监听	已有（`rooms` 集合，`/listen` 集合替换、幂等）	✅ 已有	已对齐
仅 @ 时回复	已有（`atOnly`，注释判据 `markup`）	✅ 已有	已对齐；仅对群生效，私聊没有 @ 概念
私聊监听 / 私聊回复	已有（`room: ""` 单聊流，id → 流有映射）	✅ 已有（从会话下拉选人）	已对齐；缺"私聊"这一处的措辞与提示
群组图片	已有（入站 `media` + `/media` 下载，真机验过 PNG 魔数）	🔶 无开关	需加 `images` 开关（关掉只留 `[图片]` 占位）

SiverWXBot 的能力	本仓库现状	面板可配置?	结论
私聊图片	同一条链路，真机验过（69 字节 PNG）	🔶 无开关	同上
语音识别	已实现，**未真机验证**；每次占用 UI ≤8s	🔶 无开关	需加 `voice` 开关，默认开，并在真机语音消息上验收
回复时 @ 发言人	**未实现**（sidecar 明确不传 `at`）	❌	可做：`mentionBack` 开关 + sidecar `at=[发送人显示名]`；仅群聊、且需真机验证
回复时引用消息	**未实现**	❌	**库不支持**：无 quote 参数。替代方案 `quoteStyle: 'text'`，把引用写进正文
群聊关键词回复	未实现	❌	可做，见 §5
私聊关键词回复	未实现	❌	可做，见 §5
出站图片/文件	未实现（契约只发文本）	❌	可做（`SendFiles(filepath, who)` 存在），但需要核心 `OutboundMessage` 支持附件，属于另一个量级
面板手动添加联系人	只能从桥见过的会话里挑	✅ 已有（候选下拉）	没有真 wxid 可取，手填无从校验，保持"只挑候选"更安全

3. 技术证据（本次真机核对）

```
WeChat.SendMsg(self, msg: str, who: str = None, clear: bool = True,
               at: Union[str, List[str]] = None, exact: bool = False,
               max_retries: int = 3) -> WxResponse
Chat.SendMsg(self, msg: str, who: str = None, clear: bool = True,
             at: Union[str, List[str]] = None, exact: bool = False) -> WxResponse
Chat 公开成员: AddGroupMembers AtAll CancelCall ChatInfo Close EditFriendInfo
               GetAllMessage GetDialog GetNewMessage LoadMoreCache SendAudio
               SendFavorite SendFiles SendMsg SetGroupAnnouncement
               SetGroupMyNickname SetGroupName SetGroupRemark Show VideoCall
               VoiceCall chat_type
```

- 私聊文本：`/messages?room=` 返回 `room: ""` 的事件（已修）。
- 私聊图片：事件带 `media: [{"id":"#0","kind":"image"}]`，`/media` 返回 69 字节、`89 50 4e 47` = 合法 PNG。
- `/listen` 重复推送已在 0.02s 内返回且不触界面（修掉了通讯录窗口反复弹出）。

4. 面板字段设计（拟新增）

key	类型	默认	作用
`rooms`	rows（已有）	`[]`	**改标签**为「群与私聊」；选群 = 群聊，选人 = 私聊
`atOnly`	switch（已有）	`true`	仅 @ 时回复（群）
`mentionBack`	switch	`true`	回复群里 @ 我的人时带上 @（**仅群**）

key	类型	默认	作用
quoteStyle	select none/text	text	无真引用接口；text = 正文前缀注明引用对象
images	switch	true	是否把图片作为附件交给模型
voice	switch	true	是否转写语音（关掉用 [语音消息] 占位，省 UI）
keywords	rows [{ any, note }]	[]	命中任一关键词即视为**明确请求**（走既有准入模型）
autoReplies	rows [{ any, reply, stopAgent }]	[]	命中即**直接回固定文案**（不经 agent），可选是否同时抑制交给 agent

5. 分期

期	内容	影响面	估计
P1	面板措辞与提示（群与私聊、姓名 自动填的说明）；images / voice 两个开关接到入站渲染	kind.ts、sidecar _render_text/_to_event 之前的分支	0.5 天
P2	mentionBack：sidecar send(..., at=[名]) + 契约字段 + Node 侧按开关传发送人 + **真机群内验证**	sidecar、bridge.ts、contract.md	1 天
P3	keywords → 准入（命中即 explicit）；autoReplies → 由监听循环直接 /send	kind.ts 的 scope/explicit、weixin-watch.ts	1.5 天
已实施	P1：措辞 + voice/images 开关	kind.ts、sidecar options、verify/options-check.py	完成
已实施	P2：mentionBack（sidecar at、契约、面板、verify/mention-check.py）	sidecar、bridge.ts、kind.ts	完成（真机群内未跑，避免在真实群里发言）
已实施	P3：keywords + autoReplies	scope()、weixin-watch.ts、ports.ts	完成
已实施	P4：即时回执 + 处理中提醒（acknowledge/ackText/progressAfterSeconds）	weixin-watch.ts 的 pollChat、kind.ts	完成
已实施	P5：监听自愈（10 分钟无事件 → 先摘后挂，每 15 分钟最多一次；/health 报「最近事件 / 已重挂次数」）	server.py 的 _reattach_if_quiet、_start_listener_watchdog；verify/listener-check.py	完成
已实施	P6：游标跨重启单调（序号时钟播种 + 响应带 head）；客户端游标超前时从头读	server.py、bridge.ts、weixin-watch.ts	完成

期	内容	影响面	估计
—	`quoteStyle: 'text'`	现状已是「引用写进正文」，没有独立开关	不需要
—	出站图片/文件	需要核心 `OutboundMessage` 附件模型	另立 ADR

5.1 P4 的依据（实测时间线，来自 `state/superagent/orchestrator.sqlite`）

消息	系统动作	时延
07:14 一条「test」	07:14:53 判定 ignore	53 秒，无回执（正确：没有请求）
07:14 「用网页做一个…」	07:16:09 判定 accept，同时发出回执	**2 分 09 秒**
07:30 「发布到公网…」	07:32:00 并入已有工单，**没有任何回执**	2 分钟全程无声

结论：等待感来自两处——**评估本身就是几十秒到两分钟**，以及**追加到已有工单的消息不带回执。**所以回执由源侧在评估前发出，并对「机器人还没在这个会话里开口」补处理中提醒。缩短评估本身不在本期范围（涉及模型选择与提示词规模）。

6. 风险

87. **`at=` 的语义陷阱**：入站 `Message.at` 是"拍一拍"，出站 `SendMsg(at=…)` 是提及——两者同名不同义。P2 必须用专门的用例证明"发送后群里出现真正的 @ 提醒，且没有触发拍一拍"。
88. **语音转写占用 UI**：`to_text` 会抢客户端界面最多 8 秒，期间监听回调可能延迟。默认开但必须可关，并在 `detail` 里暴露"正在转写"。
89. **关键词自动回复绕开准入模型**：`autoReplies` 由监听循环直接发送，意味着任何人都能用一句配置好的话触发一次发送。必须限制为该来源已配置的会话，并受 sidecar 的 ≥1s 发送间隔约束。
90. **引用只能以文本呈现**：如果使用者期待微信原生的"引用"样式，这里做不到，需要在面板提示里写明。

7. 2026-11-10 收口：接收按群、名单不带来源、同步按后端能力

真机上把微信这条路走通后，三处设计按第一性原理收敛：

- **来源管感知，评估管授权。** 来源决定"看不看得见"：收哪些群、收不收私聊、每个群交多少（只交 @ 机器人的消息，或全部消息）。

评估决定"算不算工作"：谁能发起、在哪个范围、按什么强度接。所以群列表、私聊开关、每个群的接收方式在来源；名单与准入表在评估。

- **「仅接收 @」下沉到每个群。** 全局一个开关会和准入表的「名单内·未@」表达同一件事（开关一开，准入表那两格永远是「—」）。

改为群行上的「接收：仅 @ / 全部消息」，与飞书的群列表一致；旧的 `atOnly` 降级为「新群的默认接收」，只在群没设时生效， 因此旧配置行为不变。

- **名单不带「来源」列。** 群消息还是私聊消息是消息本身的属性（有没有 `room`），准入表据此自动分行走；名单只放授权，

与飞书一致地只有「用户 ID + 名字」。同步用的归属标记（哪几行是同步写进来的）转为内部数据，不占界面。

- **同步按后端真实能力工作。** wxautox4 的 `capabilities.members` 是 `observed`：它没有群成员表接口（`GetAllRecentGroups()` 只给群名与成员数），`/members` 只能返回**在这个群里发过言的人**。实测：`GET /health` → `{"mentions":"markup","members":"observed","media":true}`；`GET /members?room=AI研究群` → `{"members":[]}`。 因此同步在 `observed` 后端是**并集**（只加不删，安静成员不会被误删），只有返回完整名册（`full`）时才替换；拿到 0 人时 明确报错并说明原因，`none` 直接拒绝。

- **两层 schema 必须一致地描述同一个形状。** 来源自己的 schema 生成表单，设置层的 `AllowedSchema` 才真正校验保存；两者对不上，

面板给得出的行就存不进去（本次真实故障：`via` 被设置层判为未知字段，导致整行无效、名单恒为空）。现由测试把每个来源的 part 与带标记的名单行都过一遍设置层 schema 兜住。

证据面：`tests/admission-tables.spec.ts`（准入表形状与跨 schema）、`tests/weixin-kind.spec.ts`（接收按群、合并语义）。

ADR-008 微信出站附件：桥已经能发，模型还不能用它发

- **状态**：Accepted 并按方案 A 实施（2026-11-08；桥层真机验证，核心层见 §6）
- **决策范围**：`OutboundMessage` 是否引入附件、由谁决定发什么文件、以及目录/大小护栏
- **关联**：ADR-007（微信能力对齐，其「出站图片/文件」一项指向本文）

1. 现状（已实现，已验）

- **客户端库**：`wxautox4` 的 `Chat.SendFiles(filepath, who=None, exact=False)` 与 `WeChat.SendFiles(filepath, who=None, ...)`

都在，不需要逆向；同类还有 `SendAudio`、`SendFavorite`。

- **桥**：`POST /send` 接受可选的 `file: {name, data(base64)}`，写临时文件后 `SendFiles`，随后删除；文字与文件可同时给（先文字后文件）。

护栏：解码后 ≤ 20 MiB（413）、请求体 ≤ ~27 MiB（`MAX_SEND_BODY_BYTES`）、文件名只留 basename 且过滤到 `[A-Za-z0-9._-]`。

- **真机**：一张 2×2 PNG 经 `/send` 发出，客户端渲染为图片消息，队列回显 `text="图片"`。
- **测试**：`verify/file-check.py` 11 项（文件名逃逸、上限、只发文件、文字+文件）。

2. 缺口：模型不能用它

`OutboundMessage` 只有 `id / channel / text / kind / ticketId`，`Channel.send(host, message)` 也只读 `message.text`。 所以「让 agent 回一张图/一个文件」不是桥的问题，而是**核心出站模型没有附件**：

91. `OutboundMessage` 需要可选的附件（名字 + 字节或附件引用）；
92. `engine.reply(...)` 与 `reply` 工具需要一个参数（或一个独立工具 `send_file`）带上它；
93. 各通道声明自己是否支持附件（`capabilities`），不支持的要么拒绝要么降级为文字说明；
94. 微信通道把它转成 `BridgeSend.file`；飞书/钉钉/企微等按各自 API 另做。

估计：核心 + 微信 + 测试与文档约 **1.5～2 个工作日**；其余通道各 0.5 天。

3. 两条路，以及为什么不建议捷径

方案	做法	问题
A（建议）核心附件模型	上面 1–4，附件是一个受控对象，通道各自实现	改动面大、要过一遍所有通道；但语义一致、可审计
B 路径直传捷径	给 agent 一个工具，直接把「本机路径」交给 sidecar 的 `SendFiles`	**等于给模型一个把任意本机文件发出去的口子**（凭据、源码、会议记录）；且路径语义只在 sidecar 所在机器成立

4. 若走 A，必须同时定的护栏

95. **来源限制**：只允许发工单工作区内的文件（解析后必须落在该 ticket 的 worktree 里），符号链接按解析后的真实路径判断。
96. **大小与类型**：沿用 20 MiB；对可执行后缀（`.exe/.bat/.ps1/...`）默认拒绝，需要显式开关。
97. **次数与频率**：沿用 sidecar 的 ≥1 秒发送间隔；每个工单设上限（如 20 个附件），防刷屏。
98. **默认关闭**：面板开关 `sendFiles`，默认关；开启时提示「模型可以把你工作区里的文件发到这个会话」。
99. **可审计**：每次发送记一条 `outbound file` 事件（名字、大小、sha256 前 8 位），进 `/health`/工单日志，便于事后追。

6. 方案 A 已实施的内容

- `OutboundFile { name, data(base64) }`；`OutboundMessage.files`；base64 而不是字节，因为回复先落 outbox 再发送，

字节数组过不了那条 JSON 记录。

- `ReplyRequest` 的 `answer/result/attention` 可带 `files`，`Replies` 原样交给 outbox。
- `SourceChannel.capabilities.files = { maxBytes }`：通道自己声明能不能带文件。
- 微信通道：`sendFiles` 开关打开时把第一个文件挂在这条回复上，其余按序另发（`/send` 每次一个文件）；关着就只发文字。
- **位置：`orchestrator_reply` 的 `files`**（主 Session 回复人的那条），不是 worker 的独立工具。

真机第一版把工具放在 worker 侧、`purpose` 固定为 `result`，结果 worker 在**进行中**就想发，两次都被「Ticket is not ready for delivery」拒掉——那条规则本来就是拦「把进展当结果发出去」的。回复人是主 Session 的事， 文件跟着那条回复走；worker 侧的工具已删除，避免再把人引到同一个坑里。

- 闸门集中在 `resolveTicketFile`（有专门的测试）：真实路径必须在工单目录内（符号链接也算），普通文件、非空、

≤20 MiB、拒绝可执行后缀。

- `Runtime.canSendFiles(ticket)`：读**面板开关**（默认关）与通道能力，工具在动手前先问它。

6.1 第二版：文件从「报告」来，不靠模型临场记得

第一版把工具放在 worker 侧且固定 `result`，被「未 ready」拒；第二版挪到主 Session 的 `orchestrator_reply`，但真机上 worker 仍在调**已删除**的旧工具名，得到 `UNKNOWN_TOOL`，于是它对用户说「这个环境没有附件上传通道」——能力在，工具不在。

所以现在的链路是：**worker 在 ready 报告的 `files` 里声明产出物**（相对工单目录）→ 报告存进 assignment → 转给主 Session 的 relay 带上这份清单 → 结果回复**默认自动附上**（也可以显式给

files）。sendFiles 开关与 resolveTicketFile 的 边界照旧。这样即使模型忘了说，产出物也会跟着结果走；模型也没有「看起来能做但一定失败」的工具可调用。

7. 真机教训（第一版为什么不对）

现象	原因
worker 调 orchestrator_send_file 两次都失败，execute.failed 写着 Ticket is not ready for delivery	工具把 purpose 固定成 result，而 result 需要先有 verified ready report；worker 想的是「边做边发」
微信里只看到「已打包发到本对话（附件 ...zip）」，没有附件	工具没成功，文本是模型自己说的

结论：附件属于**回复**，而回复由主 Session 写进 orchestrator_reply；放在那里既符合准入规则，也让「说到」和 「发到」由同一次调用决定。

5. 结论

- **能做**：库有 SendFiles，桥已经实现并真机验过；剩下的是核心出站模型与一个受控工具。
- **不建议**先用路径直传把功能糊上去：那会把「模型决定发什么」变成「模型能读任何本机文件并发出」，风险与一个受控附件模型完全不是一个量级。
- 下一步：确认是否走 A；确认后我按 §4 的护栏实现，并把「文件只能来自 ticket 工作区」做成测试（越界必须失败）。

ADR-009：Projex 反馈、附件、成员同步与检索——我们能走通的那些路由

状态：已接受（2026-11-13）· 范围：`src/sources/work/**`、`src/infra/yunxiao-api.ts`、`src/app/store-read.ts`

背景

Projex 渠道已经能接收工作项并对其评论，但一次协作的三个阶段各自缺了一块：一次评论的评估只能看到那条评论本身；worker 不知道自己所在的是哪个工作项；看板从来没有动过。把这三块补齐时冒出四个问题，它们的答案无法从代码里直接看出来，因为官方 OpenAPI 把一些路由放在了客户端自身安全规则够不到的地方。本 ADR 记录我们调用什么、不调用什么，以及为什么。

决定

1. 状态按名称配置，并对该实例已读取过的状态解析

云效公布了一份工作流状态列表，但可达的形态是 `GET /oapi/v1/projex/projects/{projectId}/workitemTypes/{id}/workflows`——**仅 Region 版**：没有组织段（客户端会拒绝任何不含 `/oapi/v1/organizations/{白名单}/` 的路径），而且它需要一个适配器并不持有的 `workitemTypeId`。旧的 `GET /organization/{org}/workitems/workflow/listWorkflowStatus` 早于 `/oapi/v1/`，而网关已经下线了其它旧路径（`GET /listWorkitems` 返回 404），所以它不是一条可以依托的路由。

每个工作项**确实**都携带的是它当前的状态：`{id, name}`。因此一次轮询会记录它读到的状态名与 id（`KV.work.statuses`，上限 200），两个回写字段都接受名称或 id。一个不透明的 ASCII 记号会作为「我们没见过的 id」透传；一个我们没见过的人类名称会被拒绝，并给出获取它的位置（来源卡片上的 **Observed statuses** 动作，它列出上一次读取返回的内容）。这是用完整性换可达性：词表就是该项目近期工作项所展现的内容，而那恰好就是运维者会从中挑选的那个集合。

2. 附件先上传到工作项，再嵌入评论

`POST /oapi/v1/projex/organizations/{org}/workitems/{id}/attachments`（multipart `file`）可达且有文档。它返回 `embedMarkdown` / `embedUrl` / `url`；评论正文是工作渠道唯一拥有的写入面，所以上传返回的 Markdown 嵌入（图片）或 Markdown 链接（其它文件）会被追加到评论文本中。

三道护栏，因为这是一个让文件离开本机的渠道：worker 的 `orchestrator_report` 已经把每个路径解析到工单目录内，并拒绝可执行文件与超过 20 MiB 的文件；来源的 `sendFiles` 开关（默认**关**）在渠道里会被再读一次；渠道自己的 10 MiB 上限在 provider 里强制执行，超限的文件会在评论里被点名说明，

而不是让这次投递失败。Projex 的真实大小上限未知，所以宿主侧的拒绝仍是最终裁定——那种失败会像任何其它回复失败一样，从 outbox 浮出来。

3. 成员名单从项目同步，而不是从组织同步

`GET /oapi/v1/platform/organizations/{org}/members` 可达，但整个组织对「谁可以在本项目上发起工作」来说是错误的范围。新式的 `GET /oapi/v1/projex/organizations/{org}/projects/{id}/members` 返回带 `userId` 与 `userName` 的项目成员，这才是正确的集合与正确的 id 形态：策略用 `allowed[].id` 与工作项的 `assignedTo` 比较，而在中心版里那就是成员的 `userId`。这个假设是被记录下来的，而不是被证明的（这里没有可用的在线 token）；一旦不匹配，会表现为每个工作项都被判为不在名单内，而策略卡片已经就此给出警告。

同步是共享 work 工厂上的一个 `policy.sync`，由新的 `syncMembers` 开关控制（默认**关**），因为同步会**替换**一份手工维护的名单。与飞书的同步一样，它只作用于默认实例：`Settings.syncable()` 解析 `kind(instance.id)`，而这只找得到 id 等于其实例 id 的 kind。

4. 检索解析的是流水号，不是线程

面板检索触发与线程；而人们输入的是工作项的人类编号（`ESFX-6228`）。`projexKind.find` 把裸流水号或一个 `…/workitem/…` 地址转成检索词，`Store.read.search` 精确匹配 `meta.serial`，与它此前精确匹配飞书 `meta.messageId` 的方式相同。工作项页面按流水号寻址，而线程以内部工作项 id 为键，所以这个解析能找出该工作项的触发与工单，但不会凭一个粘贴的 URL 绑定线程；绑定仍然走轮询写入的那个会话 id。

后果

- 有两件事仍未对真实组织验证：状态回写（从名称解析 id 已被证明，PUT 没有）以及附件嵌入在 RICH TEXT 评论里的版面。两者都在 `EXTENSIONS.md` 里标记为缺口；适配器的形状来自已公布的请求语法。
- 状态词表只随工作项被读取而增长。一个项目的工作项从未使用过的状态，在有一个工作项使用它之前无法按名称指定；直接输入它的 id 仍然可行。
- `sendFiles` 与 `syncMembers` 默认关闭，所以升级在运维者主动要求之前不改变任何行为。

ADR-010：云效登录、真实状态词表、RICHTEXT 附件，以及独立的插件身份

状态：已接受（2026-11-14）· 范围：`src/infra/yunxiao-oauth.ts`、`src/infra/yunxiao-api.ts`、`src/sources/work/`、**`cordis.patch.yml`**、**`src/client/`**、`package.json`、`locale/**`

背景

工作项反馈那项工作完成后仍剩四个缺口：运维者仍要手工粘贴个人访问令牌；状态名来自近期工作项恰好展现的内容，而不是来自项目自身的工作流；附件被以 Markdown 形式粘进一个纯文本评论正文，而富文本读取方不会渲染它；并且这个 bundle 沿用了官方 superagent 的内部 id，导致两者无法装进同一个 profile 而不共享面板槽位、预设 id 或状态锁。

决定

1. 通过云效的 OAuth 2.0 服务器登录，并把令牌文件保留为凭据

云效记录了一个 OAuth 2.0 授权服务器：授权码、强制 PKCE S256、仅公共客户端（`token_endpoint_auth_method: none`）、动态客户端注册、元数据发现、不透明令牌（`oat-` 24 小时，`ort-` 90 天）。这恰好就是一个桌面插件的形态，于是 `yunxiao-oauth.ts` 实现了它：发现端点、 用 loopback 回调注册一个客户端、打开浏览器、在 `127.0.0.1:/callback` 上取授权码，然后 交换令牌。

结果写入**同一个仅所有者可读的令牌文件**，那是每个读取方已经在用的文件；刷新令牌、client id、token endpoint 与过期时间放进同级的 `.oauth.json`。这样两种凭据都只有一份契约：个人访问令牌 仍然可用（没有同级文件，没有续期），而 OAuth 路径增加了续期。客户端在收到 401 后刷新一次并重试， 所以 24 小时的访问令牌不需要每天去面板一趟；浏览器只在每 90 天需要一次。

未选择的方案：DSH 的 `ctx.credentials` 记录。它们会把令牌绑到一个 profile 上，并让 headless/CLI 路径 改用另一套机制；而这个插件本来就是以文件方式运维的，文件方式还能让令牌不进配置、不进日志、不进面板。 令牌从不经过面板：浏览器与 loopback 监听器直接对话。

2. 状态词表是项目的工作流，而不是近期工作项的状态

`GetWorkitemWorkflow` 有两种形态。Region 形态（`/oapi/v1/projex/projects/{id}/workitemTypes/{id}/workflows`）没有 组织段，而客户端的路径限制按设计会拒绝它。**中心**形态则有：`/oapi/v1/projex/organizations/{org}/projects/{id}/workitemTypes/{typeId}/workflows`，而 `ListWorkitemTypes` （`?category=Req|Bug|Task`）提供 `typeId`。于是 `listStatuses` 按类别读取已启用的类型以及每个类型的 状态，来源卡片上的动作把它们存下来。

此前的「observed statuses」（在轮询到的工作项上见到的状态）保留为回退：一个从未被任何工作项使用过的状态 在工作流里但未被观察到，而一个类型读不出来的项目仍能解析它已经读到的内容。两处都不在的名称会被 拒绝，并给出填充词表的那个动作。

3. 附件以 RICHTEXT 形式评论，使用宿主自己的嵌入

`CreateWorkitemComment` 记录了 `content` 加 `formatType`（`RICHTEXT` 或 `MARKDOWN`，默认 `RICHTEXT`），而 `CreateWorkitemAttachment` 返回 `embedHtml`、`embedMarkdown`、`embedUrl` 与 `url`。把 `embedMarkdown` 粘进一个 纯文本正文正是那个 bug：富文本读取方从不渲染它。

带附件的评论现在是 `formatType: 'RICHTEXT'`：文本被转义进 ` 块（单个换行变成 ），每次上传的 embedHtml（图片是 `，其它是链接）紧随其后，幂等标记以 HTML 注释的形式随行。**不带**附件的评论保留它 一直以来的纯文本，所以 agent 写的 Markdown 保持原样。回读时，若存储内容是 HTML，`noteBody` 会剥掉 HTML， 于是模型看到的是文本。标记在任何剥离之前由原始载荷检测，所以一次重试仍然只发一次。

4. 插件获得自己的身份，于是两个 superagent 都能装上

Bundle 条目 id（`dsh-superagent-multi-channel`、`…-`）、预设 id（`…-worker`、`…-router`）、面板槽位与 locale 命名空间现在都是本 bundle 独有的；`locale/*.json` 描述它（"Superagent multi-channel"），并且 `icon.svg` 被导出。两个 bundle 插入相同的行 id、或定义相同的预设 id、或注册相同的面板槽位，就会冲突； 有了唯一的 id 它们就能共存，每个实例只需自己的 `stateDir`（一个目录只接纳一个实例，锁会说明这一点）。

有意保留的部分，因为它们位于**本插件自己的状态之内**，对另一个 bundle 不可见：投递消息上的 `dsh-superagent:` 消息来源 kind、`dsh-superagent-` 技能名前缀，以及 `scripts/dsh-superagent` 服务脚本。重命名它们会破坏既有会话的恢复与既有技能，而换不来任何共存收益。

升级既有安装：把 profile 的 `dsh-superagent` 块重命名为 `dsh-superagent-multi-channel` 并保留其 `stateDir`；SQLite 数据库、工单、游标与设置全都在那个目录里，所以不需要任何数据迁移。安装路径（插件管理器、 社区市场、自定义索引、tarball、本地目录）见 `docs/006-deployment.md`。

后果

- 令牌文件现在多了一个可选的伴随文件；只复制了令牌文件的运维者仍可工作，直到访问令牌过期，此后的 401

会要求重新登录，而不是续期。

- 状态动作按需对每个类别下的每个类型发一次请求；类型很多的项目会花掉几次调用，但绝不会花掉一次轮询。

- 附件的好坏取决于宿主的 `embedHtml`；返回空值的宿主回退为一个链接，那在评论里可见，但不是宿主自己的预览。
- 测试覆盖全部四项：OAuth 流程（含伪造 state 的拒绝与一次真实的 loopback 回调）、状态列表、

RICHTEXT 正文及其标记检测，以及 bundle patch、预设、客户端注册与文档中重命名后的 id。发现与动态客户端注册 也在生产中心站上演练过（`https://openapi-rdc.aliyuncs.com` → 一个 `account-devops.aliyun.com` 授权端点、 一个真实的 `client_id`）； 授权同意步骤与令牌交换需要一个人，这正是 `verify/oauth-check.py` 与面板动作所提供的。

ADR-011：让微信与飞书对齐

状态：已接受（2026-11-14）· 范

围：`src/sources/chat/weixin/、**src/app/resources.ts、src/app/prompts.ts、 src/app/skill-files.ts、skills/dsh-superagent-weixin/**`

背景

飞书渠道让 agent 在干活的同时读取原始会话：`feishu//message、/root、 /parent、/thread` 与 `/around`，由随附的 `dsh-superagent-feishu` 技能描述；它的评估读取 `thread/root/parent`；面板能按消息 ID 检索；一次回复的父消息会被映射回它所回应的触发。微信已经能 接收、结构化地判定 `@`、发文本与文件、转写语音、同步成员，并重挂它的监听器，但 agent 无法读取微信会话， 面板无法按 ID 找到消息，回复链也没有被连起来。

决定

1. 同一套资源语法，在 sidecar 之上实现

`weixin//message、/thread` 与 `/around` 镜像飞书的名称与响应形状（`{ status, message }、 { status, messages, next_resource }`），于是 worker 以同样的方式读取任一渠道，技能只在后端能力上有所差别。 `/thread` 用桥的游标分页；`/around` 是包含该触发消息的那一页（微信不提供时间窗查询，所以用这一页 顶替飞书的前五/后五窗口）。读取按触发划定范围：`readWeixin` 会拒绝 trigger id 不匹配的资源，这与飞书读取方 用的是同一道护栏。

2. `root` 回答 `no_root`，因为没有主题

一条飞书消息可以属于一个以更早消息为根的主题；而一次微信会话是一条平坦的流。把房间的第一条消息当作 "root" 返回会是一个凭空发明的锚点，而被告知"这就是根"的模型会从错误的消息出发推理。所以 `/root` 与 `root` 这个线程部分回答 `no_root`，而 `parent` **只在桥报告了引用消息时**才回答那条被引用的消息； 否则回答 `no_parent`。技能也这么说，于是模型不会把缺失的根当作一次失败的读取。

3. 按平台消息 ID 检索；把回复映射到它的父消息

共享的 chat kind 已经把 ID 形状的词转成检索词，并把 `meta.parentId` 映射成触发 ID；微信这个 kind 两样 都没接。现在两样都接上了，`Store.read.search` 精确匹配 `meta.nativeId`，与它此前已匹配的飞书 `meta.messageId` 并列。因此粘贴一条消息 ID 就能找到那个触发，而一条被引用的回复会展现出与飞书相同的 那一段对话。

4. readPageSize 与一个微信技能

readPageSize（1–50，桥的上限）进入配置，使评估读取与 worker 资源用同样的页大小分页，与飞书的 readPageSize 一致。一个随附的 dsh-superagent-weixin 技能在每个会话里注册，与飞书和 GitHub 技能 并列；它描述这些资源、[image:] 标记，以及微信做不到的那两件事。

仍然不可能、并且如实说明而不是伪造的部分

- **没有消息深链。** sidecar 不暴露任何深链，所以 ChannelLink.url 为空，面板只报出来源。编造一个 weixin:// URL 并不能打开那条消息。
- **没有机器人列表。** 飞书能靠 sender_type 认出机器人发送者；微信没有这个概念，桥对所有人报 bot:false。一份"其消息可能成为候选的机器人"名单会是一个永远匹配不上的控件。
- **没有主题根**——见决定 2。

变成可能的部分，以及怎么做的

- **图片字节能到 worker。** 技能记录了一种下载方式：DSH_SUPERAGENT_WEIXIN_BRIDGE 与 DSH_SUPERAGENT_WEIXIN_TOKEN_FILE 被导出进 worker 的环境，然后用一次 curl/Invoke-WebRequest 打 GET /media/（百分号编码，bearer 令牌来自 0600 文件）写下一个文件，read_image 能打开它。这就是飞书的 "用 lark-cli 下载，再读它"，但不必附带一个 CLI，因为桥本身就是那个已鉴权的端点。/media 与 /send 串行，所以下载一次只进行一个，而 401 会被如实报出而不是被猜。
- **首见回填是存在的，现在被标记出来了。** sidecar 在重挂之后（或某个会话刚开始被监听时）已经把会话的最后几条

消息读回来并排队；那就是微信的 backfillChat，每个会话上限 CATCHUP_MAX_MESSAGES （20）条，每轮最多 CATCHUP_MAX_CHATS（3）个会话，因为每次读取都要打开客户端窗口。这些事件现在 带上 historical: true（契约 §3），而 pollChat 把它们交给评估时不带回执、不自动回复、也不发处理中提示： 旧话必须可读，但不应被回答。它不是一次完整的历史扫描——一个从未被监听过的会话没有队列可读——技能也是这么说的。

后果

- worker 能读取它工单的微信会话并分页，这正是飞书的资源语法所给予的。
- 现在随附两个渠道专属技能；Prompts.skills 无条件注册 weixin（它不需要机器人，与飞书不同），

而面板列出它的部分以便覆盖。

- 资源路由从实例配置（weixinBridge）构建桥，所以一次资源读取每页只花一次 sidecar 调用，并且从不启动桥。

- 测试覆盖引用/语法、每种读取形状及其状态（ok、no_room、not_found）、ThreadContext 的各个部分

（root/parent/thread）、检索词与回复映射。

ADR-012：让企业微信与飞书对齐

状态：已接受（2026-11-14）· 范围：`src/sources/chat/wecom/、**src/sources/wecom-watch.ts、src/app/resources.ts、 skills/dsh-superagent-wecom/**`

背景

飞书渠道能接收消息、携带图片、让 agent 读取原始消息（及其线程/主题）、把回复映射到它的父消息，并能按消息 ID 检索。企业微信已有 kind、XML 解析器、发送者与一个纯回调处理器，但**没有任何路由挂载它**，所以什么都进不来；它只携带文本；它没有 ThreadContext、没有 worker 读取、也没有图片处理。本 ADR 在后端能办到的地方补上缺口，其余部分如实说明。

决定

1. 插件自己挂载回调路由

`wecom-watch` 现在在 Host 的 web 服务器上注册一条 `/wecom/` 的公开 `prefix` 路由。它在来源注册表里查出该实例的配置，以 1 MB 上限读取请求体，调用既有的 `handleWebhook`（验签 → 解密 → 解析 → 按实例去重环），用 `wecomTriggerOf` 把每个事件转成触发，并交给该 kind 的 `SourceContext.receive`。回复是加密的 `success`（或在 URL 验证时是解密后的 `echostr`）。这条路由之所以公开，是因为企业微信对每次调用都签名；运维把应用的接收 URL 指向它。这是插件唯一一处对外提供不可信请求的地方，签名不对就产不出事件。

2. 图片通过 `MediaId` 传递，评估可以下载它们

一个 `image` 回调携带 `MediaId` 且没有文本。`parseXml` 现在返回带 `media: [{ id, kind: 'image' }]` 的这类事件；读取方与 ThreadContext 渲染 `[image:]`，而 ThreadContext 的 `image` 用发送者的 `media` 调用（`GET /cgi-bin/media/get`）下载字节，其 JSON 错误形状会在请求体被当作字节处理之前被识别出来。worker 看到标记；持有实例配置的评估则附上图片。文本与其它消息类型不变，语音/视频/文件/位置仍然不产事件——如实说明，而不是伪造。

3. 一个回调能回答与不能回答什么

一个回调就是一条消息：`thread` 就是那条消息，`root` 回答 `no_root`，`parent` 回答 `no_parent`，没有历史回填，也没有下一页。`wecom//message` 是 worker 的读取（一条消息）。检索把十进制 `MsgId` 转成触发检索词，`repliesTo` 存在但回调不携带被引用的 ID，所以它通常是空的。

在这个后端上仍然不可能的部分

- **没有历史。** 接收 API 是推送式回调；没有可用来分页会话的回读端点，所以没有回填，也没有 `/thread` 分页。
- **没有消息深链。** 不存在按消息的 URL；渠道链接为空，面板只报出来源。
- **没有会话成员名册。** 对自建应用来说，枚举一个群的成员需要客户联系相关 API 以及各企业不同的权限；把应用可见的用户等同于某个会话的成员会是错的，所以不提供同步。
- **worker 拿不到图片字节。** worker 有标记但没有企业密钥，所以它无法调用媒体 API；改为由评估下载图片。

后果

- 企业微信第一次变得可用：一个回调变成一个触发，而一条回复回到同一个会话或同一个人。
- 为验签新增了两个配置字段（`token`、内联 `encodingAesKey`），两者都是可选的，所以既有的已存储配置仍能解析；没有 `token` 时每个回调都会被拒绝，`facts` 与字段提示都这么说。
- 测试覆盖图片解析、触发地址（`chatid` 与 `touser`）、资源语法以及 ThreadContext 的各个部分。